

CS336 作业 1（基础）

从零构建 Transformer 语言模型

中文译本 · Version 26.0.3 · Spring 2026

说明：本译本完整翻译题目说明与课程文字，不包含任何题目答案。代码、命令、公式、变量名和链接尽量保持原样；文末附原始 PDF，供核对图表、数学排版及技术细节。

术语约定：token 译为“词元”，tokenizer 译为“分词器”，embedding 译为“嵌入”，logits 保留英文，perplexity 译为“困惑度”。

中文翻译 · 原文第 1 页

CS336 作业 1(基础):构建 Transformer LM

版本26.0.3

CS336工作人员

2026 年春季

1 作业概述

在此作业中,你将构建从头开始训练标准 Transformer 语言模型 (LM) 所需的所有组件并训练一些模型。

你将实现什么

1. 字节对编码 (BPE) 分词器(第 2 节)
2. Transformer语言模型(LM)(第3节)
3. 交叉熵损失函数和 AdamW 优化器(第 4 节)
4. 训练循环,支持序列化和加载模型以及优化器状态(第 5 节)

你将运行什么

1. 在 TinyStories 数据集上进行 BPE 分词器训练。
2. 在数据集上训练分词器编码,将其转换为整数 ID 序列。
3. 在 TinyStories 数据集上进行 Transformer LM 训练。
4. 使用经过训练的 Transformer LM 进行样本生成和困惑度评估。
5. 在 OpenWebText 上进行模型训练,并将你获得的困惑度提交到排行榜。

你可以使用什么

我们希望你能从头开始构建每个组件。特别是,你不得使用 `torch.nn`、`torch.nn.functional` 或 `torch.optim` 中的任何定义,但以下情况除外:

- `torch.nn.Parameter`
- `torch.nn` 中的容器类(例如, `Module`、`ModuleList`、`Sequential` 等)。¹
- `torch.optim.Optimizer` 基类

你可以使用任何其他 PyTorch 定义。如果你想使用某个函数或类,但不确定是否允许,请随时在 Slack 上提问。如有疑问,请考虑使用它是否会损害作业的“从头开始”的精神。

关于人工智能工具的声明

人工智能可以完全自主地解决作业的许多部分。这使得深入参与课程材料并从中学习变得更加困难。

允许使用人工智能工具来回答高级概念问题,或提供低级编程文档,例如函数签名和库 API。但是,人工智能工具不得用于执行任何任务的任何部分。这包括两种编码代理(例如,光标 Agents、Codex、Claude Code)和 AI 自动完成(例如 Cursor Tab、GitHub Copilot)。当使用 AI 代理,确保它使用提供的 AGENTS.md 文件。使用聊天机器人时还应包含提示。

¹完整列表请参见 pytorch.org/docs/stable/nn.html#containers。

中文翻译 · 原文第 2 页

我们强烈建议你在完成作业时在 IDE 中禁用 AI 自动完成(例如,Cursor Tab、GitHub Copilot)(尽管非 AI 自动完成,例如自动完成函数名称完全没问题)。以前的学生强调,禁用人工智能自动完成功能可以更轻松地深入学习材料。

有关完整的人工智能政策,请参阅本文档。

代码是什么样的

作业代码以及本文可在 GitHub 上找到:

github.com/stanford-cs336/assignment1-basics

请 git 克隆存储库。如果有任何更新,我们会通知你,以便你可以 git pull 获取最新信息。

1. `cs336_basics/*`:这是你编写代码的地方。请注意,这里没有代码——你可以这样做 无论你想要什么,从头开始!

2. `adapters.py`:你的代码必须具有一组功能。对于每一项功能 (例如,缩放点积注意力),填写其实现(例如, `run_scaled_dot_product_attention`)只需调用你的代码即可。注意:你对 `adapters.py` 的更改 不应包含任何实质性逻辑;这是胶水代码。

3. `test_*.py`:这包含你必须通过的所有测试(例如,`test_scaled_dot_product_attention`), 它将调用 `adapters.py` 中定义的挂钩 不要编辑测试文件。

如何提交

为了提交,运行 `make_submission.sh` 构建提交 zip 文件。如果你不想将大型数据文件或检查点包含在提交 zip 中,请确保将任何其他文件添加到脚本中的排除列表中。

你将向 Gradescope 提交以下文件:

- `writeup.pdf`:回答所有书面问题。请排版你的回复。
- `code.zip`:包含你编写的所有代码。

要提交到排行榜,请将 PR 提交至:

github.com/stanford-cs336/assignment1-basics-leaderboard

有关详细的提交说明,请参阅排行榜存储库中的 `README.md`。

从哪里获取数据集

本作业将使用两个预处理的数据集:TinyStories [R. Eldan 等人,2023] 和 OpenWebText [A. Gokaslan 等人,2019]。两个数据集都是单个大型纯文本文件。

如果你要在课堂上完成作业,你可以在计算指南中找到下载数据集的说明。

如果你在家中进行操作,则可以使用 `README.md` 中的命令下载这些文件。

低资源提示:Init

在整个课程的作业讲义中,我们将提供完成部分作业的建议 具有较少或没有 GPU 资源的分配。例如,我们有时会建议 缩小数据集或模型大小,或解释如何在 Mac 上运行训练代码

中文翻译 · 原文第 3 页

集成 GPU 或 CPU。你会在蓝色框中找到这些“资源匮乏的提示”(就像这个)。即使你是斯坦福大学的注册学生,可以使用课程机器,这些提示可能会对你有所帮助 迭代速度更快并节省时间,因此我们建议阅读它们!

资源不足提示:Apple Silicon 或 CPU 上的作业 1

借助 Staff 解决方案代码,我们可以训练 LM 在 Apple 上生成相当流畅的文本 配备 36 GB RAM 的 M4 Max 芯片,在 Metal GPU (MPS) 上不到 5 分钟,在大约 30 分钟内 使用CPU。如果这些话对你来说意义不大,别担心!只要知道如果你有一个 相当最新的笔记本电脑并且你的实现正确且高效,你将能够 训练一个小型 LM,生成简单且流利的儿童故事。

稍后在作业中,我们将解释如果你使用的是 CPU 或 MPS,则需要进行哪些更改。

2 字节对编码 (BPE) 分词器

在作业的第一部分中,我们将训练并实现字节级字节对编码(BPE)分词器[R.森里奇等人,2016; C. Wang 等人,2019]。特别是,我们将代表任意 (Unicode) 字符串作为字节序列,并在此字节序列上训练我们的 BPE 分词器。稍后,我们将使用此分词器将文本(字符串)编码为标记(整数序列)以进行语言建模。

2.1 统一码标准

Unicode 是一种将字符映射到整数代码点的文本编码标准。自 Unicode 17.0 起 (于 2025 年 9 月发布),该标准定义了 172 个脚本的 159,801 个字符。例如,字符 “s” 的代码点为 115(通常表示为 U+0073,其中 U+ 是常规前缀,0073 是十六进制的 115),字符 “牛” 的代码点是 29275。在 Python 中,你可以使用 `ord()` 函数将单个 Unicode 字符转换为其整数表示形式。`chr()` 函数将整数 Unicode 代码点转换为具有相应字符的字符串。

```
>>> ord('牛')
29275
>>> chr(29275)
'牛'
```

问题(unicode1):理解Unicode(1分)

(a) `chr(0)` 返回什么 Unicode 字符?

提交内容:一句话回应。

(b) 该字符的字符串表示形式 (`_repr_()`) 与其打印的有何不同 代表?

提交内容:一句话回应。

(c) 当该字符出现在文本中时会发生什么?尝试一下可能会有所帮助 在你的 Python 解释器中执行以下操作,看看它是否符合你的期望:

```
>>> chr(0)
>>> print(chr(0))
```

中文翻译 · 原文第 4 页

```
>>> "this is a test" + chr(0) + "string"
>>> print("this is a test" + chr(0) + "string")
```

提交内容:一句话回应。

2.2 统一码编码

虽然 Unicode 标准定义了从字符到代码点(整数)的映射,但直接在 Unicode 代码点上训练分词器是不切实际的,因为词汇量会太大 (大约 150K 项)且稀疏(因为许多字符非常罕见)。相反,我们将使用 Unicode 编码,它将 Unicode 字符转换为字节序列。Unicode 标准本身定义了三种编码:UTF-8、UTF-16 和 UTF-32,其中 UTF-8 是主要的编码 互联网(超过 98% 的网页)。

要将 Unicode 字符串编码为 UTF-8,我们可以使用 Python 中的`encode()` 函数。要访问 Python 字节对象的底层字节值,我们可以迭代它(例如,调用 `list()`)。最后,我们可以使用`decode()`函数将UTF-8字节字符串解码为Unicode字符串。

```
>>> test_string = "hello! こんにちは!"
>>> utf8_encoded = test_string.encode("utf-8")
>>> print(utf8_encoded) b'hello! \xe3\x81\x93\xe3\x82\x93\xe3\x81\xab\xe3\x81\xa1\xe3\x81\xaf!'
>>> print(type(utf8_encoded))
<类 "字节" >
>>> # Get the byte values for the encoded string (integers from 0 to 255).
>>> list(utf8_encoded)
[104, 101, 108, 108, 111, 33, 32, 227, 129, 147, 227, 130, 147, 227, 129, 171, 227, 129,
161, 227, 129, 175, 33]
>>> # One byte does not necessarily correspond to one Unicode character!
>>> print(len(test_string))
13
>>> print(len(utf8_encoded))
23
>>> print(utf8_encoded.decode("utf-8")) hello! こんにちは!
```

通过将 Unicode 代码点转换为字节序列(例如,通过 UTF-8 编码),我们实质上是采用代码点序列(具有 159,801 个有效值的 21 位整数)并将其转换为字节值序列(0 到 255 范围内的整数)。256 长度字节的词汇表更容易处理。当使用字节级标记化时,我们不需要担心词汇表外的标记,因为我们知道任何输入文本都可以表示为 0 到 255 之间的整数序列。

问题(unicode2):Unicode 编码(3 分)

(a) 更喜欢在 UTF-8 编码字节上训练我们的分词器而不是 UTF-16 还是 UTF-32?比较这些编码的输出可能会有所帮助 输入字符串。

提交内容:一到两句话的回应。

(b) 考虑以下(不正确的)函数,该函数旨在解码 UTF-8 字节字符串 转换为 Unicode 字符串。为什么这个函数不正确?提供输入字节的示例 产生错误结果的字符串。

中文翻译 · 原文第 5 页

```
def decode_utf8_bytes_to_str_wrong(bytestring: bytes):
    return "".join([bytes([b]).decode("utf-8") for b in bytestring])

>>> decode_utf8_bytes_to_str_wrong("hello".encode("utf-8"))
'你好'
```

提交内容:decode_utf8_bytes_to_str_wrong 的示例输入字节字符串 产生不正确的输出,并用一句话解释该函数为何不正确。

(c) 给出一个不解码为任何 Unicode 字符的两字节序列。

提交内容:一个示例,附有一句话解释。

2.3 子词标记化

虽然字节级标记化可以缓解单词级标记化器所面临的词汇外问题,但将文本标记化为字节会导致输入序列极长。这会减慢模型训练速度,因为在单词级语言模型中,包含 10 个单词的句子可能只有 10 个标记长,但也可能是 字符级模型中长度为 50 个或更多标记(取决于单词的长度)。处理这些较长的序列需要在模型的每个步骤进行更多的计算。此外,字节序列的语言建模很困难,因为较长的输入序列会在数据中产生长期依赖性。

子字标记化是字级标记化器和字节级标记化器之间的中间点。请注意,字节级分词器的词汇表有 256 个条目(字节值为 0 到 255)。子字分词器会牺牲更大的词汇量来更好地压缩输入字节序列。例如,如果字节序列 b'the' 经常出现在我们的原始文本训练数据中,则为其分配词汇表中的一个条目会将这个 3 标记序列减少为单个标记。

我们如何选择这些子词单元来添加到我们的词汇表中? R.森里奇等人。[3] 建议使用字节对编码(BPE;P. Gage [5]),这是一种压缩算法,它用单个新的未使用索引迭代地替换(“合并”)最频繁的字节对。请注意,该算法将子词标记添加到我们的词汇表中,以最大限度地压缩输入序列 - 如果一个词在我们的输入文本中出现足够多次,它将被表示为单个子词单元。

通过 BPE 构建词汇表的子词分词器通常称为 BPE 分词器。在本次作业中,我们将实现一个字节级 BPE 分词器,其中词汇项是字节或合并的字节序列,这在词汇表外处理和可管理的输入序列长度方面为我们提供了两全其美的优势。构建 BPE 分词器词汇表的过程称为 “训练” BPE 分词器。

2.4 BPE 分词器训练

BPE 分词器训练过程由三个主要步骤组成。

词汇初始化

分词器词汇表是从字节串标记到整数 ID 的一对一映射。由于我们正在训练字节级 BPE 分词器,因此我们的初始词汇表只是所有字节的集合。由于有 256 个可能的字节值,我们的初始词汇表大小为 256。

预标记化

一旦有了词汇表,原则上你就可以计算文本中字节相邻出现的频率,并从最常见的字节对开始合并它们。然而,这在计算上是相当昂贵的,因为每次合并时我们都必须对语料库进行一次完整的遍历。在

中文翻译·原文第 6 页

此外,直接合并语料库中的字节可能会导致标记仅在标点符号上有所不同(例如,狗!与狗。)。这些标记将获得完全不同的词元 ID,即使它们可能具有很高的语义相似性(因为它们仅标点符号不同)。

为了避免这种情况,我们对语料库进行预先标记。你可以将其视为对语料库的粗粒度标记化,帮助我们计算字符对出现的频率。例如,单词“text”可能是出现 10 次的预标记。在这种情况下,当我们计算字符“t”和“e”彼此相邻出现的频率时,我们会看到单词“text”有相邻的“t”和“e”,我们可以将它们的计数增加 10,而不用查看语料库。由于我们正在训练字节级 BPE 模型,因此每个预词元都表示为 UTF-8 字节序列。

R. Sennrich 等人最初的 BPE 实现。[3] 通过简单地分割空格来预标记(即 `s.split(" ")`)。这种方法仍然存在于基于 SentencePiece 的分词器中(例如 Llama 1 和 2 分词器)。

大多数现代分词器都使用基于正则表达式的预分词器,这是 GPT-2 的做法; A.雷德福等人。[6]。我们将使用原始正则表达式的稍微漂亮的形式,从 github.com/openai/tiktoken/pull/234/files 获取

```
>>> PAT = r'""(?:[sdmt]||[ve]|re)| ?\p{L}+| ?\p{N}+| ?(?:^\s\p{L}\p{N})+|\s+(?!\S)\s+'''
```

使用此预分词器以交互方式分割一些文本可能会很有用,以便更好地了解其行为:

```
>>> # requires `regex` package
>>> import regex as re
>>> re.findall(PAT, "some text that i'll pre-tokenize")
['一些'、'文本'、'那个'、'我'、'将'、'前'、'-'、'标记化']
```

但是,在代码中使用它时,你应该使用 `re.finditer` 以避免在构建从预标记到其计数的映射时存储预标记化单词。

计算 BPE 合并

现在我们已经将输入文本转换为预标记并将每个预标记表示为一系列 UTF-8 字节,我们可以计算 BPE 合并(即训练 BPE 分词器)。在较高级别上,BPE 算法迭代地计算每对字节并识别频率最高的对(“A”、“B”)。然后合并该最频繁对(“A”、“B”)的每次出现,即用新的标记“AB”替换。这个新的合并标记已添加到我们的词汇表中;因此,最终的词汇量为 BPE 训练是初始词汇量的大小(在我们的例子中为 256)加上训练期间执行的 BPE 合并操作的数量。为了提高 BPE 训练期间的效率,我们不考虑跨越预标记边界的对。2 计算合并时,通过优先选择字典顺序上较大的对来确定性地打破对频率的联系。例如,如果(“A”,“B”)、(“A”,“C”)、(“B”,“ZZ”)和(“BA”,“A”)都具有最高频率,我们将合并(“BA”,“A”):

```
>>> max([(("A", "B"), ("A", "C"), ("B", "ZZ"), ("BA", "A"))],
        ('BA', 'A'))
```

特殊词元

通常,一些字符串(例如 `<|endoftext|>`)用于对元数据(例如文档之间的边界)进行编码。在对文本进行编码时,通常需要将某些字符串视为“特殊标记”,这些标记应该

2请注意,最初的 BPE 配方是 R. Sennrich 等人。[3] 指定包含词尾标记。在训练字节级 BPE 模型时,我们不会添加词尾标记,因为所有字节(包括空格和标点符号)都包含在模型的词汇表中。由于我们明确表示空格和标点符号,学习到的 BPE 合并自然会反映这些单词边界。

中文翻译 · 原文第 7 页

永远不会被分割成多个词元(即,将始终保留为单个词元)。例如,序列结束字符串 `<|endoftext|>` 应始终保留为单个标记(即单个整数 ID),以便我们知道何时停止从语言模型生成。这些特殊的token必须被添加到词汇表中,因此它们有一个对应的固定token ID。

R. Sennrich 等人的算法 1。[3] 包含 BPE 分词器训练的低效实现(基本上遵循我们上面概述的步骤)。作为第一个练习,实现和测试此函数以检查你的理解可能会很有用。

示例(bpe_example):BPE训练示例

这是 R. Sennrich 等人的一个程式化示例。[3]。考虑一个由以下组成的语料库 以下文字

低低低低低 低 低 最宽 最宽 最新最新最新最新最新最新最新

并且词汇表有一个特殊的标记`<|endoftext|>`。

词汇

我们使用特殊标记 `<|endoftext|>` 和 256 字节值来初始化词汇表。

预标记化

为了简单起见并专注于合并过程,我们假设在此示例中预先 标记化只是在空白处进行分割。当我们预先标记并计数时,我们最终得到 频率表。

{最低:5,较低:2,最宽:3,最新:6}

将其表示为 `dict[tuple[bytes, ...], int]` 很方便,例如`{(l,o,w):5,...}`。注意事项 在 Python 中,即使是单个字节也是字节对象。Python 中没有字节类型来表示 单个字节,就像Python中没有char类型来表示单个字符一样。

合并

我们首先查看每个连续的字节对,并对它们所在的单词的频率进行求和。出现 `{lo: 7, ow: 7, we: 8, er: 2, wi: 3, id: 3, de: 3, es: 9, st: 9, ne: 6, ew: 6}`。对 `('e', 's')` 和 `('s', 't')` 是相连的,因此我们采用字典顺序上更大的对 `('s', 't')`。那时我们会 合并前标记,最终得到 `{(l,o,w): 5, (l,o,w,e,r): 2, (w,i,d,e,st): 3, (n, e, w, e, st):6}`。

在第二轮中,我们看到 `(e, st)` 是最常见的对(计数为 9),我们 将合并为 `{(l,o,w): 5, (l,o,w,e,r): 2, (w,i,d,est): 3, (n,e,w,est): 6}`。继续这个,我们最终得到的合并序列将是 `['s t', 'e st', 'o w', 'l ow', 'w est', 'n e', 'ne west', 'w i', 'wi d', 'wid est', 'low e', 'lowe r']`。

如果我们进行 6 次合并,我们就有 `['s t', 'e st', 'o w', 'l ow', 'w est', 'n e']` 和我们的 词汇元素将是 `[<|endoftext|>, [...256 BYTE CHARS], st, est, ow, low, west, ne]`。

有了这个词汇表和一组合并,“最新”一词将被标记为 `[ne, west]`。

中文翻译 · 原文第 8 页

2.5 尝试 BPE 分词器训练

让我们在 TinyStories 数据集上训练字节级 BPE 分词器。有关查找/下载数据集的说明,请参阅第 1 节。在开始之前,我们建议你查看一下 TinyStories 数据集,以了解数据中的内容。

并行预标记化

你会发现一个主要瓶颈是预标记化步骤。你可以通过使用内置库多重处理并行化代码来加速预标记化。具体来说,我们建议在预标记化的并行实现中,对语料库进行分块,同时确保块边界出现在特殊标记的开头。你可以逐字使用以下链接中的起始代码来获取块边界,然后你可以使用它来跨进程分配工作:

https://github.com/stanford-cs336/assignment1-basics/blob/main/cs336_basics/pretokenization_example.py

这种分块总是有效的,因为我们永远不想跨文档边界合并。出于分配的目的,你始终可以以这种方式进行拆分。不用担心收到不包含 `<|endoftext|>` 的非常大的语料库的边缘情况。

在预标记化之前删除特殊标记

在使用正则表达式模式(使用 `re.finditer`)运行预标记化之前,你应该从语料库(或块,如果使用并行实现)中删除所有特殊标记。确保对特殊标记进行拆分,以便在它们分隔的文本中不会发生合并。例如,如果你有一个像 `[Doc 1]<|endoftext|>[Doc 2]` 这样的语料库(或块),则应该在特殊标记 `<|endoftext|>` 上进行拆分,并分别对 `[Doc 1]` 和 `[Doc 2]` 进行预标记,以便不会跨文档边界发生合并。换句话说,特殊标记在训练期间定义硬分割边界,但它们本身不应该有助于合并计数。这可以使用 `re.split` 来完成 `"|".join(special_tokens)` 作为分隔符(谨慎使用 `re.escape`,因为 `|` 可能出现在特殊标记中)。测试 `test_train_bpe_special_tokens` 将对此进行测试。

优化合并步骤

在上面的程式化示例中,BPE 训练的简单实现速度很慢,因为对于每次合并,它都会迭代所有字节对以识别最频繁的对。然而,每次合并后唯一改变的对计数是与合并对重叠的对计数。因此,可以通过索引所有对的计数并增量更新这些计数来提高 BPE 训练速度,而不是显式迭代每对字节来计算对频率。尽管我们注意到 BPE 训练的合并部分在 Python 中不可并行化,但你可以通过此缓存过程获得显著的加速。

低资源提示:分析

你应该使用 `cProfile` 或 `py-spy` 等分析工具来识别你的瓶颈,落实,并重点优化。

低资源提示:“缩小规模”

我们建议你不要直接在完整的 TinyStories 数据集上训练分词器。首先训练一小部分数据:“调试数据集”。例如,你可以训练你的相反, TinyStories 验证集上有 22K 文档而不是 212M 的分词器。这说明了尽可能缩小规模以加速开发的总体策略:

中文翻译 · 原文第 9 页

例如,使用较小的数据集、较小的模型大小等。选择调试数据集的大小 或超参数配置需要仔细考虑:你希望调试集很大 足以具有与完整配置相同的瓶颈(以便你所做的优化 会泛化),但不会大到需要永远运行。

问题(train_bpe):BPE Tokenizer 训练(15 分)

提交内容:编写一个函数,在给定输入文本文件的路径的情况下,训练(字节级)BPE 分词器。你的 BPE 训练函数应该(至少)处理以下输入参数:

输入

input_path:str 包含 BPE 分词器训练数据的文本文件的路径。

vocab_size: int 定义最大最终词汇量的正整数(包括 初始字节词汇、合并产生的词汇项以及任何特殊标记)。

special_tokens: list[str] 要添加到词汇表的字符串列表。训练期间,对待 它们作为硬边界,防止跨跨度合并,但在以下情况下不包括它们: 计算合并统计数据。

你的 BPE 训练函数应返回生成的词汇表并合并:

输出

vocab: dict[int, bytes] 分词器词汇表,从 int 映射(词汇)到字节(词元字节)。

merges: list[tuple[bytes, bytes]] 训练生成的 BPE 合并列表。每个列表 item 是一个字节元组 (<token1>, <token2>),表示 <token1> 已与 <token2>。合并应按创建顺序排序。

要根据我们提供的测试来测试你的 BPE 培训功能,你首先需要实现 测试适配器位于 [adapters.run_train_bpe]。然后,运行 `uv run pytest test/train_bpe.py`。你的实现应该能够通过所有测试。可选(这可以 是大量的时间投入),你可以使用一些方法来实现训练方法的关键部分 系统语言,例如 C++(考虑 `cppyy` 或 `nanobind`)或 Rust(使用 `PyO3`)。如果你这样做 请注意哪些操作需要复制,哪些操作需要直接从 Python 内存中读取,以及 确保留下构建指令,或者确保它仅使用 `pyproject.toml` 进行构建。另请注意 大多数正则表达式引擎都没有很好地支持 GPT-2 正则表达式,并且在大多数正则表达式引擎中速度太慢 那样做。我们已经验证 `Oniguruma` 相当快并且支持负向前瞻,但是 如果有的话,Python 中的 `regex` 包甚至更快。

问题(train_bpe_tinystories):TinyStories 上的 BPE 培训(2 分)

(a) 使用最大词汇量在 TinyStories 数据集上训练字节级 BPE 分词器 大小为 10,000。确保将 TinyStories <|endoftext|> 特殊词元添加到 词汇。序列化生成的词汇表并合并到磁盘以供进一步检查。如何 训练花费了多少时间和记忆?词汇表中最长的标记是什么?是否有道理吗?

资源要求:≤30 分钟(无 GPU)、≤30 GB RAM

中文翻译 · 原文第 10 页

提示你应该能够在 2 分钟内使用多处理进行 BPE 训练。在预标记化期间以及以下两个事实:

(a) `<|endoftext|>` 词元分隔数据文件中的文档。

(b) 在应用 BPE 合并之前, `<|endoftext|>` 词元作为特殊情况进行处理。

提交内容:一到两句话的回应。

(b) 分析你的代码。分词器训练过程的哪一部分花费最多时间?

提交内容:一到两句话的回应。

接下来,我们将尝试在 OpenWebText 数据集上训练字节级 BPE 分词器。和以前一样,我们建议查看数据集以更好地理解其内容。

问题(train_bpe_expts_owt):OpenWebText 上的 BPE 培训(2 分)

(a) 使用最大词汇量在 OpenWebText 数据集上训练字节级 BPE 分词器。规模为 32,000。序列化生成的词汇表并合并到磁盘以供进一步检查。词汇表中最长的标记是什么?有道理吗?

资源要求:≤12 小时(无 GPU)、≤100 GB RAM

提交内容:一到两句话的回应。

(b) 比较和对比你在 TinyStories 上接受训练的分词器。打开网络文本。

提交内容:一到两句话的回应。

2.6 BPE Tokenizer:编码和解码

在作业的前一部分中,我们实现了一个函数,用于在输入文本上训练 BPE 分词器,以获得分词器词汇表和 BPE 合并列表。现在,我们将实现一个 BPE 分词器,该分词器加载提供的词汇表和合并列表,并使用它们对与标记之间的文本进行编码和解码。身份证。

2.6.1 编码文本

BPE 编码文本的过程反映了我们训练 BPE 词汇的方式。有几个主要步骤。

步骤 1:预标记化。我们首先对序列进行预标记,并将每个预标记表示为一个序列 UTF-8 字节,就像我们在 BPE 训练中所做的那样。我们将把每个预标记中的这些字节合并到词汇元素中,独立处理每个预标记(不跨预标记边界进行合并)。

步骤 2:应用合并。然后,我们获取期间创建的词汇元素合并序列 BPE 训练,并按照创建顺序将其应用到我们的预词元中。

示例(bpe_encoding):BPE编码示例

例如,假设我们的输入字符串是 'the cat ate',我们的词汇表是 {0: b' ', 1: b'a', 2: b'c', 3: b'e', 4: b'h', 5: b't', 6: b'th', 7: b'c', 8: b'a', 9: b'the', 10: b'at'},以及 我们学到的合并是 [(b't', b'h'), (b' ', b'c'), (b' ', b'a'), (b'th', b'e'), (b'a',

中文翻译 · 原文第 11 页

b't']。首先,我们的预分词器会将这个字符串拆分为 ['the', 'cat', 'ate']。然后,我们将 查看每个预词元并应用 BPE 合并。

第一个前标记 “the” 最初表示为 [b't', b'h', b'e']。看看我们的清单 合并时,我们将第一个适用的合并确定为 (b't', b'h'),并使用它来转换 将预标记放入 [b'th', b'e'] 中。然后,我们返回合并列表并确定下一个 适用的合并为 (b'th', b'e'),它将前词元转换为 [b'the']。最后,回顾合并列表,我们发现没有更多适用于该字符串的内容(因为 整个预词元已合并为单个词元),因此我们已完成 BPE 的应用 合并。对应的整数序列为[9]。

对剩余的前标记重复此过程,我们看到前标记 “cat” 是 应用 BPE 合并后表示为 [b' c', b'a', b't'],变成整数 序列 [7, 1, 5]。应用 BPE 合并后,最终的预词元 “ate” 是 [b'at', b'e'], 变成整数序列 [10, 3]。因此,对输入字符串进行编码的最终结果是 [9、7、1、5、10、3]。

特殊词元

在编码文本时,你的分词器应该能够正确处理用户定义的特殊标记 (在构建分词器时提供)。

内存注意事项

假设我们想要标记一个无法放入内存的大型文本文件。为了有效地标记这个大文件(或任何其他数据流),我们需要将其分解为可管理的块并依次处理每个块,以便内存复杂性是恒定的,而不是与文本大小呈线性关系。在此过程中,我们需要确保标记不会跨越块边界,否则我们将得到与在内存中标记整个序列的简单方法不同的标记化。

2.6.2 解码文本

要将整数词元 ID 序列解码回原始文本,我们可以简单地在词汇表(字节序列)中查找每个 ID 的对应条目,将它们连接在一起,然后将字节解码为 Unicode 字符串。请注意,不保证输入 ID 映射到有效的 Unicode 字符串 (因为用户可以输入任何整数 ID 序列)。如果输入词元 ID 未生成有效的 Unicode 字符串,则应使用官方 Unicode 替换字符 U+FFFD 替换格式错误的字节。3 bytes.decode 的错误参数控制如何处理 Unicode 解码错误,并且使用 error='replace' 将自动使用替换标记替换格式错误的字节。

问题(分词器):实现分词器(15 分)

提交内容:实现一个 Tokenizer 类,在给定词汇表和合并列表的情况下进行编码 将文本转换为整数 ID,并将整数 ID 解码为文本。你的分词器还应该支持用户- 提供特殊标记(如果词汇表中尚不存在,则将其附加到词汇表中)。我们 推荐以下接口:

3有关 Unicode 替换字符的更多信息,请参阅 [en.wikipedia.org/wiki/Specials_\(Unicode_block\)#Replacement_character](https://en.wikipedia.org/wiki/Specials_(Unicode_block)#Replacement_character)。

中文翻译 · 原文第 12 页

```
def __init__(self, vocab, merges, special_tokens=None) Construct a tokenizer from a given
    词汇表、合并列表和(可选)特殊标记列表。这个函数应该接受
    以下参数:
```

词汇:dict[int, bytes]

合并:列表[元组[字节,字节]]

special_tokens:列表[str] |无=无

def from_files(cls, vocab_filepath, merges_filepath, special_tokens=None) Class method 从序列化词汇表和合并列表(在 与 BPE 训练代码输出的格式相同)和(可选)特殊标记列表。此方法应接受以下附加参数:

vocab_filepath:str

merges_filepath:str

special_tokens:列表[str] |无=无

```
def encode(self, text: str) -> list[int] Encode an input text into a sequence of token IDs.
```

def encode_iterable(self, iterable: Iterable[str]) -> Iterator[int] Given an iterable of 字符串(例如,Python 文件句柄),返回一个延迟生成词元 ID 的生成器。这是 对我们无法直接加载的大文件进行内存高效标记化所需 记忆。

```
def decode(self, ids: list[int]) -> str Decode a sequence of token IDs into text.
```

要根据我们提供的测试来测试你的 Tokenizer,你首先需要实现测试 适配器位于 [adapters.get_tokenizer]。然后,运行uv run pytest 测试/test_tokenizer.py。你的 实现应该能够通过所有测试。

2.7 实验

问题(tokenizer_experiments):分词器实验(4分)

(a) 来自 TinyStories 和 OpenWebText 的示例 10 文档。使用你之前训练过的 TinyStories 和 OpenWebText 分词器(分别为 10K 和 32K 词汇量),将这些采样文档编码为整数 ID。每个分词器的压缩比是多少 (字节/词元)?

提交内容:一到两句话的回应。

(b) 如果使用 TinyStories 分词器对 OpenWebText 示例进行标记,会发生什么? 比较压缩比和/或定性描述发生的情况。

提交内容:一到两句话的回应。

(c) 估计分词器的吞吐量(例如,以字节/秒为单位)。需要多长时间才能 对 Pile 数据集(825GB 文本)进行标记?

提交内容:一到两句话的回应。

(d) 使用你的 TinyStories 和 OpenWebText 分词器,对各自的训练进行编码并 将开发数据集转换为整数词元 ID 序列。稍后我们将使用它来训练我们的

中文翻译 · 原文第 13 页

语言模型。我们建议将词元 ID 序列化为数据类型的 NumPy 数组 uint16。为什么 uint16 是一个合适的选择？

提交内容:一到两句话的回应。

3 Transformer语言模型架构

语言模型将整数词元 ID 的批处理序列(即形状为 `torch.Tensor`)作为输入 (`batch_size, sequence_length`),并返回词汇表上的(批量)归一化概率分布(即形状为(`batch_size, sequence_length, vocab_size`)的PyTorch张量),其中预测分布针对每个输入标记的下一个单词。在训练语言模型时,我们使用这些下一个单词预测来计算实际下一个单词和预测下一个单词之间的交叉熵损失。在推理期间从语言模型生成文本时,我们从最终时间步(即序列中的最后一项)获取预测的下一个单词分布,以生成序列中的下一个标记(例如,通过获取概率最高的标记、从分布中采样等),将生成的标记添加到输入序列中,然后重复。

在这部分作业中,你将从头开始构建此 Transformer 语言模型。我们将从模型的高级描述开始,然后逐步详细说明各个组件。

3.1 TransformerLM

给定一系列词元 ID,Transformer 语言模型使用输入嵌入将词元 ID 转换为密集向量,将嵌入的词元传递到 `num_layers` Transformer 块,然后应用学习的线性投影(“输出嵌入”或“LM head”)来生成预测的下一词元 logits。示意图见图 1。

图 1(Transformer 语言模型概览):输入 → 词元嵌入 → `num_layers` 个 Transformer 块 → 归一化 → 线性输出嵌入(LM 头) → softmax → 输出概率。

图 2(预归一化 Transformer 块):输入先经归一化和带 RoPE 的因果多头自注意力,再与残差相加;随后经归一化和逐位置前馈网络,并再次与残差相加得到输出。

本页含图表或算法版式; 请参阅文末原文对照附录中的同页, 以核对图形与公式布局。

中文翻译 · 原文第 14 页

词元嵌入

在第一步中,Transformer 将(批量)词元 ID 序列嵌入到包含词元身份信息的向量序列中(图 1 中的红色块)。

更具体地说,给定词元 ID 序列,Transformer 语言模型使用词元嵌入层来生成向量序列。每个嵌入层都接受一个形状整数的张量 (batch_size,sequence_length)并生成形状向量序列(batch_size,sequence_length,d_model)。

预归一化Transformer块

嵌入后,激活由几个结构相同的神经网络层进行处理。标准的纯解码器 Transformer 语言模型由 num_layers 相同的层(通常称为 Transformer “块”)组成。每个 Transformer 块接受形状 (batch_size、sequence_length、d_model) 的输入并返回形状 (batch_size、sequence_length、d_model) 的输出。每个块都会聚合序列中的信息(通过自注意力)并对其进行非线性转换(通过前馈层)。

在 num_layers Transformer 块之后,我们将采用最终的激活并将它们转换为词汇表的分布。

我们将实现“预归一化”Transformer 块(详见第 3.4 节),这还需要在最终 Transformer 块之后使用层归一化(详见下文),以确保其输出正确缩放。

在此归一化之后,我们将使用标准的学习线性变换将 Transformer 块的输出转换为预测的下一个词元 logits(例如,参见 A. Radford 等人 [7] 方程 2)。

3.2 备注:批处理、Einsum和高效计算

在整个 Transformer 中,我们将执行应用于许多批处理输入的不同计算。以下是一些示例:

- 批次的元素:我们对每个批次元素应用相同的 Transformer 前向操作。
- 序列长度:“位置方式”操作(如 RMSNorm 和前馈)的操作方式相同 在序列的每个位置上。
- 注意力头:注意力操作以“多头”的方式在注意力头上进行批处理 注意操作。

采用符合人体工程学的方式来充分利用 GPU 来执行此类操作非常有用,并且易于阅读和理解。许多 PyTorch 操作可以在张量开始时采用多余的“类批”维度,并在这些维度上有效地重复/广播操作。

例如,假设我们正在进行按位置的批量操作。我们有一个形状为 D 的“数据张量” (batch_size、sequence_length、d_model),我们想对形状为(d_model、d_model)的矩阵 A 进行批量向量矩阵乘法。在这种情况下,D @ A 将执行批量矩阵乘法,这是 PyTorch 中的高效原语,其中 (batch_size, sequence_length) 维度被批量处理。

因此,假设你的函数可能会被赋予额外的类似批处理的维度并将这些维度保留在 PyTorch 形状的开头是有帮助的。为了组织张量以便能够以这种方式进行批处理,可能需要使用许多查看、重塑和转置步骤来对它们进行整形。这可能有点麻烦,而且通常很难阅读代码的作用以及张量的形状。

中文翻译 · 原文第 15 页

更符合人体工程学的选择是在 `torch.einsum` 中使用 `einsum` 表示法,或者使用与框架无关的库,例如 `einops` 或 `einx`。两个关键操作是 `einsum`,它可以对输入张量的任意维度进行张量收缩,以及重新排列,它可以重新排序、连接和分割任意维度。事实证明,机器学习中的几乎所有操作都是维度杂乱和张量收缩与偶尔(通常是逐点)非线性函数的某种组合。这意味着使用 `einsum` 表示法时,许多代码可以更具可读性和灵活性。

我们强烈建议在课堂上学习和使用 `einsum` 表示法。以前没有接触过 `einsum` 表示法的学生应该使用 `einops`([文档在这里](#)),而已经熟悉 `einops` 的学生应该学习更通用的 `einx`([这里](#))。4 这两个包都已经安装在我们提供的环境中。

这里我们给出一些如何使用 `einsum` 表示法的示例。这些是 `einops` 文档的补充,你应该首先阅读。

示例(einstein_example1):与 `einops.einsum` 的批量矩阵乘法

```
import torch
from einops import rearrange, einsum

## Basic implementation
Y = D @ A.T
# Hard to tell the input and output shapes and what they mean.
# What shapes can D and A have, and do any of these have unexpected behavior?

## Einsum is self-documenting and robust
#      D      A      ->      Y
Y = einsum(D, A, "batch sequence d_in, d_out d_in -> batch sequence d_out")

## Or, a batched version where D can have any leading dimensions but A is constrained.
Y = einsum(D, A, "... d_in, d_out d_in -> ... d_out")
```

示例(einstein_example2):使用 `einops.rearrange` 广播操作

我们有一批图像,对于每张图像,我们希望根据以下条件生成 10 个变暗版本: 一些比例因子:

```
images = torch.randn(64, 128, 128, 3) # (batch, height, width, channel)
dim_by = torch.linspace(start=0.0, end=1.0, steps=10)

## Reshape and multiply
dim_value = rearrange(dim_by, "dim_value -> 1 dim_value 1 1 1")
images_rearr = rearrange(images, "b height width channel -> b 1 height width channel")
dimmed_images = images_rearr * dim_value

## Or in one go:
dimmed_images = einsum(
    图像, dim_by,
    "批次高度宽度通道, dim_value -> 批次 dim_value 高度宽度通道"
)
```

4值得注意的是,虽然 `einops` 拥有大量支持,但 `einx` 并未经过实战检验。如果你发现 `einx` 中存在任何限制或错误,你应该可以随意使用 `einops` 和一些更简单的 `PyTorch`。

示例(einstein_example3):使用 einops.rearrange 进行像素混合

假设我们有一批图像表示为形状张量(批次、高度、宽度、通道),我们想要对图像的所有像素执行线性变换,但这 每个通道的转型应该独立进行。我们的线性变换是 表示为形状为(高度*宽度,高度*宽度)的矩阵B。

```
channels_last = torch.randn(64, 32, 32, 3) # (batch, height, width, channel)
B = torch.randn(32*32, 32*32)

## Rearrange an image tensor for mixing across all pixels
channels_last_flat = channels_last.view(
    -1, channels_last.尺寸(1) * channels_last.尺寸(2), channels_last.尺寸(3)
)
channels_first_flat = channels_last_flat.transpose(1, 2)
channels_first_flat_transformed = channels_first_flat @ B.T
channels_last_flat_transformed = channels_first_flat_transformed.transpose(1, 2)
channels_last_transformed = channels_last_flat_transformed.view(*channels_last.shape)
```

相反,使用 einops:

```
height = width = 32 ## Rearrange replaces clunky torch view + transpose
channels_first = rearrange( channels_last, "批次高度宽度通道 -> 批次通道(高度宽度)" )
channels_first_transformed = einsum( channels_first,B, "批量通道 pixel_in、pixel_out pixel_in -> 批量通道 pixel_out" )
channels_last_transformed = rearrange( channels_first_transformed, "批量通道(高度宽度)->批量高度宽度通道" , height=height, width=width )
```

或者,如果你感到疯狂:使用 einx.dot(einx 相当于 einops.einsum)一次性完成所有操作

```
height = width = 32
channels_last_transformed = einx.dot(
    "批次row_in col_in通道,(row_out col_out)(row_in col_in)"
    "-> 批次 row_out col_out 通道",
    channels_last,B,
    col_in=width, col_out=width
)
```

这里的第一个实现可以通过在之前和之后添加注释来改进 指示输入和输出形状是什么,但这很笨重并且容易出现错误。与 einsum 表示法,文档就是实现!

Einsum 表示法可以处理任意输入批处理维度,而且还具有自记录的主要优点。代码中输入和输出张量的相关形状更加清晰

中文翻译 · 原文第 17 页

使用 einsum 表示法。对于其余张量,你可以考虑使用张量类型提示,例如使用 jaxtyping 库(并非特定于 JAX)。

我们将更多地讨论在作业 2 中使用 einsum 表示法对性能的影响,但现在知道它们几乎总是比替代方案更好!

3.2.1 数学符号和内存排序

许多机器学习论文在其表示法中使用行向量,这导致表示与 NumPy 和 PyTorch 中默认使用的行主内存排序很好地吻合。对于行向量,线性变换看起来像

$$y = xW^T, (1)$$

对于行主 $W \in \mathbb{R}^{d_{out} \times d_{in}}$ 和行向量 $x \in \mathbb{R}^{1 \times d_{in}}$ 。请注意,这允许我们通过增加 x 的最外层维度来批量输入,这意味着我们可以用向量输入 x 替换矩阵输入 $X \in \mathbb{R}^{batch \times d_{in}}$ 。

在线性代数中,通常更常见的是使用列向量,其中线性变换看起来像

$$y = Wx, (2)$$

给定行主 $W \in \mathbb{R}^{d_{out} \times d_{in}}$ 和列向量 $x \in \mathbb{R}^{d_{in}}$ 。要在此设置中对输入进行批处理, x 的批处理维度必须放在最后,因此 x 需要替换为矩阵 $X \in \mathbb{R}^{d_{in} \times batch}$ 批处理。

在这个作业中,我们将主要使用列向量来表示数学符号,因为数学通常遵循这种符号。你应该记住,如果你想使用普通矩阵乘法表示法,则必须应用带有转置的矩阵,如公式 1 中的行向量约定所示,因为 PyTorch 使用行优先内存排序。如果你使用 einsum 进行线性代数运算,只要正确标记轴,这应该不是问题。顺便说一句,值得注意的是,其他语言/线性代数软件包(如 Matlab、Julia 和 Fortran)都使用列优先内存排序,这意味着批处理维度排在最后,但 Python 和相关软件包采用了行优先排序的 C 标准。

3.3 基本构建模块:线性和嵌入模块

3.3.1 参数初始化

有效地训练神经网络通常需要仔细初始化模型参数——错误的初始化可能会导致不良行为,例如梯度消失或爆炸。预范数变换器对于初始化异常鲁棒,但它们仍然对训练速度和收敛产生重大影响。由于此作业已经很长,因此我们将保存作业 3 的详细信息,并为你提供一些适合大多数情况的近似初始化。现在,使用: • 线性权重: $N(\mu = 0, \sigma^2 = \frac{1}{d_{in} + d_{out}})$ 在 $[-3\sigma, 3\sigma]$ 处截断 • Embedding: $N(\mu = 0, \sigma^2 = 1)$ truncated at $[-3, 3]$

• RMSNorm: 1

你应该使用 `torch.nn.init.trunc_normal_` 来初始化截断的正常权重。

3.3.2 线性模块

一般来说,线性层是 Transformer 和神经网络的基本构建块。首先,你将实现自己的 Linear 类,该类继承自 `torch.nn.Module` 并执行线性变换:

中文翻译 · 原文第 18 页

$y = Wx$. (3)

请注意,在大多数现代法学硕士之后,我们不包括偏差术语。

问题(线性):实现线性模块(1 分)

提交内容:实现一个继承自 `torch.nn.Module` 的 `Linear` 类并执行线性 转变。你的实现应遵循 PyTorch 内置 `nn.Linear` 的接口 模块,除了没有偏差参数或参数。我们推荐以下内容 接口:

```
def __init__(self, in_features, out_features, device=None, dtype=None) Construct a linear  
转换模块。该函数应接受以下参数:
```

`in_features`:int 输入的最终尺寸

`out_features`:int 输出的最终尺寸

设备:`torch.device` | `None` = `None` 存储参数的设备

数据类型:`torch.dtype` | `None` = `None` 参数的数据类型

```
def forward(self, x: torch.Tensor) -> torch.Tensor Apply the linear transformation to the  
输入。
```

确保:

- 子类 `nn.Module`
- 调用超类构造函数
- 构造参数并将其存储为 `W`(不是将其放入 `nn.Parameter` 中的 `W^T`),)
- 当然,不要使用`nn.Linear`或`nn.functional.linear`

对于初始化,请使用上面的设置以及 `torch.nn.init.trunc_normal_` 初始化权重。

要测试你的线性模块,请在 `[adapters.run_linear]` 处实现测试适配器。适配器 应该将给定的权重加载到线性模块中。你可以使用 `Module.load_state_dict` 这个目的。然后,运行 `uv run pytest -k test_linear`。

3.3.3 嵌入模块

如上所述,Transformer 的第一层是嵌入层,它将整数词元 ID 映射到维度为 `d_model` 的向量空间。我们将实现一个继承自 `torch.nn.Module` 的自定义 `Embedding` 类(因此你不使用 `nn.Embedding`)。前向方法应通过使用形状为 `(batch_size, sequence_length)` 的词元 ID 的 `torch.LongTensor` 索引到形状为 `(vocab_size, d_model)` 的嵌入矩阵来为每个词元 ID 选择嵌入向量。

问题(嵌入):实现嵌入模块(1 分)

提交内容:实现继承自 `torch.nn.Module` 的 `Embedding` 类并执行 嵌入查找。你的实现应该遵循 PyTorch 内置的接口 `nn.Embedding` 模块。我们推荐使用以下接口:

中文翻译 · 原文第 19 页

```
def __init__(self, num_embeddings, embedding_dim, device=None, dtype=None) Construct an  
嵌入模块。该函数应接受以下参数:
```

num_embeddings:int 词汇表大小

embedding_dim:int 嵌入向量的维度,即 dmodel 设备:torch.device | None = None 存储参数的设备

数据类型:torch.dtype | None = None 参数的数据类型

```
def forward(self, token_ids: torch.Tensor) -> torch.Tensor Look up the embedding vectors  
对于给定的词元 ID。
```

确保:

- 子类 nn.Module
- 调用超类构造函数
- 将嵌入矩阵初始化为 nn.Parameter
- 存储嵌入矩阵,d_model 为最终维度
- 当然,不要使用nn.Embedding或nn.functional.embedding

再次使用上面的设置进行初始化,并使用 torch.nn.init.trunc_normal_ 初始化权重。

要测试你的实现,请在 [adapters.run_embedding] 实现测试适配器。然后,运行 `uv run pytest -k test_embedding`。

3.4 预归一化Transformer块

每个 Transformer 块都有两个子层:多头自注意力机制和逐位置前馈网络([A. Vaswani 等人,2017],第 3.1 节)。

在最初的 Transformer 论文中,模型在两个子层周围使用残差连接,然后进行层归一化。这种架构通常被称为“后归一化”Transformer,因为层归一化应用于子层输出。然而,各种工作发现,将层归一化从每个子层的输出移动到每个子层的输入(在最终 Transformer 块之后进行附加层归一化)提高了 Transformer 训练稳定性 [T. Q. Nguyen 等人,2019; R. Xiong 等人,2020] — 请参阅图 2,了解此“预归一化”Transformer 块的视觉表示。然后,每个 Transformer 块子层的输出通过残差连接添加到子层输入(A. Vaswani 等人[8],第 5.4 节)。预归一化的直觉是,从输入嵌入到 Transformer 的最终输出,有一个干净的“残差流”,没有任何规范化,这据称是为了改善梯度流。此次预归一化 Transformer 现在是当今语言模型中使用的标准(例如 GPT-3、LLaMA、PaLM 等),因此我们将实现此变体。我们将逐步介绍预归一化 Transformer 块的每个组件,并按顺序实现它们。

3.4.1 均方根层归一化

A. Vaswani 等人最初的 Transformer 实现。[8]使用层归一化 [J. L. Ba et al., 2016] 标准化激活。继 H. Touvron 等人之后。[12],我们将使用均方根层归一化(RMSNorm;B.Zhang 等人[13],方程 4)进行层归一化。给定一个激活向量 $a \in \mathbb{R}^{d_{\text{model}}}$,RMSNorm 将重新缩放每个激活 a_i ,如下所示: $\text{ai RMSNorm}(a_i) = \text{RMS}(a)g_i$, (4)

中文翻译 · 原文第 20 页

其中 $\text{RMS}(a) = \sqrt{\frac{1}{d_{\text{model}}} \sum a_i^2 + \epsilon}$. 这里 g_i 是一个可学习的“增益”参数(有 d_{model} 等 $i=1 \dots d_{\text{model}}$ 参数总计),而 ϵ 是一个超参数,通常固定为 $1e-5$ 。

你应该将输入向上转换为 `torch.float32`,以防止在对输入求平方时溢出。总的来说,你的前向方法应该如下所示:

```
in_dtype = x.dtype
x = x.to(torch.float32)

# Your code here performing RMSNorm
...
result = ...

# Return the result in the original dtype
return result.to(in_dtype)
```

问题(rmsnorm):均方根层归一化(1 分)

提交内容:将 `RMSNorm` 实现为 `torch.nn.Module`. 我们建议使用以下接口:

```
def __init__(self, d_model: int, eps: float = 1e-5, device=None, dtype=None) Construct the
RMSNorm 模块。该函数应接受以下参数:
```

`d_model`: int 模型的隐藏维度

`eps`: float = $1e-5$ 用于数值稳定性的 Epsilon 值

设备: `torch.device` | `None` = `None` 存储参数的设备

数据类型: `torch.dtype` | `None` = `None` 参数的数据类型

```
def forward(self, x: torch.Tensor) -> torch.Tensor Process an input tensor of shape
(batch_size, sequence_length, d_model)并返回相同形状的张量。
```

注意:请记住在执行标准化之前将输入向上转换为 `torch.float32` (然后向下转换为原始数据类型),如上所述。

要测试你的实现,请在 `[adapters.run_rmsnorm]` 处实现测试适配器。然后,运行 `uv run pytest -k test_rmsnorm`。

3.4.2 逐位置前馈网络

图 3:SiLU(又称 Swish)与 ReLU 激活函数的对比(图见原文对照附录)。

在最初的 Transformer 论文中(A. Vaswani 等人的第 3.3 节 [8]),Transformer 前馈网络由两个线性变换组成,它们之间有一个 ReLU 激活 ($\text{ReLU}(x) = \max(0, x)$)。在原始架构中,内部前馈层的维度通常是输入维度的 4 倍。

然而,与原始设计相比,现代语言模型往往包含两个主要变化:它们使用另一种激活函数并采用门控机制。具体来说,我们将实现 Llama 3 [A. Grattafiori 等人,2024] 和 Qwen 2.5 [A. Yang et al., 2024],它将 SiLU(通常称为 Swish)激活与称为门控线性单元(GLU)的门控机制相结合。我们还将省略线性层中有时使用的偏差项,遵循自 PaLM [A. Chowdhery 等人,2022] 和 LLaMA [H. Touvron 等人,2023]。

SiLU 或 Swish 激活函数 [D.亨德里克斯等人,2016; S. Elfwing et al., 2017]定义如下:

$$x \text{ SiLU}(x) = x \cdot \sigma(x) = (5) \frac{1}{1 + e^{-x}}$$

从图 3 中可以看出,SiLU 激活函数与 ReLU 激活函数类似,但在零处平滑。

门控线性单元 (GLU) 最初由 Y. N. Dauphin 等人定义。[19] 作为通过 sigmoid 函数和另一个线性变换的线性变换的逐元素乘积:

$$\text{GLU}(x, W1, W2) = \sigma(W1x) \odot W2x, (6)$$

其中 $\odot$ 表示逐元素乘法。建议门控线性单元 “通过为梯度提供线性路径,同时保留非线性功能,减少深层架构的梯度消失问题。”

本页含图表或算法版式; 请参阅文末原文对照附录中的同页, 以核对图形与公式布局。

中文翻译 · 原文第 22 页

将 SiLU/Swish 和 GLU 放在一起,我们得到 SwiGLU,我们将其用于前馈网络:

$$\text{FFN}(x) = \text{SwiGLU}(x, W_1, W_2, W_3) = W_2(\text{SiLU}(W_1x) \odot W_3x), \quad (7)$$

其中 $x \in \mathbb{R}^{d_{\text{model}}}$, $W_1, W_3 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$, $W_2 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$, 规范地, $d_{\text{ff}} = (8/3) d_{\text{model}}$. 对于具体实现,可以将其四舍五入到附近64 的倍数以提高硬件效率。

N. Shazeer [20] 首先提出将 SiLU/Swish 激活与 GLU 结合起来,并进行了实验,结果表明 SwiGLU 在语言建模任务上优于 ReLU 和 SiLU(无门控)等基线。在稍后的作业中,你将比较 SwiGLU 和 SiLU。尽管我们已经提到了这些组件的一些启发式论证(并且论文提供了更多支持证据),但最好保持实证视角:Shazeer 论文中现在著名的引用是

“我们没有解释为什么这些架构似乎有效;我们将它们的成功归因于 其他一切,皆为上天的仁慈。”

问题(positionwise_feedforward):实现逐位置前馈网络(2 点)

提交内容:实现 SwiGLU 前馈网络,由 SiLU 激活组成 函数和 GLU。

注意:在这种情况下,你应该在实现中随意使用 torch.sigmoid 为了数值稳定性。

你应该在实现中将 d_{ff} 设置为大约 $(8/3) \times d_{\text{model}}$,同时确保 内部前馈层的维数是 64 的倍数,以充分利用你的 硬件。要根据我们提供的测试来测试你的实现,你需要实现 测试适配器位于 [adapters.run_swiglu]。然后,运行 `uv run pytest -k test_swiglu` 来测试你的实现。

3.4.3 相对位置嵌入

为了将位置信息注入模型,我们将实现旋转位置嵌入 [J. Su et al., 2021],通常称为 RoPE。对于给定的查询词元 $q(i) = W_q x(i) \in \mathbb{R}^d$ 在词元位置 i ,我们将应用成对旋转矩阵 R_i , 给出 $q'(i) = R_i q(i) = R_i W_q x(i)$. 在这里, R_i 将旋转成对 对于 $k \in \{1, \dots, d/2\}$ 和一些嵌入元素 $q(i)_{2k-1:2k}$ 作为 $2d$ 向量,角度为 $\theta_{i,k} = \Theta(2k-2)/d_i$ 常数 Θ . 因此,我们可以将 R_i 视为块对角矩阵尺寸 $d \times d$, 带块 $R_{i,k}$ 适用于 $k \in \{1, \dots, d/2\}$, with

$$R_{i,k} = \begin{pmatrix} \cos(\theta_{i,k}) & \sin(\theta_{i,k}) \\ -\sin(\theta_{i,k}) & \cos(\theta_{i,k}) \end{pmatrix} \quad (8)$$

这样我们就得到了完整的旋转矩阵

中文翻译 · 原文第 23 页

$R_i 1 \ 0 \ 0 \ \dots \ 0 \ 0 \ R_i 2 \ 0 \ \dots \ 0 \ R_i = 0 \ 0 \ R_i 3 \ \dots \ 0 \ , \ (9) \ \vdots \ \vdots \ 0 \ 0 \ 0 \ \dots \ R_i \ (d/2)$

其中 0 代表 2×2 个零矩阵。虽然可以构造完整的 $d \times d$ 矩阵,但一个好的解决方案应该使用该矩阵的属性来更有效地实现转换。由于我们只关心给定序列中标记的相对旋转,因此我们可以跨层和不同批次重复使用为 $\cos(\theta_i, k)$ 和 $\sin(\theta_i, k)$ 计算的值。如果你想优化它,你可以使用所有层引用的单个 RoPE 模块,并且它可以具有在 init 期间使用 `self.register_buffer(persistent=False)` 创建的 2d 预先计算的 sin 和 cos 值缓冲区,而不是 `nn.Parameter` (因为我们不想学习这些固定的余弦和正弦值)。然后,我们对 $q(i)$ 执行完全相同的旋转过程,然后通过相应的 R_j 旋转 $k(j)$ 。请注意,该层没有可学习的参数。

问题(绳索):实现 RoPE(2 分)

提交内容:实现一个 `RotaryPositionalEmbedding` 类,将 RoPE 应用于输入 张量。

推荐使用以下接口:

```
def __init__(self, theta: float, d_k: int, max_seq_len: int, device=None) Construct the  
RoPE 模块并根据需要创建缓冲区。
```

theta:RoPE 的浮点 Θ 值

d_k:查询和键向量的 int 维度

max_seq_len: int 将输入的最大序列长度

设备:torch.device | None = None 存储缓冲区的设备

`def forward(self, x: torch.Tensor, token_positions: torch.Tensor) -> torch.Tensor` Process 形状为 $(..., \text{seq_len}, d_k)$ 的输入张量并返回相同形状的张量。注意事项 你应该容忍 x 具有任意数量的批量维度。你应该假设 词元位置是形状 $(..., \text{seq_len})$ 的张量,指定的词元位置 x 沿序列维度。

你应该使用词元位置来分割你的(可能是预先计算的)cos 和 sin 张量 序列维度。

要测试你的实现,请完成 `[adapters.run_rope]` 并确保它通过 `uv run pytest -k test_rope`。

3.4.4 缩放点积注意力

我们现在将实现缩放点积注意力,如 A. Vaswani 等人所述。[8](第 3.2.1 节)。作为初步步骤,注意力操作的定义将利用 softmax,该操作采用非归一化分数向量并将其转换为归一化分布:

$\exp(v_i) \text{softmax}(v)_i = (10) \sum_{j=1}^n \exp(v_j).$

中文翻译 · 原文第 24 页

请注意,对于较大值, $\exp(v_i)$ 可能会变为 inf (则 $\text{inf} - \text{inf} = \text{NaN}$)。我们可以通过注意到 softmax 操作对于向所有输入添加任何常量 c 不变来避免这种情况。我们可以利用此属性来实现数值稳定性 - 通常,我们将从 v_i 的所有元素中减去 v 的最大条目,从而使新的最大条目为 0。你现在将使用此技巧实现 softmax,以实现数值稳定性。

问题(softmax):实现softmax(1分)

提交内容:编写一个函数以对张量应用 softmax 运算。你的函数应该 采用两个参数:张量和维度 i ,并将 softmax 应用于 i -th 维度 输入张量。输出张量应与输入张量具有相同的形状,但其 i -th 维度现在将具有归一化的概率分布。使用减法的技巧 i -th 维度的所有元素中 i -th 维度中的最大值,以避免数值 稳定性问题。

要测试你的实现,请完成 [adapters.run_softmax] 并确保它通过 `uv run pytest -k test_softmax_matches_pytorch`。

我们现在可以用数学方式定义注意力操作,如下所示:

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) V \quad (11)$$

其中 $Q \in \mathbb{R}^{n \times d_k}$, $K \in \mathbb{R}^{m \times d_k}$, 和 $V \in \mathbb{R}^{m \times d_v}$ 。这里, Q , K 和 V 都是此操作的输入 - 请注意,这些不是可学习的参数。

屏蔽:有时屏蔽注意力操作的输出很方便。掩码的形状应为 $M \in \{\text{True}, \text{False}\}^{n \times m}$,并且该布尔矩阵的每一行 i 指示查询 i 应关注哪些键。按照规范(有点令人困惑),位置 (i, j) 处的 True 值表示查询 i 确实关注键 j ,而 False 值表示查询不关注该键。换句话说, (i, j) 处的 “信息流” 与值 True 配对。例如,考虑一个 1×3 掩码矩阵,其条目为 $[[\text{True}, \text{True}, \text{False}]]$ 。单个查询向量仅关注前两个键。

在计算上,使用掩码比计算子序列的注意力要高效得多,我们可以通过获取 pre-softmax 值 $(QK^T / \sqrt{d_k})$ 并将 $-\infty$ 添加到掩码矩阵中任何为 False 的条目来实现这一点。

问题(scaled_dot_product_attention):实现缩放点积注意力(5 点)

提交内容:实现缩放点积注意力函数。你的实现应该 处理形状的键和查询 $(\text{batch_size}, \dots, \text{seq_len}, d_k)$ 和形状的值 $(\text{batch_size}, \dots, \text{seq_len}, d_v)$, 其中 $\dots$ 代表任意数量的其他批处理 尺寸(如果提供)。该实现应返回形状为 $(\text{batch_size}, \dots, \text{seq_len}, d_v)$ 。有关批量维度的讨论请参见第 3.2 节。

你的实现还应该支持可选的用户提供的布尔形状掩码 $(\text{seq_len}, \text{seq_len})$ 。掩码值为 True 的位置的注意概率应为 总和为 1,掩码值为 False 的位置的注意力概率 应该为零。

中文翻译 · 原文第 25 页

要根据我们提供的测试来测试你的实现,你需要实现测试适配器位于 [adapters.run_scaled_dot_product_attention]。uv run pytest -k test_scaled_dot_product_attention 在三阶输入张量上测试你的实现,同时 uv run pytest -k test_4d_scaled_dot_product_attention 在第四个测试你的实现 - 对输入张量进行排序。

3.4.5 因果多头自注意力

我们将实现多头自注意力,如 A. Vaswani 等人的第 3.2.2 节中所述。[8]。回想一下,在数学上,应用多头注意力的操作定义如下:

$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)$ (12)

for $\text{head}_i = \text{Attention}(Q_i, K_i, V_i)$ (13)

其中 Q_i, K_i, V_i 是切片编号 $i \in \{1, \dots, h\}$ 的大小为 d_k 或 d_v 的嵌入维度 K_i 和 V_i 分别为 V 。Attention 是第 3.4.4 节中定义的缩放点积注意力运算。由此我们可以形成多头自注意力操作:

$\text{MultiHeadSelfAttention}(x) = \text{WOMultiHead}(WQx, WKx, WVx)$ (14)

这里,可学习的参数是 $WQ \in \mathbb{R}^{d_k \times d_{\text{model}}}$, $WK \in \mathbb{R}^{d_k \times d_{\text{model}}}$, $WV \in \mathbb{R}^{d_v \times d_{\text{model}}}$, 和 $WO \in \mathbb{R}^{d_{\text{model}} \times h d_v}$ 。由于 Q_s 、 K_s 和 V_s 在多头注意力操作中被切片,我们可以认为 WQ , WK 和 WV 沿输出尺寸为每个头分开。当你完成此操作后,你应该在总共三个矩阵乘法中计算键、值和查询投影。5

因果掩蔽

你的实现应该防止模型关注序列中的未来标记。换句话说,如果给模型一个 token 序列 $t_1, \dots, t_n$, 并且我们想要计算前缀 $t_1, \dots, t_i$ 的下一个单词预测(其中 $i < n$), 模型应该无法访问(参与)位置 $t_{i+1}, \dots, t_n$ 的标记表示,因为在推理期间生成文本时它将无法访问这些标记(并且这些未来标记会泄露有关真实下一个单词的身份的信息,从而使语言建模预训练目标变得微不足道)。对于输入标记序列 $t_1, \dots, t_n$, 我们可以天真地阻止访问通过运行多头自注意力 n 次(对于序列中的 n 唯一前缀),我们将使用因果注意力掩码,这允许词元 i 关注序列中的所有位置 $j \leq i$ 或广播索引。比较来构造这个掩码,并且你应该利用第 3.4.4 节中的缩放点积注意力实现已经支持注意力掩码这一事实。

应用 RoPE

RoPE 应该应用于查询和键向量,而不是值向量。此外,头部维度应作为批量维度处理,因为在多头注意力中,注意力是独立应用于每个头部的。这意味着完全相同的 RoPE 旋转应该应用于每个头的查询向量和密钥向量。

问题(multihead_self_attention):实现因果多头自注意力(5分)

5作为延伸目标,尝试将键、查询和值投影组合到单个权重矩阵中,这样你只需要一个矩阵乘法。

中文翻译 · 原文第 26 页

提交内容:以 `torch.nn.Module` 的形式实现因果多头自注意力 实现应该(至少)接受以下参数:

`d_model`:Transformer 模块输入的 int 维数。

`num_heads`:int 多头自注意力中使用的头数。继 A. Vaswani 等人之后。[8],设置 `dk= dv= dmodelh`. 来根据我们的测试测试你的实现 提供测试,在 `[adapters.run_multihead_self_attention]` 实现测试适配器。然后,运行 `uv run pytest -k test_multihead_self_attention` 来测试你的实现。

3.5 完整的 Transformer LM

让我们从组装 Transformer 块开始(回顾图 2 会很有帮助)。一个 Transformer 块包含两个“子层”,一个用于多头自注意力,另一个用于 SwiGLU 前馈网络。在每个子层中,我们首先执行 RMSNorm,然后进行主要操作 (MHA/FF),最后添加剩余连接。

具体来说,Transformer 块的前半部分(第一个“子层”)应该实现以下一组更新,以从输入 x , 生成输出 y

$y = x + \text{MultiHeadSelfAttention}(\text{RMSNorm}(x))$. (15)

问题(transformer_block):实现 Transformer 块(3 分)

实现预归一化 Transformer 块,如第 3.4 节所述和图 2 所示。你的 Transformer 块应该(至少)接受以下参数。

`d_model`:Transformer 模块输入的 int 维数。

`num_heads`:int 多头自注意力中使用的头数。

`d_ff`:int 按位置前馈内层的维数。

要测试你的实现,请实现适配器 `[adapters.run_transformer_block]`。然后 运行 `uv run pytest -k test_transformer_block` 来测试你的实现。

提交内容:通过所提供测试的 Transformer 块代码。

现在,我们按照图 1 中的高级图将这些块放在一起。按照第 3.1.0.1 节中对嵌入的描述,将其输入到 `num_layers` Transformer 块中,然后将其传递到最终层范数和 LM 头中,以获得词汇表(logits)上的非标准化分布。

问题(transformer_lm):实现 Transformer LM(3 分)

是时候把它们放在一起了!实现 Transformer 语言模型,如中所述 第 3.1 节并如图 1 所示。至少,你的实现应该接受所有 上述 Transformer 块的构造参数,以及这些附加的 参数:

`vocab_size`:int 词汇表的大小,用于确定词汇的维数 词元嵌入矩阵。

中文翻译 · 原文第 27 页

`context_length`: int 最大上下文长度,用于确定维数 RoPE sin 和 cos 缓冲区。

`num_layers`:int 要使用的 Transformer 块的数量。

要根据我们提供的测试来测试你的实现,你首先需要实现测试适配器位于 `[adapters.run_transformer_lm]`。然后,运行 `uv run pytest -k test_transformer_lm` 到 测试你的实现。

提交内容:通过上述测试的 Transformer LM 模块。

资源核算

能够了解 Transformer 的各个部分如何消耗计算和内存是很有用的。我们将逐步完成一些基本的“失败次数统计”。绝大多数 Transformer 中的 FLOPS 是矩阵乘法,因此我们的核心方法很简单:

1. 写下 Transformer 前向传递中的所有矩阵乘法。
2. 将每个矩阵乘法转换为所需的 FLOP 数。

对于第二步,以下事实将很有用:

规则:给定 $A \in \mathbb{R}^{m \times n}$ 和 $B \in \mathbb{R}^{n \times p}$, 矩阵-矩阵乘积 AB 需要 $2mnp$ FLOP。

要看到这一点,请注意 $(AB)[i, j] = A[i, :] \cdot B[:, j]$, 并且该点积需要 n 加法和 n 乘法($2n$ 失败)。那么,由于矩阵-矩阵乘积 AB 有 $m \times p$ 条目,因此 FLOPS 总数为 $(2n)(mp) = 2mnp$ 。

现在,在做下一个问题之前,检查一下你的问题的每个组成部分会很有帮助。Transformer 块和 Transformer LM,并列出所有矩阵乘法及其关联失败成本。

问题(transformer_accounting):Transformer LM资源核算(5分)

(a) 考虑使用我们的分配架构的 GPT-2 XL 大小的模型,其具有以下内容 配置:

`vocab_size`:50,257

`context_length`:1,024

`num_layers`:48

`d_model`:1,600

`num_heads`:25 `d_ff`: 4,288 (64 到 $83 \times 1,600$ 最接近的倍数)

假设我们使用此配置构建模型。有多少个可训练参数 我们的模型会有吗?假设每个参数都使用单精度浮点数表示 问题是,加载这个模型需要多少内存?

提交内容:一到两句话的回应。

(b) 确定完成 GPT-2 XL 形状的前向传播所需的矩阵乘法 模型。这些矩阵乘法总共需要多少次 FLOP?假设我们的输入 序列具有 `context_length` 标记。

中文翻译 · 原文第 28 页

提交内容:矩阵乘法列表(带有描述)以及 FLOP 总数 需要。

(c) 根据上面的分析,模型的哪些部分需要最多的 FLOP 次数?

提交内容:一到两句话的回应。

(d) 使用 GPT-2 小型(12 层,768 d_model,12 个头)、GPT-2 中型重复分析 (24层,1024 d_model,16个头)和GPT-2大型(36层,1280 d_model,20个头)。作为 模型尺寸增加,Transformer LM 的哪些部分按比例占据更多或 总 FLOP 次数更少?

提交内容:对于每个模型,提供模型组件及其关联的详细信息 FLOP 数(向前传球所需的 FLOP 总数的比例)。另外,提供 用一到两句话描述改变模型大小如何改变比例 每个组件的失败次数。

(e) 采用 GPT-2 XL 并将上下文长度增加到 16,384。一次的总 FLOPs 是多少 前传球变?模型组件的 FLOP 相对贡献如何 改变?

提交内容:一到两句话的回应。

4 训练 Transformer LM

我们现在有了预处理数据(通过 tokenizer)和模型(Transformer)的步骤。剩下的就是构建所有代码来支持培训。这包括以下内容:

- 损失:我们需要定义损失函数(交叉熵)。
- 优化器:我们需要定义优化器来最小化这种损失(AdamW)。
- 训练循环:我们需要所有支持基础设施来加载数据、保存检查点和 管理培训。

4.1 交叉熵损失

回想一下,Transformer 语言模型定义了一个分布 $p_{\theta}(x_{i+1} | x_{1:i})$ 对于每个长度为 $m+1$ 的序列 x 和 $i = 1, \dots, m$. 给定由长度为 $m+1$ 的序列组成的训练集 D ,我们定义标准交叉熵(负对数似然)损失函数:

$$1/m \ell(\theta; D) = \sum_{x \in D} -\log p_{\theta}(x_{i+1} | x_{1:i}). \quad (16)$$

(请注意,对于所有 $i = 1, \dots, m$),Transformer 中的一次前向传递会生成 $p_{\theta}(x_{i+1} | x_{1:i})$ 特别是,Transformer 计算每个位置 i 的对数 $o_i \in \mathbb{R}^{\text{vocab_size}}$,结果为:

$$\exp(o_i[x_{i+1}]) / \sum_{a=1}^{\text{vocab_size}} \exp(o_i[a]) = p(x_{i+1} | x_{1:i}) = \text{softmax}(o_i)[x_{i+1}]. \quad (17)$$

交叉熵损失通常是相对于 logits $o_i \in \mathbb{R}^{\text{vocab_size}}$ 和目标向量来定义的 x_{i+1} .

6请注意, $o_i[k]$ 指的是向量 o_i 的索引 k 处的值 7这对应于 x_{i+1} 上的狄拉克 delta 分布与预测的 $\text{softmax}(o_i)$ 分布之间的交叉熵。

中文翻译 · 原文第 29 页

实现交叉熵损失需要注意数值问题,就像 Softmax 的情况一样。

问题(cross_entropy):实现交叉熵(1分)

提交内容:编写一个函数来计算交叉熵损失,该函数接受预测的 logits (oi) 和目标 (xi+1) 并计算交叉熵 $li = -\log \text{softmax}(oi)[xi+1]$. 你的函数 应处理以下事项:

- 减去最大元素以保证数值稳定性。
- 尽可能取消log 和exp。
- 处理任何额外的批次尺寸并返回批次的平均值。与 第 3.2 节,我们假设批量维度总是先出现在词汇量之前 维度。

实现 [adapters.run_cross_entropy], 然后运行 `uv run pytest -k test_cross_entropy` 测试你的实现。

困惑

交叉熵足以进行训练,但是当我们评估模型时,我们还想报告困惑度。对于长度为 m 的序列,我们遭受交叉熵损失 $l_1, \dots, l_m$:

$$\frac{1}{m} \sum_{i=1}^m l_i \quad \text{perplexity} = \exp\left(\frac{1}{m} \sum_{i=1}^m l_i\right). \quad (18)$$

4.2 SGD 优化器

现在我们有损失函数,我们将开始探索优化器。最简单的基于梯度的优化器是随机梯度下降(SGD)。我们从随机初始化的参数 θ_0 开始 然后对于每个步骤 $t = 0, \dots, T-1$, 我们执行以下更新:

$$\theta_{t+1} \leftarrow \theta_t - \alpha_t \text{grad } L(\theta_t; B_t), \quad (19)$$

其中 B_t 是从数据集 D , 中采样的随机批次数据以及学习率 α_t 和批次大小 $|B_t|$ 是超参数。

4.2.1 在 PyTorch 中实现 SGD

为了实现我们的优化器,我们将继承 PyTorch `torch.optim.Optimizer` 类。Optimizer 子类必须实现两个方法:

`def __init__(self, params, ...)` should initialize your optimizer. Here, `params` will be a collection of 要优化的参数(或参数组,以防用户想要使用不同的 超参数,例如模型不同部分的学习率)。确保将参数传递给 基类的 `__init__` 方法,它将存储这些参数以供步骤中使用。你可以采取 取决于优化器的附加参数(例如,学习率是常见的),并通过 它们作为字典传递给基类构造函数,其中键是你选择的名称(字符串) 这些参数。

`def step(self)` should make one update of the parameters. During the training loop, this will be called 在向后传递之后,因此你可以访问最后一批的梯度。这个方法应该 迭代每个参数张量 `p` 并就地修改它们,即设置 `p.data`,它保存

中文翻译 · 原文第 30 页

与基于梯度 $p.\text{grad}$ (如果存在) 的参数相关联的张量, 张量表示损失相对于该参数的梯度。

PyTorch 优化器 API 有一些微妙之处, 因此通过示例更容易解释。为了使我们的示例更丰富, 我们将实现 SGD 的轻微变化, 其中学习率在训练过程中衰减, 从初始学习率 α 开始, 并随着时间的推移逐渐采取更小的步长:

$$\alpha_{t+1} = \alpha_t - \sqrt{B_t} \quad (20) \quad t+1 \text{ grad } L(\theta_t;$$

让我们看看这个版本的 SGD 如何作为 PyTorch 优化器实现:

```
from collections.abc import Callable, Iterable
from typing import Optional
import torch
import math

class SGD(torch.optim.Optimizer):
    def __init__(self, params, lr=1e-3):
        if lr < 0:
            raise ValueError(f"无效的学习率:{lr}")
        defaults = {"lr": lr}
        super().__init__(params, defaults)

    def step(self, closure: Optional[Callable] = None):
        loss = None if closure is None else closure()
        for group in self.param_groups:
            lr = group["lr"] # Get the learning rate.
            for p in group["params"]:
                if p.grad is None:
                    继续

        state = self.state[p] # Get state associated with p.
        t = state.get("t", 0) # Get iteration number from the state, or 0.
        grad = p.grad.data # Get the gradient of loss with respect to p.
        p.data -= lr / math.sqrt(t + 1) * grad # 就地更新权重张量。
        state["t"] = t + 1 # 增加迭代次数。

    return loss
```

在 `__init__` 中, 我们将参数以及默认超参数传递给基类构造函数 (参数可能成组, 每个组都有不同的超参数)。如果参数只是 `torch.nn.Parameter` 对象的单个集合, 则基本构造函数将创建一个组并为其分配默认的超参数。然后, 在步骤中, 我们迭代每个参数组, 然后迭代该组中的每个参数, 并应用公式 20。在这里, 我们将迭代次数保留为与每个参数关联的状态: 我们首先读取该值, 在梯度更新中使用它, 然后更新它。API 指定用户可以传入可调闭包以在优化器步骤之前重新计算损失。我们将使用的优化器不需要这个, 但我们添加它是为了符合 API。

要查看其工作原理, 我们可以使用以下训练循环的最小示例:

```
weights = torch.nn.Parameter(5 * torch.randn((10, 10))) opt = SGD([weights], lr=1)

for t in range(100):
    opt.zero_grad() # 重置所有可学习参数的梯度。
    loss = (weights**2).mean() # Compute a scalar loss value.
    打印(loss.cpu().item())
```

中文翻译 · 原文第 31 页

`loss.backward()` # 运行向后传递,计算梯度。`opt.step()` # 运行优化器步骤。

这是训练循环的典型结构:在每次迭代中,我们将计算损失并运行优化器的一个步骤。当训练语言模型时,我们的可学习参数将来自模型(在 PyTorch, `m.parameters()` 为我们提供了这个集合)。损失将根据一批采样数据来计算,但训练循环的基本结构将是相同的。

问题(learning_rate_tuning):调整学习率(1分)

正如我们将看到的,对训练影响最大的超参数之一是学习率。让我们在玩具示例中实际看看这一点。使用其他三个值运行上面的 SGD 示例 学习率: $1e1$ 、 $1e2$ 和 $1e3$,仅 10 次训练迭代。损失会发生什么 对于每个学习率?它衰减得更快、更慢还是发散(即随着时间的推移而增加) 培训课程)?

提交内容:针对你观察到的行为进行一到两句话的回应。

4.3 AdamW

现代语言模型通常使用更复杂的优化器而不是 SGD 进行训练。最近使用的大多数优化器都是 Adam 优化器的衍生物 [D. P. Kingma 等人, 2015]。我们将使用 AdamW [I. Loshchilov et al., 2019],在最近的工作中广泛使用。AdamW 提出了对 Adam 的修改,通过添加权重衰减(在每次迭代中,我们将参数拉向 0)来改进正则化,以与梯度更新解耦的方式。我们将按照 I. Loshchilov 等人的算法 2 中所述来实现 AdamW。[23]。

AdamW 是有状态的:对于每个参数,它都会跟踪其第一和第二时刻的运行估计。因此,AdamW 使用额外的内存来换取稳定性和收敛性的提高。除了学习率 α , AdamW 还有一对超参数 (β_1 , β_2) 控制矩估计的更新,以及权重衰减率 λ 。典型应用将 (β_1 , β_2) 设置为 (0.9, 0.999),但像 LLaMA [H. Touvron 等人, 2023] 和 GPT-3 [T. B. Brown et al., 2020] 通常使用 (0.9, 0.95) 进行训练。该算法可以写如下,其中 ϵ 是一个小值(例如, 10^{-8}) 用于提高数值稳定性,以防我们在 v : 中得到极小的值

中文翻译 · 原文第 32 页

算法 1:AdamW 优化器

1. $\text{init}(\theta)$:初始化可学习参数;令一阶矩 $m \leftarrow 0$,二阶矩 $v \leftarrow 0$ 。
2. 对 $t = 1, \dots, T$:采样数据批次 B_t ,并计算梯度 $g \leftarrow \text{grad}_{\theta} l(\theta; B_t)$ 。
3. 计算经偏差修正的学习率 α_t ;依次应用权重衰减、更新一阶矩和二阶矩,再应用按矩调整的权重更新。

请注意, t 从 1 开始。你现在将实现此优化器。

问题(adamw):实现 AdamW(2 分)

提交内容:将 AdamW 优化器实现为 `torch.optim.Optimizer` 的子类。构造函数应接收学习率 α ,以及 β 、 ϵ 和 λ 等超参数。优化器基类提供 `self.state` 字典,可把 `nn.Parameter` 映射到保存该参数状态(对 AdamW 而言即矩估计)的字典。实现 `[adapters.get_adamw_cls]`,并确保通过 `uv run pytest -k test_adamw`。

问题(adamw_accounting):AdamW 训练的资源核算(2 分)

让我们计算一下运行 AdamW 需要多少内存和计算量。假设我们正在使用 每个张量都是 `float32`。

(a) 运行 AdamW 需要多少峰值内存?根据以下内容分解你的答案 参数、激活、梯度和优化器状态的内存使用情况。表达你的 根据 `batch_size` 和模型超参数(`vocab_size`, `context_length`, `num_layers`, `d_model`, `num_heads`)。假设 $d_{ff} = (8/3) \times d_{model}$ 。

为简单起见,在计算激活的内存使用量时,仅考虑以下内容 组件:

• Transformer块 • RMSNorm(s) • 多头自注意力子层:QKV 投影、 QK^T 矩阵乘法、softmax、值的加权和,输出投影。 • 位置前馈 (SwiGLU):门分支上的 W_1 , W_2 , SiLU,逐元素 产品, W_3

本页含图表或算法版式; 请参阅文末原文对照附录中的同页, 以核对图形与公式布局。

中文翻译 · 原文第 33 页

- 最终RMSNorm
- 输出嵌入
- logits 上的交叉熵

提交内容:每个参数、激活、梯度和代数表达式 优化器状态以及总数。

(b) 将你的答案实例化为 GPT-2 XL 形模型,以获得仅 取决于 `batch_size`。你可以使用且仍然适合的最大批量大小是多少 80GB内存?

提交内容:一个类似于 $a \cdot \text{batch_size} + b$ 的表达式,其数值为 a , b , 和 代表最大批量大小的数字。

(c) AdamW 运行一个步骤需要多少次 FLOP?

提交内容:代数表达式,并附有简短的论证。

(d) 模型 FLOP 利用率 (MFU) 定义为观察到的吞吐量(每个词元数)的比率 第二)相对于硬件的理论峰值 FLOP 吞吐量 [A. Chowdhery 等人,2022]。NVIDIA H100 GPU 的理论峰值为 495 teraFLOP/s 代表 “float32” (实际上是 TensorFloat-32,实际上是 “bfloat19”)操作。假设 你可以获得 50% MFU,训练 GPT-2 XL 400K 步需要多长时间 单个 H100 上的批量大小为 1024?继 J. Kaplan 等人之后。 [25] 和 J.霍夫曼等人。 [26],假设后向传递的 FLOP 是前向传递的两倍 通过。

提交内容:培训所需的小时数,并附有简短的理由。

4.4 学习率调度

导致损失最快减少的学习率值在训练过程中通常会发生变化。在训练 Transformer 时,通常使用学习率调度,我们从较大的学习率开始,一开始进行更快的更新,并随着模型训练慢慢将其衰减到较小的值。8 在本作业中,我们将实现用于训练 LLaMA 的余弦退火计划 [H. Touvron 等人,2023]。

调度程序只是一个函数,它采用当前步骤 t 和其他相关参数(例如初始和最终学习率),并返回用于步骤 t 的梯度更新的学习率。最简单的调度是常量函数,它将返回给定任何 t 的相同学习率

余弦退火学习率调度采用 (i) 当前迭代 t , (ii) 最大学习率 $\alpha_{\max}$, (iii) 最小(最终)学习率 $\alpha_{\min}$, (iv) 预热迭代次数 T_w , 和 (v) 余弦退火的最终迭代 T_c . 迭代时的学习率 α 定义为: (Warm-up) If $t < T_w$, then $\alpha = T_w \alpha_{\max} t$ (余弦退火)如果 $T_w \leq t \leq T_c$, 则 $\alpha = \alpha_{\min} + \frac{1}{2}(1 + \cos((T_c - T_w)\pi))(\alpha_{\max} - \alpha_{\min})$.

(Post-annealing) If $t > T_c$, then $\alpha = \alpha_{\min}$.

问题(learning_rate_schedule):通过预热实现余弦学习率调度 (1分)

8有时,通常会使用学习率回升(重新启动)的计划来帮助超越局部最小值。

中文翻译 · 原文第 34 页

编写一个函数,接受 t , $\alpha_{\max}$, $\alpha_{\min}$, T_w 和 T_c , 并根据以下公式返回学习率 α_t 上面定义的调度程序。然后实现 `[adapters.get_lr_cosine_schedule]` 并确保 它通过 `uv run pytest -k test_get_lr_cosine_schedule`。

4.5 梯度裁剪

在训练过程中,我们有时会遇到产生大梯度的训练示例,这可能会破坏训练的稳定性。为了缓解这种情况,实践中经常采用的一种技术是梯度裁剪。这个想法是在执行优化器步骤之前,在每次向后传递之后对梯度范数实现限制。

给定梯度(对于所有参数) g ,我们计算其 l_2 -范数 $\|g\|_2$ 。如果该范数小于 $M_{\max}$ 值为 M ,然后我们保留 g 不变;否则,我们将 g 缩小 $+\epsilon$ 倍(其中一个小的 $\|g\|_2 \epsilon$, 与 10^{-6} 一样,是为了数值稳定性而添加的)。请注意,所得范数将刚好低于 M 。

问题(gradient_clipping):实现梯度裁剪(1分)

编写一个实现梯度裁剪的函数。你的函数应该采用参数列表 和最大 l_2 -范数。它应该就地修改每个参数梯度。使用 $\epsilon = 10^{-6}$ (PyTorch 默认)。然后,实现适配器 `[adapters.run_gradient_clipping]` 并确保 它通过 `uv run pytest -k test_gradient_clipping`。

5 训练循环

现在,我们最终将把迄今为止构建的主要组件放在一起:标记化数据、模型和优化器。

5.1 数据加载器

标记化数据(例如,你在 `tokenizer_experiments` 中准备的)是单个标记序列 $x = (x_1, \dots, x_n)$ 。即使源数据可能由单独的文档(例如,不同的网页或源代码文件)组成,但常见的做法是将所有这些文件连接到单个词元序列中,并在它们之间添加分隔符(例如 `<|endoftext|>` 词元)。

数据加载器将其转换为批次流,其中每个批次由长度为 m 的 B 序列组成,与相应的下一个词元配对,长度也为 m 。例如,对于 $B = 1$ 、 $m = 3$, $([x_2, x_3, x_4], [x_3, x_4, x_5])$ would be one potential batch.

出于多种原因,以这种方式加载数据可以简化训练。首先,任何 $1 \leq i \leq n-m$ 都会给出有效的训练序列,因此对训练序列进行采样是微不足道的。由于所有训练序列具有相同的长度,因此不需要填充输入序列,这提高了硬件利用率(也可以通过增加批量大小 B)。最后,我们也不需要加载完整的数据集来采样训练数据,从而可以轻松处理可能无法放入内存的大型数据集。

问题(data_loading):实现数据加载(2分)

提交内容:编写一个函数,该函数采用 numpy 数组 x (带有词元 ID 的整数数组)、`batch_size`、`context_length` 和 PyTorch 设备字符串(例如 `"cpu"` 或 `"cuda:0"`),并返回 一对张量:采样的输入序列和相应的下一个标记目标。两者都

中文翻译 · 原文第 35 页

张量应具有包含词元 ID 的形状 (batch_size、context_length),并且两者都应 放置在所请求的设备上。要根据我们提供的测试来测试你的实现,你 首先需要在 [adapters.run_get_batch] 处实现测试适配器。然后,运行 `uv run pytest -k test_get_batch` 来测试你的实现。

低资源提示:在 CPU 或 Apple Silicon 上加载数据

如果你计划在 CPU 或 Apple Silicon 上训练 LM,则需要将你的 将数据传输到正确的设备(同样,你应该为你的型号使用相同的设备 稍后)。

如果你使用的是 CPU,则可以使用 “cpu” 设备字符串,而在 Apple Silicon(M* 芯片)上, 你可以使用 “mps” 设备字符串。

有关 MPS 的更多信息,请查看以下资源:

- <https://docs.pytorch.org/docs/stable/mps.html>
- <https://docs.pytorch.org/docs/stable/notes/mps.html>
- <https://developer.apple.com/documentation/metalperformanceshaders>

如果数据集太大而无法加载到内存中怎么办?我们可以使用名为 mmap 的 Unix 系统调用,它将磁盘上的文件映射到虚拟内存,并在访问该内存位置时延迟加载文件内容。因此,你可以 “假装” 内存中拥有整个数据集。Numpy 通过 `np.memmap` (或者标志 `mmap_mode='r'` 到 `np.load`,如果你最初使用 `np.save` 保存数组)来实现这一点,这将返回一个类似 numpy 数组的对象,该对象在你访问条目时按需加载条目。在训练期间从数据集(即 numpy 数组)进行采样时,请务必以内存映射模式加载数据集(通过 `np.memmap` 或标志 `mmap_mode='r'` 到 `np.load`,具体取决于保存数组的方式)。确保你还指定了与你正在加载的数组匹配的数据类型。显式验证内存映射数据看起来是否正确可能会有所帮助(例如,不包含超出预期词汇量大小的值)。

5.2 检查点

除了加载数据之外,我们还需要在训练时保存模型。在运行作业时,我们通常希望能够恢复中途停止的训练运行(例如,由于作业超时、机器故障等)。即使一切顺利,我们也可能希望稍后能够访问中间模型(例如,事后研究训练动态,从训练的不同阶段的模型中获取样本等)。

检查点应该具有我们恢复训练所需的所有状态。我们当然希望能够至少恢复模型权重。如果使用有状态优化器(例如 AdamW),我们还需要保存优化器的状态(例如,在 AdamW 的情况下,是矩估计)。最后,为了恢复学习率调度,我们需要知道我们停止的迭代次数。

PyTorch 可以轻松保存所有这些:每个 `nn.Module` 都有一个 `state_dict()` 方法,该方法返回包含所有可学习权重的字典;我们可以稍后使用姐妹方法 `load_state_dict()` 恢复这些权重。对于任何 `torch.optim.Optimizer`,也是如此。最后,`torch.save(obj, dest)` 可以将一个对象(例如,包含张量作为某些值的字典,也可以是整数等常规 Python 对象)转储到文件(路径)或类似文件的对象,然后可以使用 `torch.load(src)` 将其加载回内存。

问题(检查点):实现模型检查点(1 分)

实现以下两个函数来加载和保存检查点:

`def save_checkpoint(model, optimizer, iteration, out)` should dump all the state from the 模型、优化器和迭代到类似文件的对象中。你可以使用`state_dict` 模型和优化器获取其相关状态并使用的方法 `torch.save(obj, out)` 将 `obj` 转储到 `out` 中(PyTorch 支持路径或类似文件 对象在这里)。典型的选择是让 `obj` 成为字典,但你可以使用任何格式 只要你稍后可以加载检查点即可。

该函数需要以下参数:

型号:`torch.nn.Module`

优化器:`torch.optim.Optimizer`

迭代:整数

输出:str | `os.PathLike` |打字。BinaryIO |打字.IO[字节]

`def load_checkpoint(src, model, optimizer)` should load a checkpoint from `src` (path or file-like 对象),然后从该检查点恢复模型和优化器状态。你的职能 应该返回保存到检查点的迭代编号。你可以使用 `torch.load(src)` 恢复你在 `save_checkpoint` 实现中保存的内容,以及 模型和优化器中的 `load_state_dict` 方法将它们返回到之前的状态 州。

该函数需要以下参数:

源代码:str | `os.PathLike` |打字。BinaryIO |打字.IO[字节]

型号:`torch.nn.Module`

优化器:`torch.optim.Optimizer`

实现 `[adapters.run_save_checkpoint]` 和 `[adapters.run_load_checkpoint]` 适配器,并确保它们通过 `uv run pytest -k test_checkpointing`。

5.3 训练循环

现在,终于到了将你实现的所有组件放入主训练脚本中的时候了。使用不同的超参数(例如,将它们作为命令行参数)轻松开始训练运行将会有所回报,因为稍后你将多次执行这些操作来研究不同的选择如何影响训练。

问题(training_together):放在一起(4分)

提交内容:编写一个运行训练循环的脚本,根据用户提供的输入来训练模型。 特别是,我们建议你的训练脚本(至少)允许以下内容:

- 能够配置和控制各种模型和优化器超参数。
- 使用 `np.memmap`. 以内存高效方式加载大型训练和验证数据集
- 将检查点序列化到用户提供的路径。

- 定期记录培训和验证性能(例如,记录到控制台和/或外部 权重和偏差等服务).9

6 生成文本

现在我们可以训练模型了,我们最后需要的是从模型生成文本的能力。回想一下,语言模型接受长度为 `sequence_length` 的(可能是批处理的)整数序列,并生成大小为(`sequence_length`、`vocab_size`)的矩阵,其中序列的每个元素都是预测该位置之后的下一个标记的概率分布。我们现在将编写一些函数,将其转换为新序列的采样方案。

softmax

按照标准惯例,语言模型输出是最终线性层(“logits”)的输出,因此我们必须通过 softmax 运算将其转换为归一化概率,正如我们在前面的公式 10 中看到的那样。

解码

为了从我们的模型生成文本(解码),我们将为模型提供一系列前缀标记(“提示”),并要求它生成词汇表上的概率分布,以预测序列中的下一个标记。然后,我们将从词汇项的分布中进行采样,以确定下一个输出标记。

具体来说,解码过程的一个步骤应该采用序列 $x_1 \dots x_t$ 并通过以下等式返回词元 x_{t+1} ,

$$\exp(v_i) P(x_{t+1} = i | x_1 \dots x_t) = (21) \sum_j \exp(v_j) \quad v = \text{TransformerLM}(x_1 \dots x_t) t \in \text{Rvocab_size} \quad (22)$$

其中 TransformerLM 是我们的模型,它将长度为 `sequence_length` 的序列作为输入,并生成大小为 (`sequence_length`、`vocab_size`) 的矩阵,并且我们采用该矩阵的最后一个元素,因为我们正在寻找 t -th 位置的下一个标记预测。

这为我们提供了一个基本的解码器,通过从这些单步条件中重复采样(将之前生成的输出标记附加到下一个解码时间步的输入),直到生成序列结束标记 `<|endoftext|>` (或用户指定的要生成的最大标记数)。

解码器技巧

我们将尝试使用小型模型,小型模型有时会生成质量非常低的文本。两个简单的解码器技巧可以帮助解决这些问题。首先,在温度缩放中,我们使用温度参数 τ , 修改 softmax,其中新的 softmax 是

$$\exp(v_i/\tau) \text{softmax}(v, \tau)_i = \frac{\exp(v_i/\tau)}{\sum_{j=1}^{\text{vocab_size}} \exp(v_j/\tau)} \quad (23)$$

请注意设置 $\tau \rightarrow 0$ 是如何使得 v 的最大元素占主导地位,并且 softmax 的输出成为集中在该最大元素处的 one-hot 向量。

9wandb.ai

中文翻译 · 原文第 38 页

其次,另一个技巧是核心或 top-p 采样,我们通过截断低概率标记来修改采样分布。令 q 为我们从大小为 `vocab_size` 的(温度缩放的) softmax 获得的概率分布。使用超参数 p 进行核采样,根据方程生成下一个标记

q_i if $i \in V(p)$ (24) $P(x_{t+1} = i | q) = \{ \sum_{j \in V(p)} q_j \geq p \}$ 否则为 0

其中 $V(p)$ 是最小的索引集,使得 $\sum_{j \in V(p)} q_j \geq p$. 你可以通过首先按大小对概率分布 q 进行排序,然后选择最大的词汇元素,直到达到 p . 的目标级别来轻松计算该数量

问题(解码):解码(3分)

提交内容:实现一个从语言模型进行解码的函数。我们建议 你支持以下功能:

- 为用户提供的提示生成补全(即,接受一些 $x_1 \dots x_t$ 并采样 完成,直到你击中 `<|endoftext|>` 词元)。
- 允许用户控制生成词元的最大数量。
- 给定所需的温度值,将 softmax 温度缩放应用于预测的下一个温度值。 采样前的词元分布。
- Top-p 采样([A. Holtzman et al., 2020] 也称为核采样),给定用户 指定的阈值。

7 实验

现在是时候将所有内容放在一起并在预训练数据集上训练(小型)语言模型了。

7.1 如何运行实验和提交内容

了解 Transformer 架构组件背后的基本原理的最佳方法是实际修改它并亲自运行它。实践经验是无可替代的。

为此,能够快速、一致地进行实验并记录所做的事情非常重要。为了快速进行实验,我们将在小规模模型(约 17M 总参数)和简单数据集 (TinyStories) 上运行许多实验。为了保持一致,你将以系统的方式消除组件并改变超参数,并且为了保存记录,我们将要求你提交实验日志以及与每个实验相关的学习曲线。

为了能够提交损失曲线,请确保定期评估验证损失并记录步数和挂钟时间。你可能会发现权重和偏差等日志记录基础设施很有帮助。

问题(experiment_log):实验记录(3分)

对于你的培训和评估代码,创建实验跟踪基础设施,使你能够 跟踪你的实验和损失曲线的梯度步长和挂钟时间。

中文翻译 · 原文第 39 页

提交内容:记录实验的基础设施代码和实验日志(a 对于本节下面的作业问题,你尝试过的所有事情的文档。

7.2 小故事

我们将从一个非常简单的数据集(TinyStories;R. Eldan 等人[1])开始,其中模型将快速训练,并且我们可以看到一些有趣的行为。获取此数据集的说明位于 第 1 部分。该数据集的示例如下。

示例 (tinystories_example):TinyStories 中的一个示例

从前,有一个小男孩,名叫本。本喜欢探索周围的世界。他看到了许多令人惊奇的东西,比如商店里陈列的漂亮花瓶。有一天,本 当他穿过商店时,他发现了一个非常特别的花瓶。当本看到它时,他 惊讶!他说:“哇,这真是一个很棒的花瓶!我可以买它吗?”店主微笑着说道说:“当然可以。你可以把它带回家,让你所有的朋友看看它有多神奇!”所以 本把花瓶带回家,他为此感到非常自豪!他把他的朋友叫过来并向他们展示 令人惊叹的花瓶。他所有的朋友都觉得这个花瓶很漂亮,不敢相信它有多么幸运 本是。就这样,本在商店里发现了一个令人惊叹的花瓶!

7.2.1 超参数调整

我们将告诉你一些非常基本的超参数,并要求你找到一些运行良好的其他设置。

词汇量为 10000。典型的词汇量为数万到数十万。你应该改变这一点,看看词汇表和模型行为如何变化。

上下文长度 256。简单的数据集(例如 TinyStories)可能不需要长序列长度,但对于后面的 OpenWebText 数据,你可能需要改变这一点。尝试改变这个并查看对每次迭代运行时间和最终困惑度的影响。

d_model 512。这比许多小型 Transformer 论文中使用的 768 尺寸稍小,但这将使速度更快。

8 d_ff 1344。这大约是 3dmodel,同时是 64 的倍数,这有利于 GPU 性能。

RoPE θ 参数 Θ 10000。

层数和头数 4层,16头。总之,这将提供大约 17M 个非嵌入参数,这是一个相当小的 Transformer。

处理的词元总数为 327,680,000(你的批量大小 $\times$ 总步数 $\times$ 上下文长度应大致等于此值)。

你应该进行一些试验和错误,以找到以下其他超参数的良好默认值:学习率、学习率预热、其他 AdamW 超参数 (β_1 , β_2 , ϵ), 和权重衰减。你可以在 D. P. Kingma 等人的文章中找到此类超参数的一些典型选择。 [22]。

7.2.2 组合起来

现在,你可以通过获取经过训练的 BPE 分词器、对训练数据集进行分词并在你编写的训练循环中运行来将所有内容组合在一起。重要提示:如果你的实现正确且高效,则上述超参数应导致 1 B200 上的运行时间约为 20-30 分钟

中文翻译 · 原文第 40 页

图形处理器。如果你的运行时间更长,请检查并确保你的数据加载、检查点或验证丢失代码不会成为运行时间的瓶颈,并且你的实现已正确批处理。

7.2.3 调试模型架构的提示和技巧

我们强烈建议你熟悉 IDE 的内置调试器(例如 VSCode/Zed),与使用 `print` 语句进行调试相比,这将节省你的时间。如果你使用文本编辑器,则可以使用 `ipdb` 之类的工具。调试模型架构时的其他一些良好实践是:

- 开发任何神经网络架构时常见的第一步是过度拟合单个小批量。如果 如果你的实现是正确的,你应该能够快速将训练损失降至接近于零。
- 在各个模型组件中设置调试断点,并检查中间张量的形状以 确保它们符合你的期望。
- 监控激活、模型权重和梯度的规范,以确保它们不会爆炸或 消失。

问题(learning_rate):调整学习率(2 B200 小时)(3 分)

学习率是需要调整的最重要的超参数之一。采用基础模型 你已经接受过培训,请回答以下问题:

(a) 对学习率进行超参数扫描并报告最终损失(或注意 如果优化器发散,则发散)。

提交内容:与多个学习率相关的学习曲线。解释一下你的 超参数搜索策略。

提交内容:TinyStories 上验证损失(每个词元)最多 1.45 的模型

低资源提示:在 CPU 或 Apple Silicon 上进行几个步骤的训练

如果你在 `cpu` 或 `mps` 上运行,则应该减少处理的词元总数 数到 40,000,000,这足以产生相当流畅的文本。你 还可能将目标验证损失从 1.45 增加到 2.00。

在 M4 Max 芯片和 36 GB 上以调整的学习率运行我们的解决方案代码 RAM,我们使用批量大小 $\times$ 总步数 $\times$ 上下文长度 = $32 \times 5000 \times 256 = 40,960,000$ 个词元,cpu 上需要 1 小时 22 分钟,mps 上需要 36 分钟。 在第 5000 步,我们的验证损失为 1.80。

一些额外的提示:

- 使用N训练步骤时,我们建议调整余弦学习率衰减 安排在精确的时间终止其衰减(即达到最小学习率) 步骤 N。
- 使用 `mps` 时,不要使用 TF32 内核,即不要设置

```
torch.set_float32_matmul_precision('高')
```

就像使用 `cuda` 设备一样。我们尝试使用 `mps` (torch 版本 2.9.0)并发现后端有时会使用默默损坏的内核 导致训练不稳定。

中文翻译 · 原文第 41 页

- 你可以通过使用 `torch.compile`. JIT 编译模型来加快训练速度 具体来说:

- 在 `cpu` 上,使用以下命令编译模型

```
model = torch.compile(model)
```

- 在 `mps` 上,你可以使用以下命令在某种程度上优化向后传递

```
model = torch.compile(model, backend="aot_eager")
```

自 `torch` 版本 2.9.0 起,`mps` 不支持使用 `Inductor` 进行编译。

(b) 民间智慧认为,最佳学习率是“处于稳定边缘”。调查如何 学习率的分歧点与你的最佳学习率有关。

提交内容:增加学习率的学习曲线,其中至少包括一个发散的曲线 运行并分析这与收敛速度的关系。

现在让我们改变批量大小,看看训练会发生什么。批量大小很重要——它们让我们通过进行更大的矩阵乘法来从 `GPU` 获得更高的效率,但我们真的总是希望批量大小很大吗?让我们做一些实验来找出答案。

问题 (`batch_size_experiment`):批量大小变化(1 B200 小时)(1 分)

将批量大小从 1 一直更改到 `GPU` 内存限制。至少尝试几个批量大小 介于两者之间,包括 64 和 128 等典型尺寸。

提交内容:不同批量大小运行的学习曲线。学习率应该是 如有必要,再次优化。

提交内容:用几句话讨论你对批量大小的发现及其对 培训。

有了解码器,我们现在就可以生成文本了!我们将从模型中生成并看看它有多好。作为参考,你应该获得至少与下面的示例一样好的输出。

示例 (`ts_generate_example`):`TinyStories` 语言模型的示例输出

从前,有一个美丽的女孩,名叫莉莉。她喜欢吃口香糖,尤其是大的口香糖 黑色的。有一天,莉莉的妈妈让她帮忙做饭。莉莉太兴奋了!她喜欢 帮助她妈妈。莉莉妈妈做了一大锅汤当晚餐。莉莉很高兴地说:“谢谢 你,妈妈!我爱你。她帮妈妈把汤倒进一个大碗里。晚餐后,莉莉 妈妈做了一些美味的汤。莉莉喜欢它!她说:“谢谢你,妈妈!这汤太美味了。” 好吃!”她妈妈微笑着说:“我很高兴你喜欢它,莉莉。” 他们做完饭并 继续一起做饭。结束。

低资源提示:在 CPU 或 Apple Silicon 上生成文本

中文翻译 · 原文第 42 页

如果你使用处理了 40M 词元的低资源配置,你应该看到 几代人仍然像英语,但不像上面那么流利。例如,我们的样本 在 40M 词元上训练的 TinyStories 语言模型的输出如下:

从前,有一个小女孩,名叫苏。苏有一颗她非常喜欢的牙齿。它 是他最好的头脑。有一天,苏去散步,遇到了一只瓢虫!他们成了好朋友 并一起在路上玩耍。

“嘿,波莉!我们出去吧!” 蒂姆说。苏看了看天空,发现很难找到一个 跳舞的方式闪闪发光。她微笑着答应帮忙说话!

当苏看着天空移动时,那是什么。她

以下是准确的问题陈述以及我们的要求:

问题(生成):生成文本(1 分)

使用你的解码器和经过训练的检查点,报告模型生成的文本。你 可能需要操纵解码器参数(温度、top-p 等)以获得流畅的输出。

提交内容:至少 256 个文本标记的文本转储(或直到第一个 `<|endoftext|>` 标记), 并简要评论此输出的流畅性以及至少两个影响其良好程度的因素 或者这个输出很糟糕。

7.3 消融和架构修改

了解 Transformer 的最佳方法是实际修改它并查看它的行为方式。我们现在将进行一些简单的消融和修改。

消融 1:层归一化

人们常说层归一化对于 Transformer 训练的稳定性很重要。但也许我们想要危险地生活。让我们从每个 Transformer 块中删除 RMSNorm,看看会发生什么。

问题 (layer_norm_ablation):删除 RMSNorm 并训练(0.5 B200 小时)(1 分)

从 Transformer 和火车上移除所有 RMSNorm。之前发生了什么 最佳学习率?使用较低的学习率可以获得稳定性吗?

提交内容:删除 RMSNorms 并进行训练时的学习曲线以及学习过程 最佳学习率曲线。

提交内容:关于 RMSNorm 影响的几句话评论。

现在让我们研究另一个乍一看似乎任意的层归一化选择。预归一化 Transformer块定义为

$$z = x + \text{MultiHeadSelfAttention}(\text{RMSNorm}(x)) \quad (25)$$
$$y = z + \text{FFN}(\text{RMSNorm}(z)). \quad (26)$$

中文翻译 · 原文第 43 页

这是对原始 Transformer 架构的少数“共识”修改之一,该架构使用了后归一化方法

$z = \text{RMSNorm}(x + \text{MultiHeadSelfAttention}(x))$ (27)

$y = \text{RMSNorm}(z + \text{FFN}(z))$. (28)

让我们回到后归一化方法,看看会发生什么。

问题(pre_norm_ablation):实现后归一化并训练(0.5 B200 小时)(1 分)

将预归一化 Transformer 实现修改为后归一化 Transformer 实现。与后训练 规范模型,看看会发生什么。

提交内容:与预归一化相比,后归一化 Transformer 的学习曲线。

我们看到层归一化对 Transformer 的行为有重大影响,甚至层归一化的位置也很重要。

消融 2:位置嵌入

接下来我们将研究位置嵌入对模型性能的影响。具体来说,我们将比较我们的基本模型(与 RoPE)和完全不包含位置嵌入的情况(无PE)。事实证明,仅解码器的Transformer,即我们所实现的具有因果掩码的Transformer,理论上可以推断相对或绝对位置信息,而无需明确提供位置嵌入[Y.-H. H. Tsai 等人,2019; A. Kazemnejad 等人,2023]。我们现在将根据经验测试 NoPE 与 RoPE 相比的表现。

问题 (no_pos_emb):实现 NoPE(0.5 B200 小时)(1 分)

使用 RoPE 修改 Transformer 实现以删除位置嵌入 完整的信息,看看会发生什么。

提交内容:比较 RoPE 和 NoPE 性能的学习曲线。

消融 3:SwiGLU 与 SiLU

接下来,我们将遵循 N. Shazeer [20] 并通过比较 SwiGLU 前馈网络与使用 SiLU 激活但没有门控线性单元 (GLU) 的前馈网络的性能来测试前馈网络中门控的重要性:

$\text{FFNSiLU}(x) = W_2 \text{SiLU}(W_1 x)$. (29)

回想一下,在我们的 SwiGLU 实现中,我们将内部前馈层的维度设置为大致 $\text{dff} = (8/3) \text{dmodel}$ (同时确保 $\text{dff} \bmod 64 = 0$,以利用 GPU 张量核心)。在此消融基线中,你的 FFNSiLU 实现应改为设置 $\text{dff} = 4 \times \text{dmodel}$,以大致匹配默认 SwiGLU 前馈网络(具有三个而不是两个权重矩阵)的参数计数。

问题 (swiglu_ablation):SwiGLU 与 SiLU(0.5 B200 小时)(1 分)

中文翻译 · 原文第 44 页

提交内容:比较 SwiGLU 和 SiLU 前馈性能的学习曲线 网络,具有大致匹配的参数计数。

提交内容:用几句话讨论你的发现。

低资源提示:GPU 资源有限的在线学生应测试修改 关于小故事

在剩余的作业中,我们将转向更大规模、更嘈杂的网络数据集 (OpenWebText),尝试架构修改并(可选)制作 提交到课程排行榜。

训练 LM 流利地使用 OpenWebText 需要很长时间,因此我们建议在线 GPU 访问权限有限的学生继续在 TinyStories 上测试修改(使用验证 损失作为评估性能的指标)。

7.4 在 OpenWebText 上运行

我们现在将转向通过网络爬行创建的更标准的预训练数据集。一个小样本 OpenWebText [A. Gokaslan et al., 2019] 也以单个文本文件的形式提供:有关如何访问此文件的信息,请参阅第 1 节。

这是来自 OpenWebText 的示例。请注意文本如何更加现实、复杂和多样。 你可能想要查看训练数据集,以了解网络抓取语料库的训练数据是什么样的。

示例 (owt_example):OWT 的一个示例

棒球招股说明书技术总监哈里·帕夫利迪斯在聘请乔纳森时冒了风险 法官。

帕夫利迪斯知道,正如艾伦·施瓦茨 (Alan Schwarz) 在《数字游戏》中所写,“美国没有一个角落 与棒球表演相比,文化可以更精确地计算、更热情地量化 球员们。” 只需点击几下,你就可以发现诺亚·辛德加德 (Noah Syndergaard) 的快球 在到达本垒板的过程中每分钟旋转超过 2,100 次,纳尔逊·克鲁兹 (Nelson Cruz) 拥有 2016 年合格击球手中比赛的最高平均退出速度以及无数其他花絮 似乎是从电子游戏或科幻小说中抄袭来的。数据海洋的崛起赋予了 棒球文化中一个日益重要的角色:分析爱好者。

这种授权伴随着更多的审查——不仅是对衡量标准的审查,而且是对人员的审查 以及他们背后的出版物。帕夫利迪斯通过棒球招股说明书了解了所有关于强烈反对的事情 伴随着数量上的缺陷。他还知道该网站的捕获指标需要 重新设计,这需要一个博学的头脑——一个能够处理复杂统计的人 建模问题——完成工作。

“他把我们吓坏了。” 哈里·帕夫利迪斯

根据法官的写作和他们在一次会议上的互动,帕夫利迪斯有一种预感,法官“明白了”。 现场赞助的棒球活动。 [...]

中文翻译 · 原文第 45 页

注意:你可能需要重新调整超参数,例如实验的学习率或批量大小。

问题(main_experiment):OWT 实验(2 B200 小时)(2 分)

使用相同的模型架构和总体训练在 OpenWebText 上训练你的语言模型 迭代为 TinyStories。这个模型的表现如何?

提交内容:OpenWebText 上语言模型的学习曲线。描述一下区别 TinyStories 造成的损失——我们应该如何解释这些损失?

提交内容:从 OpenWebText LM 生成的文本,格式与 TinyStories 相同 输出。这段文字的流畅程度如何?为什么即使我们有,输出质量却很差 与 TinyStories 相同的模型和计算预算?

7.5 自己修改+排行榜

恭喜你达到了这一点。你快完成了!你现在将尝试改进 Transformer 架构,看看你的超参数和架构与班上其他学生相比如何。

排行榜规则

除以下情况外,没有任何限制:

运行时间:你提交的内容在 B200 上最多可以运行 45 分钟。你可能想要强制执行此操作 如果你使用 SLURM 或 Modal,则你的提交脚本。

数据:你只能使用我们提供的 OpenWebText 训练数据集。

否则,你可以自由地做任何你想做的事。

如果你正在寻找有关实现内容的一些想法,你可以查看其中一些资源:

- 最先进的开源LLM 系列,例如Llama 3 [A. Grattafiori 等人,2024] 或 Qwen 2.5 [A.杨等人,2024]。
- NanoGPT speedrun 存储库(github.com/KellerJordan/modded-nanogpt),其中社区 成员发布了对 “speedrunning” 小规模语言模型的许多有趣的修改 预训练。例如,可以追溯到原始 Transformer 论文的一个常见修改是 将输入和输出嵌入的权重联系在一起(参见 A. Vaswani 等人 [8](第 3.4 节) 和 A.Chowdhery 等人。 [16](第 2 节))。如果你尝试绑重,你可能需要减少 嵌入/LM 头初始化的标准偏差。

在尝试完整的 45 分钟运行之前,你需要在 OpenWebText 的一小部分或 TinyStories 上测试这些。

需要注意的是,我们确实注意到,你可能会发现在此排行榜中效果良好的一些修改可能无法推广到更大规模的预训练。我们将在课程的缩放定律单元中进一步探讨这个想法。

问题(排行榜):排行榜(10 B200 小时)(6 分)

中文翻译 · 原文第 46 页

你将根据上述排行榜规则训练模型,目标是最小化 语言模型的验证在 0.75 B200 小时内丢失。

提交内容:记录的最终验证损失,明确的相关学习曲线 显示少于 45 分钟的挂钟时间 x 轴,以及你所做操作的描述。我们 预计提交的排行榜至少会超过 5.0 损失的天真基线。提交至 排行榜在这里:github.com/stanford-cs336/assignment1-basics-leaderboard。

参考书目

- [1] R. Eldan 和 Y. Li, “TinyStories:语言模型可以有多小并且仍然能够表达连贯性 英语?” 2023 年。
- [2] A. Gokaslan、V. Cohen、E. Pavlick 和 S. Tellex, “OpenWebText 语料库” 。 2019.
- [3] R. Sennrich、B. Haddow 和 A. Birch, “使用 Subword 进行生僻词的神经机器翻译” 单位,” 在Proc. ACL,2016 年。
- [4] C. Wang、K. Cho 和 J. Gu, “具有字节级子词的神经机器翻译” 。 2019.
- [5] P. Gage, “一种新的数据压缩算法” ,C 用户期刊,卷. 12、没有. 2,第 23-38 页,2 月. 1994年。
- [6] A. Radford、J. Wu、R. Child、D. Luan、D. Amodei 和 I. Sutskever, “语言模型是 无监督的多任务学习者。” 2019.
- [7] A. Radford、K. Narasimhan、T. Salimans 和 I. Sutskever, “通过 生成预训练。” 2018.
- [8] A. Vaswani 等人, “注意力就是你所需要的” ,Proc. NeurIPS,2017 年。
- [9] T. Q. Nguyen 和 J. Salazar, 《没有眼泪的变形金刚:提高自我的正常化》 注意,” 在Proc. IWSWLT,2019 年。
- [10] R. Xiong 等人, “Transformer 架构中的层归一化” ,Proc.的ICML, 2020.
- [11] J. L. Ba、J. R. Kiros 和 G. E. Hinton, “层标准化” 。 2016年。
- [12] H. Touvron 等人, “LLaMA:开放且高效的基础语言模型。” 2023 年。
- [13] B.Zhang 和 R.Sennrich, “均方根层归一化” ,Proc. NeurIPS,2019 年。
- [14] A. Grattafiori 等人, “Llama 3 模型群体” 。 [在线的]。可用: <https://arxiv.org/abs/2407.21783>
- [15] A. Yang 等人, “Qwen2.5 技术报告” ,arXiv 预印本 arXiv:2412.15115,2024 年。
- [16] A. Chowdhery 等人, “PaLM:通过路径扩展语言建模” 。 2022 年。
- [17] D. Hendrycks 和 K. Gimpel, “用高斯桥接非线性和随机正则化器” 误差线性单位。” 2016年。
- [18] S. Elfving、E. Uchibe 和 K. Doya, “神经网络函数的 S 型加权线性单元” 强化学习中的近似。” [在线的]。可用: <https://arxiv.org/abs/1702.03118>
- [19] Y. N. Dauphin、A. Fan、M. Auli 和 D. Grangier, “门控卷积语言建模” 网络。” [在线的]。可用: <https://arxiv.org/abs/1612.08083>
- [20] N. Shazeer, “GLU 变体改进了 Transformer” 。 2020.

中文翻译 · 原文第 47 页

- [21] J. Su、Y. Lu、S. Pan、B. Wen 和 Y. Liu, “RoFormer:具有旋转位置的增强型Transformer 嵌入。” 2021 年。
- [22] D. P. Kingma 和 J. Ba, “Adam:随机优化方法”,Proc. ICLR,2015 年。
- [23] I. Loshchilov 和 F. Hutter, “解耦权重衰减正则化”,Proc. ICLR,2019 年。
- [24] T. B. Brown 等人, “语言模型是少样本学习者”,Proc. NeurIPS,2020 年。
- [25] J. Kaplan 等人, “神经语言模型的缩放定律”。 2020.
- [26] J. Hoffmann 等人, “训练计算最优大型语言模型”。 2022 年。
- [27] A. Holtzman、J. Buys、L. Du、M. Forbes 和 Y. Choi, “神经文本的奇怪案例 退化,” 在Proc. ICLR,2020 年。
- [28] Y.-H. H. Tsai、S. Bai、M. Yamada、L.-P. Morency 和 R. Salakhutdinov, “Transformer剖析: 通过内核的视角对Transformer 注意力的统一理解,《Proceedings of the the Proceedings of the 2019自然语言处理实证方法会议暨第九届国际会议自然语言处理联合会议 (EMNLP-IJCNLP),K. Inui、J. Jiang、V. Ng 和 X. Wan,编辑,中国香港:计算语言学协会,2019 年 11 月,第 4344 页– 4353.doi:10.18653/v1/D19-1443。
- [29] A. Kazemnejad、I. Padhi、K. Natesan、P. Das 和 S. Reddy, “位置编码对《变形金刚中的长度概括》,第三十七届神经信息会议 处理系统,2023。[在线]。 可用:<https://openreview.net/forum?id=Drrl2gcjzl>

CS336 Assignment 1 (basics): Building a Transformer LM

Version 26.0.3

CS336 Staff

Spring 2026

1 Assignment Overview

In this assignment, you will build all the components needed to train a standard Transformer language model (LM) from scratch and train some models.

What you will implement

1. Byte-pair encoding (BPE) tokenizer ([Section 2](#))
2. Transformer language model (LM) ([Section 3](#))
3. The cross-entropy loss function and the AdamW optimizer ([Section 4](#))
4. The training loop, with support for serializing and loading model and optimizer state ([Section 5](#))

What you will run

1. BPE tokenizer training on the TinyStories dataset.
2. Trained tokenizer encoding on the dataset to convert it into a sequence of integer IDs.
3. Transformer LM training on the TinyStories dataset.
4. Sample generation and evaluation of perplexity using the trained Transformer LM.
5. Model training on OpenWebText, and submit your attained perplexities to a leaderboard.

What you can use

We expect you to build each component from scratch. In particular, you may *not* use any definitions from `torch.nn`, `torch.nn.functional`, or `torch.optim` except for the following:

- `torch.nn.Parameter`
- Container classes in `torch.nn` (e.g., `Module`, `ModuleList`, `Sequential`, etc.).¹
- The `torch.optim.Optimizer` base class

You may use any other PyTorch definitions. If you would like to use a function or class and are not sure whether it is permitted, feel free to ask on Slack. When in doubt, consider if using it compromises the “from-scratch” ethos of the assignment.

Statement on AI tools

AI can solve many parts of the assignments fully autonomously. This makes it harder to deeply engage with, and learn from, the course material.

The use of AI tools is permitted for answering high-level conceptual questions, or for providing low-level programming documentation like function signatures and library APIs. However, AI tools are not permitted for implementing any part of any assignment. This includes both coding agents (e.g., Cursor Agents, Codex, Claude Code) and AI autocomplete (e.g., Cursor Tab, GitHub Copilot). When using an AI agent, make sure it uses the AGENTS.md file provided. The prompt should also be included when using chatbots.

¹See pytorch.org/docs/stable/nn.html#containers for a full list.

We strongly encourage you to disable AI autocomplete (e.g., Cursor Tab, GitHub Copilot) in your IDE when completing assignments (though non-AI autocomplete, e.g., autocompleting function names is totally fine). Previous students have highlighted that disabling AI autocomplete made it easier to engage deeply with the material.

For the full AI Policy, see [this document](#).

What the code looks like

The assignment code, as well as this writeup, are available on GitHub at:

github.com/stanford-cs336/assignment1-basics

Please `git clone` the repository. If there are any updates, we will notify you so you can `git pull` to get the latest.

1. `cs336_basics/*`: This is where you write your code. Note that there's no code in here—you can do whatever you want from scratch!
2. `adapters.py`: There is a set of functionality that your code must have. For each piece of functionality (e.g., scaled dot product attention), fill out its implementation (e.g., `run_scaled_dot_product_attention`) by simply invoking your code. Note: your changes to `adapters.py` should not contain any substantive logic; this is glue code.
3. `test_*.py`: This contains all the tests that you must pass (e.g., `test_scaled_dot_product_attention`), which will invoke the hooks defined in `adapters.py`. Don't edit the test files.

How to submit

In order to submit, run `make_submission.sh` to construct a submission zip file. Make sure to add any additional files to the list of exclusions in the script if you have large data files or checkpoints you don't want to include in your submission zip.

You will submit the following files to Gradescope:

- `writeup.pdf`: Answer all the written questions. Please typeset your responses.
- `code.zip`: Contains all the code you've written.

To submit to the leaderboard, submit a PR to:

github.com/stanford-cs336/assignment1-basics-leaderboard

See the `README.md` in the leaderboard repository for detailed submission instructions.

Where to get datasets

This assignment will use two pre-processed datasets: TinyStories [\[R. Eldan et al., 2023\]](#) and OpenWebText [\[A. Gokaslan et al., 2019\]](#). Both datasets are single, large plaintext files.

If you are doing the assignment with the class, you can find instructions for downloading the dataset in the compute guide.

If you are following along at home, you can download these files with the commands inside the `README.md`.

Low-Resource Tip: Init

Throughout the course's assignment handouts, we will give advice for working through parts of the assignment with fewer or no GPU resources. For example, we will sometimes suggest **downscaling** your dataset or model size, or explain how to run training code on a Mac

integrated GPU or CPU. You'll find these “low-resource tips” in a blue box (like this one). Even if you are an enrolled Stanford student with access to the course machines, these tips may help you iterate faster and save time, so we recommend reading them!

Low-Resource Tip: Assignment 1 on Apple Silicon or CPU

With the staff solution code, we can train an LM to generate reasonably fluent text on an Apple M4 Max chip with 36 GB RAM, in under 5 minutes on Metal GPU (MPS) and about 30 minutes using the CPU. If these words don't mean much to you, don't worry! Just know that if you have a reasonably up-to-date laptop and your implementation is correct and efficient, you will be able to train a small LM that generates simple children's stories with decent fluency.

Later in the assignment, we will explain what changes to make if you are on CPU or MPS.

2 Byte-Pair Encoding (BPE) Tokenizer

In the first part of the assignment, we will train and implement a byte-level byte-pair encoding (BPE) tokenizer [R. Sennrich et al., 2016; C. Wang et al., 2019]. In particular, we will represent arbitrary (Unicode) strings as a sequence of bytes and train our BPE tokenizer on this byte sequence. Later, we will use this tokenizer to encode text (a string) into tokens (a sequence of integers) for language modeling.

2.1 The Unicode Standard

Unicode is a text encoding standard that maps characters to integer *code points*. As of Unicode 17.0 (released in September 2025), the standard defines 159,801 characters across 172 scripts. For example, the character “s” has the code point 115 (typically notated as U+0073, where U+ is a conventional prefix and 0073 is 115 in hexadecimal), and the character “牛” has the code point 29275. In Python, you can use the `ord()` function to convert a single Unicode character into its integer representation. The `chr()` function converts an integer Unicode code point into a string with the corresponding character.

```
>>> ord('牛')
29275
>>> chr(29275)
'牛'
```

Problem (unicode1): Understanding Unicode (1 point)

- (a) What Unicode character does `chr(0)` return?

Deliverable: A one-sentence response.

- (b) How does this character's string representation (`__repr__()`) differ from its printed representation?

Deliverable: A one-sentence response.

- (c) What happens when this character occurs in text? It may be helpful to play around with the following in your Python interpreter and see if it matches your expectations:

```
>>> chr(0)
>>> print(chr(0))
```

```
>>> "this is a test" + chr(0) + "string"
>>> print("this is a test" + chr(0) + "string")
```

Deliverable: A one-sentence response.

2.2 Unicode Encodings

While the Unicode standard defines a mapping from characters to code points (integers), it's impractical to train tokenizers directly on Unicode code points, since the vocabulary would be prohibitively large (around 150K items) and sparse (since many characters are quite rare). Instead, we'll use a Unicode encoding, which converts a Unicode character into a sequence of bytes. The Unicode standard itself defines three encodings: UTF-8, UTF-16, and UTF-32, with UTF-8 being the dominant encoding for the Internet (more than 98% of all webpages).

To encode a Unicode string into UTF-8, we can use the `encode()` function in Python. To access the underlying byte values for a Python `bytes` object, we can iterate over it (e.g., call `list()`). Finally, we can use the `decode()` function to decode a UTF-8 byte string into a Unicode string.

```
>>> test_string = "hello! こんにちは!"
>>> utf8_encoded = test_string.encode("utf-8")
>>> print(utf8_encoded)
b'hello! \xe3\x81\x93\xe3\x82\x93\xe3\x81\xab\xe3\x81\xa1\xe3\x81\xaf!'
>>> print(type(utf8_encoded))
<class 'bytes'>
>>> # Get the byte values for the encoded string (integers from 0 to 255).
>>> list(utf8_encoded)
[104, 101, 108, 108, 111, 33, 32, 227, 129, 147, 227, 130, 147, 227, 129, 171, 227, 129,
161, 227, 129, 175, 33]
>>> # One byte does not necessarily correspond to one Unicode character!
>>> print(len(test_string))
13
>>> print(len(utf8_encoded))
23
>>> print(utf8_encoded.decode("utf-8"))
hello! こんにちは!
```

By converting our Unicode code points into a sequence of bytes (e.g., via the UTF-8 encoding), we are essentially taking a sequence of code points (21-bit integers with 159,801 valid values) and transforming it into a sequence of byte values (integers in the range 0 to 255). The 256-length byte vocabulary is *much* more manageable to deal with. When using byte-level tokenization, we do not need to worry about out-of-vocabulary tokens, since we know that *any* input text can be expressed as a sequence of integers from 0 to 255.

Problem (unicode2): Unicode Encodings (3 points)

- (a) What are some reasons to prefer training our tokenizer on UTF-8 encoded bytes, rather than UTF-16 or UTF-32? It may be helpful to compare the output of these encodings for various input strings.

Deliverable: A one-to-two sentence response.

- (b) Consider the following (incorrect) function, which is intended to decode a UTF-8 byte string into a Unicode string. Why is this function incorrect? Provide an example of an input byte string that yields incorrect results.

```
def decode_utf8_bytes_to_str_wrong(bytestring: bytes):
    return "".join([bytes([b]).decode("utf-8") for b in bytestring])

>>> decode_utf8_bytes_to_str_wrong("hello".encode("utf-8"))
'hello'
```

Deliverable: An example input byte string for which `decode_utf8_bytes_to_str_wrong` produces incorrect output, with a one-sentence explanation of why the function is incorrect.

(c) Give a two-byte sequence that does not decode to any Unicode character(s).

Deliverable: An example, with a one-sentence explanation.

2.3 Subword Tokenization

While byte-level tokenization can alleviate the out-of-vocabulary issues faced by word-level tokenizers, tokenizing text into bytes results in extremely long input sequences. This slows down model training, since a sentence with 10 words might only be 10 tokens long in a word-level language model, but could be 50 or more tokens long in a character-level model (depending on the length of the words). Processing these longer sequences requires more computation at each step of the model. Furthermore, language modeling on byte sequences is difficult because the longer input sequences create long-term dependencies in the data.

Subword tokenization is a midpoint between word-level tokenizers and byte-level tokenizers. Note that a byte-level tokenizer’s vocabulary has 256 entries (byte values are 0 to 255). A subword tokenizer trades off a larger vocabulary size for better compression of the input byte sequence. For example, if the byte sequence `b'the'` often occurs in our raw text training data, assigning it an entry in the vocabulary would reduce this 3-token sequence to a single token.

How do we select these subword units to add to our vocabulary? [\[R. Sennrich et al. \[3\]\]](#) propose to use byte-pair encoding (BPE; [\[P. Gage \[5\]\]](#)), a compression algorithm that iteratively replaces (“merges”) the most frequent pair of bytes with a single, new unused index. Note that this algorithm adds subword tokens to our vocabulary to maximize the compression of our input sequences—if a word occurs in our input text enough times, it’ll be represented as a single subword unit.

Subword tokenizers with vocabularies constructed via BPE are often called BPE tokenizers. In this assignment, we’ll implement a byte-level BPE tokenizer, where the vocabulary items are bytes or merged sequences of bytes, which give us the best of both worlds in terms of out-of-vocabulary handling and manageable input sequence lengths. The process of constructing the BPE tokenizer vocabulary is known as “training” the BPE tokenizer.

2.4 BPE Tokenizer Training

The BPE tokenizer training procedure consists of three main steps.

Vocabulary initialization

The tokenizer vocabulary is a one-to-one mapping from bytestring token to integer ID. Since we’re training a byte-level BPE tokenizer, our initial vocabulary is simply the set of all bytes. Since there are 256 possible byte values, our initial vocabulary is of size 256.

Pre-tokenization

Once you have a vocabulary, you could, in principle, count how often bytes occur next to each other in your text and begin merging them starting with the most frequent pair of bytes. However, this is quite computationally expensive, since we’d have to take a full pass over the corpus each time we merge. In

addition, directly merging bytes across the corpus may result in tokens that differ only in punctuation (e.g., `dog!` vs. `dog.`). These tokens would get completely different token IDs, even though they are likely to have high semantic similarity (since they differ only in punctuation).

To avoid this, we *pre-tokenize* the corpus. You can think of this as a coarse-grained tokenization over the corpus that helps us count how often pairs of characters appear. For example, the word `'text'` might be a pre-token that appears 10 times. In this case, when we count how often the characters `'t'` and `'e'` appear next to each other, we will see that the word `'text'` has `'t'` and `'e'` adjacent and we can increment their count by 10 instead of looking through the corpus. Since we're training a byte-level BPE model, each pre-token is represented as a sequence of UTF-8 bytes.

The original BPE implementation of [R. Sennrich et al. [3]] pre-tokenizes by simply splitting on whitespace (i.e., `s.split(" ")`). This method is still found in tokenizers based on SentencePiece (for instance the Llama 1 and 2 tokenizer).

Most modern tokenizers use a regex-based pre-tokenizer, a practice from GPT-2; [A. Radford et al. [6]]. We'll use a slightly prettier form of the original regex, fetched from github.com/openai/tiktoken/pull/234/files:

```
>>> PAT = r"\"(?:[sdmt]|ll|ve|re)| ?\p{L}+| ?\p{N}+| ?(?:\s\p{L}\p{N}]|\s+(?!S)|\s+\""
```

It may be useful to interactively split some text with this pre-tokenizer to get a better sense of its behavior:

```
>>> # requires `regex` package
>>> import regex as re
>>> re.findall(PAT, "some text that i'll pre-tokenize")
['some', ' text', ' that', ' i', 'll', ' pre', '-', 'tokenize']
```

When using it in your code, however, you should use `re.finditer` to avoid storing the pre-tokenized words as you construct your mapping from pre-tokens to their counts.

Compute BPE merges

Now that we've converted our input text into pre-tokens and represented each pre-token as a sequence of UTF-8 bytes, we can compute the BPE merges (i.e., train the BPE tokenizer). At a high level, the BPE algorithm iteratively counts every pair of bytes and identifies the pair with the highest frequency ("A", "B"). Every occurrence of this most frequent pair ("A", "B") is then *merged*, i.e., replaced with a new token "AB". This new merged token is added to our vocabulary; as a result, the final vocabulary after BPE training is the size of the initial vocabulary (256 in our case), plus the number of BPE merge operations performed during training. For efficiency during BPE training, we do not consider pairs that cross pre-token boundaries.² When computing merges, deterministically break ties in pair frequency by *preferring the lexicographically greater pair*. For example, if the pairs ("A", "B"), ("A", "C"), ("B", "ZZ"), and ("BA", "A") all have the highest frequency, we'd merge ("BA", "A"):

```
>>> max([(("A", "B"), ("A", "C"), ("B", "ZZ"), ("BA", "A"))],
('BA', 'A'))
```

Special tokens

Often, some strings (e.g., `<|endoftext|>`) are used to encode metadata (e.g., boundaries between documents). When encoding text, it's often desirable to treat some strings as "special tokens" that should

²Note that the original BPE formulation [R. Sennrich et al. [3]] specifies the inclusion of an end-of-word token. We do not add an end-of-word-token when training byte-level BPE models because all bytes (including whitespace and punctuation) are included in the model's vocabulary. Since we're explicitly representing spaces and punctuation, the learned BPE merges will naturally reflect these word boundaries.

never be split into multiple tokens (i.e., will always be preserved as a single token). For example, the end-of-sequence string `<|endoftext|>` should always be preserved as a single token (i.e., a single integer ID), so we know when to stop generating from the language model. These special tokens must be added to the vocabulary, so they have a corresponding fixed token ID.

Algorithm 1 of [R. Sennrich et al. [3]] contains an inefficient implementation of BPE tokenizer training (essentially following the steps that we outlined above). As a first exercise, it may be useful to implement and test this function to check your understanding.

Example (bpe_example): BPE training example

Here is a stylized example from [R. Sennrich et al. [3]]. Consider a corpus consisting of the following text

```
low low low low low
lower lower widest widest widest
newest newest newest newest newest newest
```

and the vocabulary has a special token `<|endoftext|>`.

Vocabulary

We initialize our vocabulary with our special token `<|endoftext|>` and the 256 byte values.

Pre-tokenization

For simplicity and to focus on the merge procedure, we assume in this example that pre-tokenization simply splits on whitespace. When we pre-tokenize and count, we end up with the frequency table.

```
{low: 5, lower: 2, widest: 3, newest: 6}
```

It is convenient to represent this as a `dict[tuple[bytes, ...], int]`, e.g. `{(l,o,w): 5, ...}`. Note that even a single byte is a `bytes` object in Python. There is no `byte` type in Python to represent a single byte, just as there is no `char` type in Python to represent a single character.

Merges

We first look at every successive pair of bytes and sum the frequency of the words where they appear `{lo: 7, ow: 7, we: 8, er: 2, wi: 3, id: 3, de: 3, es: 9, st: 9, ne: 6, ew: 6}`. The pairs `('e', 's')` and `('s', 't')` are tied, so we take the lexicographically greater pair, `('s', 't')`. We would then merge the pre-tokens so that we end up with `{(l,o,w): 5, (l,o,w,e,r): 2, (w,i,d,e,s,t): 3, (n,e,w,e,s,t): 6}`.

In the second round, we see that `(e, st)` is the most common pair (with a count of 9) and we would merge into `{(l,o,w): 5, (l,o,w,e,r): 2, (w,i,d,est): 3, (n,e,w,est): 6}`. Continuing this, the sequence of merges we get in the end will be `['s t', 'e st', 'o w', 'l ow', 'w est', 'n e', 'ne west', 'w i', 'wi d', 'wid est', 'low e', 'lowe r']`.

If we take 6 merges, we have `['s t', 'e st', 'o w', 'l ow', 'w est', 'n e']` and our vocabulary elements would be `[<|endoftext|>, [...256 BYTE CHARS], st, est, ow, low, west, ne]`.

With this vocabulary and set of merges, the word `newest` would tokenize as `[ne, west]`.

2.5 Experimenting with BPE Tokenizer Training

Let's train a byte-level BPE tokenizer on the TinyStories dataset. Instructions to find / download the dataset can be found in [Section 1](#). Before you start, we recommend taking a look at the TinyStories dataset to get a sense of what's in the data.

Parallelizing pre-tokenization

You will find that a major bottleneck is the pre-tokenization step. You can speed up pre-tokenization by parallelizing your code with the built-in library `multiprocessing`. Concretely, we recommend that in parallel implementations of pre-tokenization, you chunk the corpus while ensuring your chunk boundaries occur at the beginning of a special token. You are free to use the starter code at the following link verbatim to obtain chunk boundaries, which you can then use to distribute work across your processes:

https://github.com/stanford-cs336/assignment1-basics/blob/main/cs336_basics/pretokenization_example.py

This chunking will always be valid, since we never want to merge across document boundaries. For the purposes of the assignment, you can always split in this way. Don't worry about the edge case of receiving a very large corpus that does not contain `<|endoftext|>`.

Removing special tokens before pre-tokenization

Before running pre-tokenization with the regex pattern (using `re.finditer`), you should strip out all special tokens from your corpus (or your chunk, if using a parallel implementation). Make sure that you **split** on your special tokens, so that no merging can occur across the text they delimit. For example, if you have a corpus (or chunk) like `[Doc 1]<|endoftext|>[Doc 2]`, you should split on the special token `<|endoftext|>`, and pre-tokenize `[Doc 1]` and `[Doc 2]` separately, so that no merging can occur across the document boundary. In other words, special tokens define hard segmentation boundaries during training, but they should not themselves contribute to merge counts. This can be done using `re.split` with `"|".join(special_tokens)` as the delimiter (with careful use of `re.escape` since `|` may occur in the special tokens). The test `test_train_bpe_special_tokens` will test for this.

Optimizing the merging step

The naïve implementation of BPE training in the stylized example above is slow because for every merge, it iterates over all byte pairs to identify the most frequent pair. However, the only pair counts that change after each merge are those that overlap with the merged pair. Thus, BPE training speed can be improved by indexing the counts of all pairs and incrementally updating these counts, rather than explicitly iterating over each pair of bytes to count pair frequencies. You can get significant speedups with this caching procedure, though we note that the merging part of BPE training is *not* parallelizable in Python.

Low-Resource Tip: Profiling

You should use profiling tools like `cProfile` or `py-spy` to identify the bottlenecks in your implementation, and focus on optimizing those.

Low-Resource Tip: "Downscaling"

Instead of jumping to training your tokenizer on the full TinyStories dataset, we recommend you first train on a small subset of the data: a “debug dataset”. For example, you could train your tokenizer on the TinyStories validation set instead, which is 22K documents instead of 2.12M. This illustrates a general strategy of downscaling whenever possible to speed up development: for

example, using smaller datasets, smaller model sizes, etc. Choosing the size of the debug dataset or hyperparameter config requires careful consideration: you want your debug set to be large enough to have the same bottlenecks as the full configuration (so that the optimizations you make will generalize), but not so big that it takes forever to run.

Problem (`train_bpe`): BPE Tokenizer Training (15 points)

Deliverable: Write a function that, given a path to an input text file, trains a (byte-level) BPE tokenizer. Your BPE training function should handle (at least) the following input parameters:

Input

input_path: `str` Path to a text file with BPE tokenizer training data.

vocab_size: `int` A positive integer that defines the maximum final vocabulary size (including the initial byte vocabulary, vocabulary items produced from merging, and any special tokens).

special_tokens: `list[str]` A list of strings to add to the vocabulary. During training, treat them as hard boundaries that prevent merges across their spans, but do not include them when computing merge statistics.

Your BPE training function should return the resulting vocabulary and merges:

Output

vocab: `dict[int, bytes]` The tokenizer vocabulary, a mapping from `int` (token ID in the vocabulary) to `bytes` (token bytes).

merges: `list[tuple[bytes, bytes]]` A list of BPE merges produced from training. Each list item is a `tuple` of `bytes` (`<token1>`, `<token2>`), representing that `<token1>` was merged with `<token2>`. The merges should be ordered by order of creation.

To test your BPE training function against our provided tests, you will first need to implement the test adapter at `[adapters.run_train_bpe]`. Then, run `uv run pytest tests/test_train_bpe.py`. Your implementation should be able to pass all tests. Optionally (this could be a large time-investment), you can implement the key parts of your training method using some systems language, for instance C++ (consider `cppyy` or `nanobind`) or Rust (using `PyO3`). If you do this, be aware of which operations require copying vs reading directly from Python memory, and make sure to leave build instructions, or make sure it builds using only `pyproject.toml`. Also note that the GPT-2 regex is not well-supported in most regex engines and will be too slow in most that do. We have verified that Oniguruma is reasonably fast and supports negative lookahead, but the `regex` package in Python is, if anything, even faster.

Problem (`train_bpe_tinystories`): BPE Training on TinyStories (2 points)

- (a) Train a byte-level BPE tokenizer on the TinyStories dataset, using a maximum vocabulary size of 10,000. Make sure to add the TinyStories `<|endoftext|>` special token to the vocabulary. Serialize the resulting vocabulary and merges to disk for further inspection. How much time and memory did training take? What is the longest token in the vocabulary? Does it make sense?

Resource requirements: ≤ 30 minutes (no GPUs), ≤ 30 GB RAM

Hint You should be able to get under 2 minutes for BPE training using `multiprocessing` during pre-tokenization and the following two facts:

- (a) The `<|endoftext|>` token delimits documents in the data files.
- (b) The `<|endoftext|>` token is handled as a special case before the BPE merges are applied.

Deliverable: A one-to-two sentence response.

- (b) Profile your code. What part of the tokenizer training process takes the most time?

Deliverable: A one-to-two sentence response.

Next, we'll try training a byte-level BPE tokenizer on the OpenWebText dataset. As before, we recommend taking a look at the dataset to better understand its contents.

Problem (`train_bpe_expts_owt`): BPE Training on OpenWebText (2 points)

- (a) Train a byte-level BPE tokenizer on the OpenWebText dataset, using a maximum vocabulary size of 32,000. Serialize the resulting vocabulary and merges to disk for further inspection. What is the longest token in the vocabulary? Does it make sense?

Resource requirements: ≤ 12 hours (no GPUs), ≤ 100 GB RAM

Deliverable: A one-to-two sentence response.

- (b) Compare and contrast the tokenizer that you get training on TinyStories versus OpenWebText.

Deliverable: A one-to-two sentence response.

2.6 BPE Tokenizer: Encoding and Decoding

In the previous part of the assignment, we implemented a function to train a BPE tokenizer on input text to obtain a tokenizer vocabulary and a list of BPE merges. Now, we will implement a BPE tokenizer that loads a provided vocabulary and list of merges and uses them to encode and decode text to/from token IDs.

2.6.1 Encoding text

The process of encoding text by BPE mirrors how we train the BPE vocabulary. There are a few major steps.

Step 1: Pre-tokenize. We first pre-tokenize the sequence and represent each pre-token as a sequence of UTF-8 bytes, just as we did in BPE training. We will be merging these bytes within each pre-token into vocabulary elements, handling each pre-token independently (no merges across pre-token boundaries).

Step 2: Apply the merges. We then take the sequence of vocabulary element merges created during BPE training, and apply it to our pre-tokens *in the same order of creation*.

Example (`bpe_encoding`): BPE encoding example

For example, suppose our input string is 'the cat ate', our vocabulary is `{0: b' ', 1: b'a', 2: b'c', 3: b'e', 4: b'h', 5: b't', 6: b'th', 7: b' c', 8: b' a', 9: b'the', 10: b' at'}`, and our learned merges are `[(b't', b'h'), (b' ', b'c'), (b' ', b'a'), (b'th', b'e'), (b' a',`

`b't']`). First, our pre-tokenizer would split this string into `['the', ' cat', ' ate']`. Then, we'll look at each pre-token and apply the BPE merges.

The first pre-token `'the'` is initially represented as `[b't', b'h', b'e']`. Looking at our list of merges, we identify the first applicable merge to be `(b't', b'h')`, and use that to transform the pre-token into `[b'th', b'e']`. Then, we go back to the list of merges and identify the next applicable merge to be `(b'th', b'e')`, which transforms the pre-token into `[b'the']`. Finally, looking back at the list of merges, we see that there are no more that apply to the string (since the entire pre-token has been merged into a single token), so we are done applying the BPE merges. The corresponding integer sequence is `[9]`.

Repeating this process for the remaining pre-tokens, we see that the pre-token `' cat'` is represented as `[b' c', b'a', b't']` after applying the BPE merges, which becomes the integer sequence `[7, 1, 5]`. The final pre-token `' ate'` is `[b' at', b'e']` after applying the BPE merges, which becomes the integer sequence `[10, 3]`. Thus, the final result of encoding our input string is `[9, 7, 1, 5, 10, 3]`.

Special tokens

Your tokenizer should be able to properly handle user-defined special tokens when encoding text (provided when constructing the tokenizer).

Memory considerations

Suppose we want to tokenize a large text file that we cannot fit in memory. To efficiently tokenize this large file (or any other stream of data), we need to break it up into manageable chunks and process each chunk in turn, so that the memory complexity is constant as opposed to linear in the size of the text. In doing so, we need to make sure that a token doesn't cross chunk boundaries, else we'll get a different tokenization than the naïve method of tokenizing the entire sequence in-memory.

2.6.2 Decoding text

To decode a sequence of integer token IDs back to raw text, we can simply look up each ID's corresponding entries in the vocabulary (a byte sequence), concatenate them together, and then decode the bytes to a Unicode string. Note that input IDs are not guaranteed to map to valid Unicode strings (since a user could input any sequence of integer IDs). In the case that the input token IDs do not produce a valid Unicode string, you should replace the malformed bytes with the official Unicode replacement character `U+FFFD`.³ The `errors` argument of `bytes.decode` controls how Unicode decoding errors are handled, and using `errors='replace'` will automatically replace malformed data with the replacement marker.

Problem (tokenizer): Implementing the tokenizer (15 points)

Deliverable: Implement a `Tokenizer` class that, given a vocabulary and a list of merges, encodes text into integer IDs and decodes integer IDs into text. Your tokenizer should also support user-provided special tokens (appending them to the vocabulary if they aren't already there). We recommend the following interface:

³See [en.wikipedia.org/wiki/Specials_\(Unicode_block\)#Replacement_character](https://en.wikipedia.org/wiki/Specials_(Unicode_block)#Replacement_character) for more information about the Unicode replacement character.

def __init__(self, vocab, merges, special_tokens=None) Construct a tokenizer from a given vocabulary, list of merges, and (optionally) a list of special tokens. This function should accept the following parameters:

vocab: `dict[int, bytes]`

merges: `list[tuple[bytes, bytes]]`

special_tokens: `list[str] | None = None`

def from_files(cls, vocab_filepath, merges_filepath, special_tokens=None) Class method that constructs and returns a `Tokenizer` from a serialized vocabulary and list of merges (in the same format that your BPE training code output) and (optionally) a list of special tokens. This method should accept the following additional parameters:

vocab_filepath: `str`

merges_filepath: `str`

special_tokens: `list[str] | None = None`

def encode(self, text: str) -> list[int] Encode an input text into a sequence of token IDs.

def encode_iterable(self, iterable: Iterable[str]) -> Iterator[int] Given an iterable of strings (e.g., a Python file handle), return a generator that lazily yields token IDs. This is required for memory-efficient tokenization of large files that we cannot directly load into memory.

def decode(self, ids: list[int]) -> str Decode a sequence of token IDs into text.

To test your `Tokenizer` against our provided tests, you will first need to implement the test adapter at `[adapters.get_tokenizer]`. Then, run `uv run pytest tests/test_tokenizer.py`. Your implementation should be able to pass all tests.

2.7 Experiments

Problem (tokenizer_experiments): Experiments with tokenizers (4 points)

- (a) Sample 10 documents from TinyStories and OpenWebText. Using your previously-trained TinyStories and OpenWebText tokenizers (10K and 32K vocabulary size, respectively), encode these sampled documents into integer IDs. What is each tokenizer's compression ratio (bytes/token)?

Deliverable: A one-to-two sentence response.

- (b) What happens if you tokenize your OpenWebText sample with the TinyStories tokenizer? Compare the compression ratio and/or qualitatively describe what happens.

Deliverable: A one-to-two sentence response.

- (c) Estimate the throughput of your tokenizer (e.g., in bytes/second). How long would it take to tokenize the Pile dataset (825GB of text)?

Deliverable: A one-to-two sentence response.

- (d) Using your TinyStories and OpenWebText tokenizers, encode the respective training and development datasets into a sequence of integer token IDs. We'll use this later to train our

language model. We recommend serializing the token IDs as a NumPy array of datatype `uint16`. Why is `uint16` an appropriate choice?

Deliverable: A one-to-two sentence response.

3 Transformer Language Model Architecture

A language model takes as input a batched sequence of integer token IDs (i.e., `torch.Tensor` of shape $(\text{batch_size}, \text{sequence_length})$), and returns a (batched) normalized probability distribution over the vocabulary (i.e., a PyTorch Tensor of shape $(\text{batch_size}, \text{sequence_length}, \text{vocab_size})$), where the predicted distribution is over the next word for each input token. When training the language model, we use these next-word predictions to calculate the cross-entropy loss between the actual next word and the predicted next word. When generating text from the language model during inference, we take the predicted next-word distribution from the final time step (i.e., the last item in the sequence) to generate the next token in the sequence (e.g., by taking the token with the highest probability, sampling from the distribution, etc.), add the generated token to the input sequence, and repeat.

In this part of the assignment, you will build this Transformer language model from scratch. We will begin with a high-level description of the model before progressively detailing the individual components.

3.1 Transformer LM

Given a sequence of token IDs, the Transformer language model uses an input embedding to convert token IDs to dense vectors, passes the embedded tokens through `num_layers` Transformer blocks, and then applies a learned linear projection (the “output embedding” or “LM head”) to produce the predicted next-token logits. See [Figure 1](#) for a schematic representation.

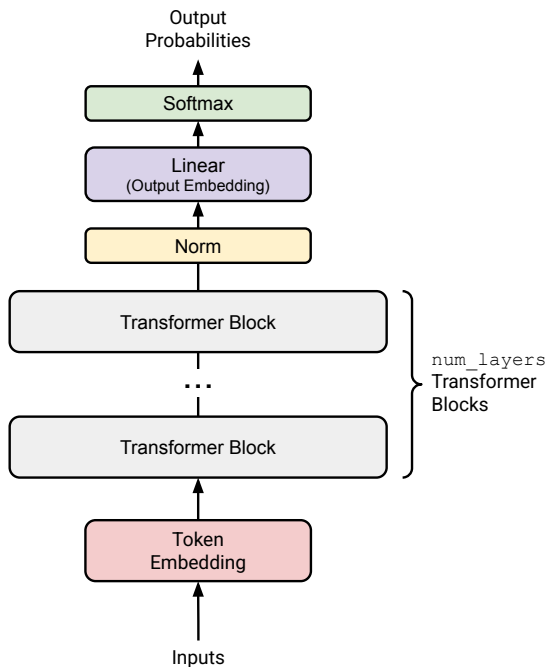


Figure 1: An overview of our Transformer language model.

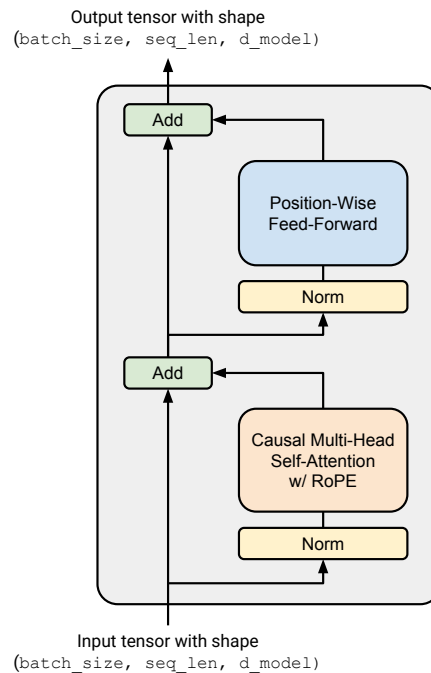


Figure 2: A pre-norm Transformer block.

Token Embeddings

In the very first step, the Transformer *embeds* the (batched) sequence of token IDs into a sequence of vectors containing information on the token identity (red blocks in [Figure 1](#)).

More specifically, given a sequence of token IDs, the Transformer language model uses a token embedding layer to produce a sequence of vectors. Each embedding layer takes in a tensor of integers of shape `(batch_size, sequence_length)` and produces a sequence of vectors of shape `(batch_size, sequence_length, d_model)`.

Pre-norm Transformer Block

After embedding, the activations are processed by several identically structured neural net layers. A standard decoder-only Transformer language model consists of `num_layers` identical layers (commonly called Transformer “blocks”). Each Transformer block takes in an input of shape `(batch_size, sequence_length, d_model)` and returns an output of shape `(batch_size, sequence_length, d_model)`. Each block aggregates information across the sequence (via self-attention) and non-linearly transforms it (via the feed-forward layers).

After `num_layers` Transformer blocks, we will take the final activations and turn them into a distribution over the vocabulary.

We will implement the “pre-norm” Transformer block (detailed in [Section 3.4](#)), which additionally requires the use of layer normalization (detailed below) after the final Transformer block to ensure its outputs are properly scaled.

After this normalization, we will use a standard learned linear transformation to convert the output of the Transformer blocks into predicted next-token logits (see, e.g., [A. Radford et al. \[7\]](#) equation 2).

3.2 Remark: Batching, Einsum and Efficient Computation

Throughout the Transformer, we will be performing the same computation applied to many batch-like inputs. Here are a few examples:

- **Elements of a batch:** we apply the same Transformer `forward` operation on each batch element.
- **Sequence length:** the “position-wise” operations like RMSNorm and feed-forward operate identically on each position of a sequence.
- **Attention heads:** the attention operation is batched across attention heads in a “multi-headed” attention operation.

It is useful to have an ergonomic way of performing such operations in a way that fully utilizes the GPU, and is easy to read and understand. Many PyTorch operations can take in excess “batch-like” dimensions at the start of a tensor and repeat/broadcast the operation across these dimensions efficiently.

For instance, say we are doing a position-wise, batched operation. We have a “data tensor” D of shape `(batch_size, sequence_length, d_model)`, and we would like to do a batched vector-matrix multiply against a matrix A of shape `(d_model, d_model)`. In this case, `D @ A` will do a batched matrix multiply, which is an efficient primitive in PyTorch, where the `(batch_size, sequence_length)` dimensions are batched over.

Because of this, it is helpful to assume that your functions may be given additional batch-like dimensions and to keep those dimensions at the start of the PyTorch shape. To organize tensors so they can be batched in this manner, they might need to be shaped using many steps of `view`, `reshape` and `transpose`. This can be a bit of a pain, and it often gets hard to read what the code is doing and what the shapes of your tensors are.

A more ergonomic option is to use *einsum notation* within `torch.einsum`, or rather use framework-agnostic libraries like `einops` or `einx`. The two key ops are `einsum`, which can do tensor contractions with arbitrary dimensions of input tensors, and `rearrange`, which can reorder, concatenate, and split arbitrary dimensions. It turns out almost all operations in machine learning are some combination of dimension juggling and tensor contraction with the occasional (usually pointwise) nonlinear function. This means that a lot of your code can be more readable and flexible when using einsum notation.

We **strongly** recommend learning and using einsum notation for the class. Students who have not been exposed to einsum notation before should use `einops` (docs [here](#)), and students who are already comfortable with `einops` should learn the more general `einx` ([here](#)).⁴ Both packages are already installed in the environment we've supplied.

Here we give some examples of how einsum notation can be used. These are a supplement to the documentation for `einops`, which you should read first.

Example (`einstein_example1`): Batched matrix multiplication with `einops.einsum`

```
import torch
from einops import rearrange, einsum

## Basic implementation
Y = D @ A.T
# Hard to tell the input and output shapes and what they mean.
# What shapes can D and A have, and do any of these have unexpected behavior?

## Einsum is self-documenting and robust
#           D           A           ->           Y
Y = einsum(D, A, "batch sequence d_in, d_out d_in -> batch sequence d_out")

## Or, a batched version where D can have any leading dimensions but A is constrained.
Y = einsum(D, A, "... d_in, d_out d_in -> ... d_out")
```

Example (`einstein_example2`): Broadcasted operations with `einops.rearrange`

We have a batch of images, and for each image we want to generate 10 dimmed versions based on some scaling factor:

```
images = torch.randn(64, 128, 128, 3) # (batch, height, width, channel)
dim_by = torch.linspace(start=0.0, end=1.0, steps=10)

## Reshape and multiply
dim_value = rearrange(dim_by, "dim_value -> 1 dim_value 1 1 1")
images_rearr = rearrange(images, "b height width channel -> b 1 height width channel")
dimmed_images = images_rearr * dim_value

## Or in one go:
dimmed_images = einsum(
    images, dim_by,
    "batch height width channel, dim_value -> batch dim_value height width channel"
)
```

⁴It's worth noting that while `einops` has a great amount of support, `einx` is not as battle-tested. You should feel free to fall back to using `einops` with some more plain PyTorch if you find any limitations or bugs in `einx`.

Example (einstein_example3): Pixel mixing with `einops.rearrange`

Suppose we have a batch of images represented as a tensor of shape (batch, height, width, channel), and we want to perform a linear transformation across all pixels of the image, but this transformation should happen independently for each channel. Our linear transformation is represented as a matrix B of shape (height * width, height * width).

```
channels_last = torch.randn(64, 32, 32, 3) # (batch, height, width, channel)
B = torch.randn(32*32, 32*32)

## Rearrange an image tensor for mixing across all pixels
channels_last_flat = channels_last.view(
    -1, channels_last.size(1) * channels_last.size(2), channels_last.size(3)
)
channels_first_flat = channels_last_flat.transpose(1, 2)
channels_first_flat_transformed = channels_first_flat @ B.T
channels_last_flat_transformed = channels_first_flat_transformed.transpose(1, 2)
channels_last_transformed = channels_last_flat_transformed.view(*channels_last.shape)
```

Instead, using `einops`:

```
height = width = 32
## Rearrange replaces clunky torch view + transpose
channels_first = rearrange(
    channels_last,
    "batch height width channel -> batch channel (height width)"
)
channels_first_transformed = einsum(
    channels_first, B,
    "batch channel pixel_in, pixel_out pixel_in -> batch channel pixel_out"
)
channels_last_transformed = rearrange(
    channels_first_transformed,
    "batch channel (height width) -> batch height width channel",
    height=height, width=width
)
```

Or, if you're feeling crazy: all in one go using `einx.dot` (`einx` equivalent of `einops.einsum`)

```
height = width = 32
channels_last_transformed = einx.dot(
    "batch row_in col_in channel, (row_out col_out) (row_in col_in)"
    "-> batch row_out col_out channel",
    channels_last, B,
    col_in=width, col_out=width
)
```

The first implementation here could be improved by placing comments before and after to indicate what the input and output shapes are, but this is clunky and susceptible to bugs. With `einsum` notation, documentation *is* implementation!

`Einsum` notation can handle arbitrary input batching dimensions, but also has the key benefit of being *self-documenting*. It's much clearer what the relevant shapes of your input and output tensors are in code

that uses einsum notation. For the remaining tensors, you can consider using Tensor type hints, for instance using the `jaxtyping` library (not specific to JAX).

We will talk more about the performance implications of using einsum notation in assignment 2, but for now know that they're almost always better than the alternative!

3.2.1 Mathematical Notation and Memory Ordering

Many machine learning papers use row vectors in their notation, which result in representations that mesh well with the row-major memory ordering used by default in NumPy and PyTorch. With row vectors, a linear transformation looks like

$$y = xW^\top, \tag{1}$$

for row-major $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ and row-vector $x \in \mathbb{R}^{1 \times d_{\text{in}}}$. Notice that this lets us batch inputs by increasing the outermost dimension of x , meaning we can substitute vector input x for matrix input $X \in \mathbb{R}^{\text{batch} \times d_{\text{in}}}$.

In linear algebra it's generally more common to use column vectors, where linear transformations look like

$$y = Wx, \tag{2}$$

given a row-major $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ and column-vector $x \in \mathbb{R}^{d_{\text{in}}}$. To batch the input in this setting, the batch dimension to x would have to come last, so x would need to be replaced with a matrix $\tilde{X} \in \mathbb{R}^{d_{\text{in}} \times \text{batch}}$.

We will use mostly column vectors for mathematical notation in this assignment, since math generally follows this notation. You should keep in mind that if you want to use plain matrix multiplication notation, you will have to apply matrices with a transpose as seen in the row vector convention in [Equation 1](#), since PyTorch uses row-major memory ordering. If you use `einsum` for your linear algebra operations, this should be a non-issue as long as you label your axes correctly. As an aside, it's worth noting that other languages/linear algebra packages like Matlab, Julia and Fortran all use column-major memory ordering, meaning *batching dimensions come last*, but Python and related packages have adopted the C-standard of row-major ordering.

3.3 Basic Building Blocks: Linear and Embedding Modules

3.3.1 Parameter Initialization

Training neural networks effectively often requires careful initialization of the model parameters—bad initializations can lead to undesirable behavior such as vanishing or exploding gradients. Pre-norm transformers are unusually robust to initializations, but they can still have a significant impact on training speed and convergence. Since this assignment is already long, we will save the details for assignment 3, and instead give you some approximate initializations that should work well for most cases. For now, use:

- Linear weights: $\mathcal{N}\left(\mu = 0, \sigma^2 = \frac{2}{d_{\text{in}} + d_{\text{out}}}\right)$ truncated at $[-3\sigma, 3\sigma]$.
- Embedding: $\mathcal{N}(\mu = 0, \sigma^2 = 1)$ truncated at $[-3, 3]$
- RMSNorm: $\mathbb{1}$

You should use `torch.nn.init.trunc_normal_` to initialize the truncated normal weights.

3.3.2 Linear Module

Linear layers are a fundamental building block of Transformers and neural nets in general. First, you will implement your own `Linear` class that inherits from `torch.nn.Module` and performs a linear transformation:

$$y = Wx. \tag{3}$$

Note that we do not include a bias term, following most modern LLMs.

Problem (linear): Implementing the linear module (1 point)

Deliverable: Implement a `Linear` class that inherits from `torch.nn.Module` and performs a linear transformation. Your implementation should follow the interface of PyTorch's built-in `nn.Linear` module, except for not having a `bias` argument or parameter. We recommend the following interface:

```
def __init__(self, in_features, out_features, device=None, dtype=None) Construct a linear
    transformation module. This function should accept the following parameters:
    in_features: int final dimension of the input
    out_features: int final dimension of the output
    device: torch.device | None = None Device to store the parameters on
    dtype: torch.dtype | None = None Data type of the parameters
def forward(self, x: torch.Tensor) -> torch.Tensor Apply the linear transformation to the
    input.
```

Make sure to:

- subclass `nn.Module`
- call the superclass constructor
- construct and store your parameter as W (not W^T), putting it in an `nn.Parameter`
- of course, don't use `nn.Linear` or `nn.functional.linear`

For initializations, use the settings from above along with `torch.nn.init.trunc_normal_` to initialize the weights.

To test your `Linear` module, implement the test adapter at `[adapters.run_linear]`. The adapter should load the given weights into your `Linear` module. You can use `Module.load_state_dict` for this purpose. Then, run `uv run pytest -k test_linear`.

3.3.3 Embedding Module

As discussed above, the first layer of the Transformer is an embedding layer that maps integer token IDs into a vector space of dimension `d_model`. We will implement a custom `Embedding` class that inherits from `torch.nn.Module` (so you should not use `nn.Embedding`). The `forward` method should select the embedding vector for each token ID by indexing into an embedding matrix of shape `(vocab_size, d_model)` using a `torch.LongTensor` of token IDs with shape `(batch_size, sequence_length)`.

Problem (embedding): Implement the embedding module (1 point)

Deliverable: Implement the `Embedding` class that inherits from `torch.nn.Module` and performs an embedding lookup. Your implementation should follow the interface of PyTorch's built-in `nn.Embedding` module. We recommend the following interface:

```
def __init__(self, num_embeddings, embedding_dim, device=None, dtype=None) Construct an
    embedding module. This function should accept the following parameters:
    num_embeddings: int Size of the vocabulary
    embedding_dim: int Dimension of the embedding vectors, i.e.,  $d_{\text{model}}$ 
    device: torch.device | None = None Device to store the parameters on
    dtype: torch.dtype | None = None Data type of the parameters
def forward(self, token_ids: torch.Tensor) -> torch.Tensor Lookup the embedding vectors
    for the given token IDs.
```

Make sure to:

- subclass `nn.Module`
- call the superclass constructor
- initialize your embedding matrix as an `nn.Parameter`
- store the embedding matrix with the `d_model` being the final dimension
- of course, don't use `nn.Embedding` or `nn.functional.embedding`

Again, use the settings from above for initialization, and use `torch.nn.init.trunc_normal_` to initialize the weights.

To test your implementation, implement the test adapter at `[adapters.run_embedding]`. Then, run `uv run pytest -k test_embedding`.

3.4 Pre-Norm Transformer Block

Each Transformer block has two sub-layers: a multi-head self-attention mechanism and a position-wise feed-forward network ([A. Vaswani et al., 2017], section 3.1).

In the original Transformer paper, the model uses a residual connection around each of the two sub-layers, followed by layer normalization. This architecture is commonly known as the “post-norm” Transformer, since layer normalization is applied to the sub-layer output. However, a variety of work has found that moving layer normalization from the output of each sub-layer to the input of each sub-layer (with an additional layer normalization after the final Transformer block) improves Transformer training stability [T. Q. Nguyen et al., 2019; R. Xiong et al., 2020] — see Figure 2 for a visual representation of this “pre-norm” Transformer block. The output of each Transformer block sub-layer is then added to the sub-layer input via the residual connection ([A. Vaswani et al. [8], section 5.4). An intuition for pre-norm is that there is a clean “residual stream” without any normalization going from the input embeddings to the final output of the Transformer, which is purported to improve gradient flow. This pre-norm Transformer is now the standard used in language models today (e.g., GPT-3, LLaMA, PaLM, etc.), so we will implement this variant. We will walk through each of the components of a pre-norm Transformer block, implementing them in sequence.

3.4.1 Root Mean Square Layer Normalization

The original Transformer implementation of [A. Vaswani et al. [8]] uses layer normalization [J. L. Ba et al., 2016] to normalize activations. Following [H. Touvron et al. [12]], we will use root mean square layer normalization (RMSNorm; [B. Zhang et al. [13], equation 4) for layer normalization. Given a vector $a \in \mathbb{R}^{d_{\text{model}}}$ of activations, RMSNorm will rescale each activation a_i as follows:

$$\text{RMSNorm}(a_i) = \frac{a_i}{\text{RMS}(a)} g_i, \quad (4)$$

where $\text{RMS}(a) = \sqrt{\frac{1}{d_{\text{model}}} \sum_{i=1}^{d_{\text{model}}} a_i^2 + \varepsilon}$. Here, g_i is a learnable “gain” parameter (there are `d_model` such parameters total), and ε is a hyperparameter that is often fixed at $1e-5$.

You should upcast your input to `torch.float32` to prevent overflow when you square the input. Overall, your `forward` method should look like:

```
in_dtype = x.dtype
x = x.to(torch.float32)

# Your code here performing RMSNorm
...
result = ...

# Return the result in the original dtype
return result.to(in_dtype)
```

Problem (rmsnorm): Root Mean Square Layer Normalization (1 point)

Deliverable: Implement `RMSNorm` as a `torch.nn.Module`. We recommend the following interface:

def `__init__`(self, d_model: int, eps: float = 1e-5, device=None, dtype=None) Construct the `RMSNorm` module. This function should accept the following parameters:

d_model: int Hidden dimension of the model

eps: float = 1e-5 Epsilon value for numerical stability

device: torch.device | None = None Device to store the parameters on

dtype: torch.dtype | None = None Data type of the parameters

def `forward`(self, x: torch.Tensor) -> torch.Tensor Process an input tensor of shape `(batch_size, sequence_length, d_model)` and return a tensor of the same shape.

Note: Remember to upcast your input to `torch.float32` before performing the normalization (and later downcast to the original dtype), as described above.

To test your implementation, implement the test adapter at `[adapters.run_rmsnorm]`. Then, run `uv run pytest -k test_rmsnorm`.

3.4.2 Position-Wise Feed-Forward Network

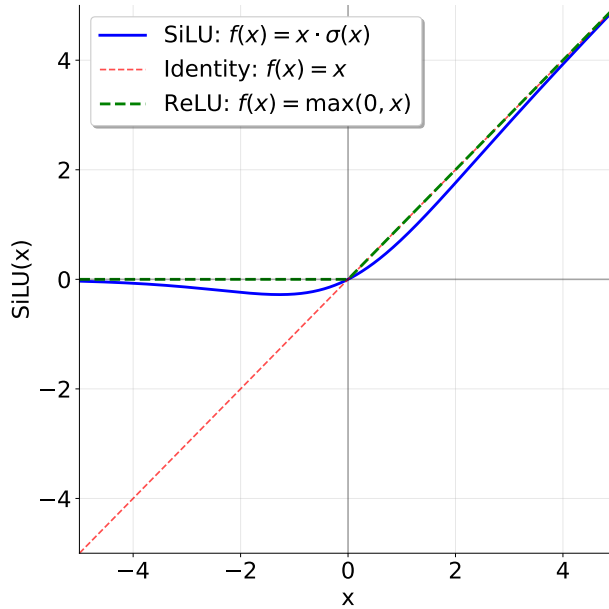


Figure 3: Comparing the SiLU (aka Swish) and ReLU activation functions.

In the original Transformer paper (section 3.3 of [A. Vaswani et al., 8]), the Transformer feed-forward network consists of two linear transformations with a ReLU activation ($\text{ReLU}(x) = \max(0, x)$) between them. In that original architecture, the dimensionality of the inner feed-forward layer is typically 4x the input dimensionality.

However, modern language models tend to incorporate two main changes compared to this original design: they use another activation function and employ a gating mechanism. Specifically, we will implement the “SwiGLU” activation function adopted in LLMs like Llama 3 [A. Grattafiori et al., 2024] and Qwen 2.5 [A. Yang et al., 2024], which combines the SiLU (often called Swish) activation with a gating mechanism called a Gated Linear Unit (GLU). We will also omit the bias terms sometimes used in linear layers, following most modern LLMs since PaLM [A. Chowdhery et al., 2022] and LLaMA [H. Touvron et al., 2023].

The SiLU or Swish activation function [D. Hendrycks et al., 2016; S. Elfwing et al., 2017] is defined as follows:

$$\text{SiLU}(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}} \quad (5)$$

As can be seen in Figure 3, the SiLU activation function is similar to the ReLU activation function, but is smooth at zero.

Gated Linear Units (GLUs) were originally defined by [Y. N. Dauphin et al., 19] as the element-wise product of a linear transformation passed through a sigmoid function and another linear transformation:

$$\text{GLU}(x, W_1, W_2) = \sigma(W_1 x) \odot W_2 x, \quad (6)$$

where $\odot$ represents element-wise multiplication. Gated Linear Units are suggested to “reduce the vanishing gradient problem for deep architectures by providing a linear path for the gradients while retaining non-linear capabilities.”

Putting the SiLU/Swish and GLU together, we get the SwiGLU, which we will use for our feed-forward networks:

$$\text{FFN}(x) = \text{SwiGLU}(x, W_1, W_2, W_3) = W_2(\text{SiLU}(W_1 x) \odot W_3 x), \quad (7)$$

where $x \in \mathbb{R}^{d_{\text{model}}}$, $W_1, W_3 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$, $W_2 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$, and canonically, $d_{\text{ff}} = \frac{8}{3}d_{\text{model}}$. For concrete implementations, it is fine to round this to a nearby multiple of 64 for hardware efficiency.

N. Shazeer [20] first proposed combining the SiLU/Swish activation with GLUs and conducted experiments showing that SwiGLU outperforms baselines like ReLU and SiLU (without gating) on language modeling tasks. Later in the assignment, you will compare SwiGLU and SiLU. Though we've mentioned some heuristic arguments for these components (and the papers provide more supporting evidence), it's good to keep an empirical perspective: a now famous quote from Shazeer's paper is

“We offer no explanation as to why these architectures seem to work; we attribute their success, as all else, to divine benevolence.”

Problem (positionwise_feedforward): Implement the position-wise feed-forward network (2 points)

Deliverable: Implement the SwiGLU feed-forward network, composed of a SiLU activation function and a GLU.

Note: in this particular case, you should feel free to use `torch.sigmoid` in your implementation for numerical stability.

You should set d_{ff} to approximately $\frac{8}{3} \times d_{\text{model}}$ in your implementation, while ensuring that the dimensionality of the inner feed-forward layer is a multiple of 64 to make good use of your hardware. To test your implementation against our provided tests, you will need to implement the test adapter at `[adapters.run_swiglu]`. Then, run `uv run pytest -k test_swiglu` to test your implementation.

3.4.3 Relative Positional Embeddings

To inject positional information into the model, we will implement Rotary Position Embeddings

[J. Su et al., 2021], often called RoPE. For a given query token $q^{(i)} = W_q x^{(i)} \in \mathbb{R}^d$ at token position i , we will apply a pairwise rotation matrix R^i , giving us $q'^{(i)} = R^i q^{(i)} = R^i W_q x^{(i)}$. Here, R^i will rotate pairs of embedding elements $q_{2k-1:2k}^{(i)}$ as 2d vectors by the angle $\theta_{i,k} = \frac{i}{\Theta(2k-2)/d}$ for $k \in \{1, \dots, d/2\}$ and some constant Θ . Thus, we can consider R^i to be a block-diagonal matrix of size $d \times d$, with blocks R_k^i for $k \in \{1, \dots, \frac{d}{2}\}$, with

$$R_k^i = \begin{pmatrix} \cos(\theta_{i,k}) & -\sin(\theta_{i,k}) \\ \sin(\theta_{i,k}) & \cos(\theta_{i,k}) \end{pmatrix} \quad (8)$$

Thus we get the full rotation matrix

$$R^i = \begin{pmatrix} R_1^i & 0 & 0 & \dots & 0 \\ 0 & R_2^i & 0 & \dots & 0 \\ 0 & 0 & R_3^i & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & R_{d/2}^i \end{pmatrix}, \quad (9)$$

where 0s represent 2×2 zero matrices. While one could construct the full $d \times d$ matrix, a good solution should use the properties of this matrix to implement the transformation more efficiently. Since we only care about the relative rotation of tokens within a given sequence, we can reuse the values we compute for $\cos(\theta_{i,k})$ and $\sin(\theta_{i,k})$ across layers, and different batches. If you would like to optimize it, you may use a single RoPE module referenced by all layers, and it can have a 2d pre-computed buffer of sin and cos values created during init with `self.register_buffer(persistent=False)`, instead of an `nn.Parameter` (because we do not want to learn these fixed cosine and sine values). The exact same rotation process we did for our $q^{(i)}$ is then done for $k^{(j)}$, rotating by the corresponding R^j . Notice that this layer has no learnable parameters.

Problem (rope): Implement RoPE (2 points)

Deliverable: Implement a class `RotaryPositionalEmbedding` that applies RoPE to the input tensor.

The following interface is recommended:

```
def __init__(self, theta: float, d_k: int, max_seq_len: int, device=None) Construct the
    RoPE module and create buffers if needed.

    theta: float  $\Theta$  value for the RoPE
    d_k: int dimension of query and key vectors
    max_seq_len: int Maximum sequence length that will be input
    device: torch.device | None = None Device to store the buffer on

def forward(self, x: torch.Tensor, token_positions: torch.Tensor) -> torch.Tensor Process
    an input tensor of shape  $(\dots, \text{seq\_len}, d_k)$  and return a tensor of the same shape. Note
    that you should tolerate  $x$  with an arbitrary number of batch dimensions. You should assume
    that the token positions are a tensor of shape  $(\dots, \text{seq\_len})$  specifying the token positions of
     $x$  along the sequence dimension.
```

You should use the token positions to slice your (possibly precomputed) cos and sin tensors along the sequence dimension.

To test your implementation, complete `[adapters.run_rope]` and make sure it passes `uv run pytest -k test_rope`.

3.4.4 Scaled Dot-Product Attention

We will now implement scaled dot-product attention as described in [\[A. Vaswani et al. \[8\]\]](#) (section 3.2.1). As a preliminary step, the definition of the Attention operation will make use of softmax, an operation that takes an unnormalized vector of scores and turns it into a normalized distribution:

$$\text{softmax}(v)_i = \frac{\exp(v_i)}{\sum_{j=1}^n \exp(v_j)}. \quad (10)$$

Note that $\exp(v_i)$ can become $\inf$ for large values (then, $\frac{\inf}{\inf} = \text{NaN}$). We can avoid this by noticing that the softmax operation is invariant to adding any constant c to all inputs. We can leverage this property for numerical stability—typically, we will subtract the largest entry of v from all elements of v , making the new largest entry 0. You will now implement softmax, using this trick for numerical stability.

Problem (softmax): Implement softmax (1 point)

Deliverable: Write a function to apply the softmax operation on a tensor. Your function should take two parameters: a tensor and a *dimension* i , and apply softmax to the i -th dimension of the input tensor. The output tensor should have the same shape as the input tensor, but its i -th dimension will now have a normalized probability distribution. Use the trick of subtracting the maximum value in the i -th dimension from all elements of the i -th dimension to avoid numerical stability issues.

To test your implementation, complete `[adapters.run_softmax]` and make sure it passes `uv run pytest -k test_softmax_matches_pytorch`.

We can now define the Attention operation mathematically as follows:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (11)$$

where $Q \in \mathbb{R}^{n \times d_k}$, $K \in \mathbb{R}^{m \times d_k}$, and $V \in \mathbb{R}^{m \times d_v}$. Here, Q , K and V are all inputs to this operation—note that these are not the learnable parameters.

Masking: It is sometimes convenient to *mask* the output of an attention operation. A mask should have the shape $M \in \{\text{True}, \text{False}\}^{n \times m}$, and each row i of this boolean matrix indicates which keys the query i should attend to. Canonically (and slightly confusingly), a value of `True` at position (i, j) indicates that the query i *does* attend to the key j , and a value of `False` indicates that the query *does not* attend to the key. In other words, “information flows” at (i, j) pairs with value `True`. For example, consider a 1×3 mask matrix with entries `[[True, True, False]]`. The single query vector attends only to the first two keys.

Computationally, it will be much more efficient to use masking than to compute attention on subsequences, and we can do this by taking the pre-softmax values $(QK^T / \sqrt{d_k})$ and adding a $-\infty$ to any entry of the mask matrix that is `False`.

Problem (scaled_dot_product_attention): Implement scaled dot-product attention (5 points)

Deliverable: Implement the scaled dot-product attention function. Your implementation should handle keys and queries of shape `(batch_size, ..., seq_len, d_k)` and values of shape `(batch_size, ..., seq_len, d_v)`, where `...` represents any number of other batch-like dimensions (if provided). The implementation should return an output with the shape `(batch_size, ..., seq_len, d_v)`. See [Section 3.2](#) for a discussion on batch-like dimensions.

Your implementation should also support an optional user-provided boolean mask of shape `(seq_len, seq_len)`. The attention probabilities of positions with a mask value of `True` should collectively sum to 1, and the attention probabilities of positions with a mask value of `False` should be zero.

To test your implementation against our provided tests, you will need to implement the test adapter at `[adapters.run_scaled_dot_product_attention]`. `uv run pytest -k test_scaled_dot_product_attention` tests your implementation on third-order input tensors, while `uv run pytest -k test_4d_scaled_dot_product_attention` tests your implementation on fourth-order input tensors.

3.4.5 Causal Multi-Head Self-Attention

We will implement multi-head self-attention as described in section 3.2.2 of [\[A. Vaswani et al. \[8\]\]](#). Recall that, mathematically, the operation of applying multi-head attention is defined as follows:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \quad (12)$$

$$\text{for head}_i = \text{Attention}(Q_i, K_i, V_i) \quad (13)$$

with Q_i, K_i, V_i being slice number $i \in \{1, \dots, h\}$ of size d_k or d_v of the embedding dimension for Q, K , and V respectively. With `Attention` being the scaled dot-product attention operation defined in [\[Section 3.4.4\]](#). From this we can form the multi-head *self*-attention operation:

$$\text{MultiHeadSelfAttention}(x) = W_O \text{MultiHead}(W_Q x, W_K x, W_V x) \quad (14)$$

Here, the learnable parameters are $W_Q \in \mathbb{R}^{h d_k \times d_{\text{model}}}$, $W_K \in \mathbb{R}^{h d_k \times d_{\text{model}}}$, $W_V \in \mathbb{R}^{h d_v \times d_{\text{model}}}$, and $W_O \in \mathbb{R}^{d_{\text{model}} \times h d_v}$. Since the Q s, K s, and V s are sliced in the multi-head attention operation, we can think of W_Q, W_K and W_V as being separated for each head along the output dimension. When you have this working, you should be computing the key, value, and query projections in a total of three matrix multiplies.⁵

Causal masking

Your implementation should prevent the model from attending to future tokens in the sequence. In other words, if the model is given a token sequence $t_1, \dots, t_n$, and we want to calculate the next-word predictions for the prefix $t_1, \dots, t_i$ (where $i < n$), the model should *not* be able to access (attend to) the token representations at positions $t_{i+1}, \dots, t_n$ since it will not have access to these tokens when generating text during inference (and these future tokens leak information about the identity of the true next word, trivializing the language modeling pre-training objective). For an input token sequence $t_1, \dots, t_n$ we can naively prevent access to future tokens by running multi-head self-attention n times (for the n unique prefixes in the sequence). Instead, we'll use causal attention masking, which allows token i to attend to all positions $j \leq i$ in the sequence. You can use `torch.triu` or a broadcasted index comparison to construct this mask, and you should take advantage of the fact that your scaled dot-product attention implementation from [\[Section 3.4.4\]](#) already supports attention masking.

Applying RoPE

RoPE should be applied to the query and key vectors, but not the value vectors. Also, the head dimension should be handled as a batch dimension, because in multi-head attention, attention is being applied independently for each head. This means that precisely the same RoPE rotation should be applied to the query and key vectors for each head.

Problem (multihead_self_attention): Implement causal multi-head self-attention (5 points)

⁵As a stretch goal, try combining the key, query, and value projections into a single weight matrix so you only need a single matrix multiply.

Deliverable: Implement causal multi-head self-attention as a `torch.nn.Module`. Your implementation should accept (at least) the following parameters:

d_model: `int` Dimensionality of the Transformer block inputs.

num_heads: `int` Number of heads to use in multi-head self-attention.

Following [A. Vaswani et al. [8]], set $d_k = d_v = \frac{d_{\text{model}}}{h}$. To test your implementation against our provided tests, implement the test adapter at `[adapters.run_multihead_self_attention]`. Then, run `uv run pytest -k test_multihead_self_attention` to test your implementation.

3.5 The Full Transformer LM

Let's begin by assembling the Transformer block (it will be helpful to refer back to [Figure 2](#)). A Transformer block contains two 'sub-layers', one for the multihead self attention, and another for the SwiGLU feed-forward network. In each sub-layer, we first perform RMSNorm, then the main operation (MHA/FF), finally adding in the residual connection.

To be concrete, the first half (the first 'sub-layer') of the Transformer block should be implementing the following set of updates to produce an output y from an input x ,

$$y = x + \text{MultiHeadSelfAttention}(\text{RMSNorm}(x)). \quad (15)$$

Problem (transformer_block): Implement the Transformer block (3 points)

Implement the pre-norm Transformer block as described in [Section 3.4](#) and illustrated in [Figure 2](#). Your Transformer block should accept (at least) the following parameters.

d_model: `int` Dimensionality of the Transformer block inputs.

num_heads: `int` Number of heads to use in multi-head self-attention.

d_ff: `int` Dimensionality of the position-wise feed-forward inner layer.

To test your implementation, implement the adapter `[adapters.run_transformer_block]`. Then run `uv run pytest -k test_transformer_block` to test your implementation.

Deliverable: Transformer block code that passes the provided tests.

Now we put the blocks together, following the high-level diagram in [Figure 1](#). Follow our description of the embedding in [Section 3.1.0.1](#), feed this into `num_layers` Transformer blocks, and then pass that into the final layer norm and LM head to obtain an unnormalized distribution over the vocabulary (the logits).

Problem (transformer_lm): Implementing the Transformer LM (3 points)

Time to put it all together! Implement the Transformer language model as described in [Section 3.1](#) and illustrated in [Figure 1](#). At minimum, your implementation should accept all the aforementioned construction parameters for the Transformer block, as well as these additional parameters:

vocab_size: `int` The size of the vocabulary, necessary for determining the dimensionality of the token embedding matrix.

context_length: `int` The maximum context length, necessary for determining the dimensionality of the RoPE sin and cos buffer.

num_layers: `int` The number of Transformer blocks to use.

To test your implementation against our provided tests, you will first need to implement the test adapter at `[adapters.run_transformer_lm]`. Then, run `uv run pytest -k test_transformer_lm` to test your implementation.

Deliverable: A Transformer LM module that passes the above tests.

Resource accounting

It is useful to be able to understand how the various parts of the Transformer consume compute and memory. We will go through the steps to do some basic “FLOPs accounting.” The *vast* majority of FLOPS in a Transformer are matrix multiplies, so our core approach is simple:

1. Write down all the matrix multiplies in a Transformer forward pass.
2. Convert each matrix multiply into FLOPs required.

For this second step, the following fact will be useful:

Rule: Given $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{n \times p}$, the matrix-matrix product AB requires $2mnp$ FLOPs.

To see this, note that $(AB)[i, j] = A[i, :] \cdot B[:, j]$, and that this dot product requires n additions and n multiplications ($2n$ FLOPs). Then, since the matrix-matrix product AB has $m \times p$ entries, the total number of FLOPS is $(2n)(mp) = 2mnp$.

Now, before you do the next problem, it can be helpful to go through each component of your Transformer block and Transformer LM, and list out all the matrix multiplies and their associated FLOPs costs.

Problem (transformer_accounting): Transformer LM resource accounting (5 points)

- (a) Consider a GPT-2 XL-sized model using our assignment architecture, which has the following configuration:

vocab_size: 50,257

context_length: 1,024

num_layers: 48

d_model: 1,600

num_heads: 25

d_ff: 4,288 (the nearest multiple of 64 to $\frac{8}{3} \times 1,600$)

Suppose we constructed our model using this configuration. How many trainable parameters would our model have? Assuming each parameter is represented using single-precision floating point, how much memory is required to just load this model?

Deliverable: A one-to-two sentence response.

- (b) Identify the matrix multiplies required to complete a forward pass of our GPT-2 XL-shaped model. How many FLOPs do these matrix multiplies require in total? Assume that our input sequence has `context_length` tokens.

Deliverable: A list of matrix multiplies (with descriptions), and the total number of FLOPs required.

- (c) Based on your analysis above, which parts of the model require the most FLOPs?

Deliverable: A one-to-two sentence response.

- (d) Repeat your analysis with GPT-2 small (12 layers, 768 `d_model`, 12 heads), GPT-2 medium (24 layers, 1024 `d_model`, 16 heads), and GPT-2 large (36 layers, 1280 `d_model`, 20 heads). As the model size increases, which parts of the Transformer LM take up proportionally more or less of the total FLOPs?

Deliverable: For each model, provide a breakdown of model components and its associated FLOPs (as a proportion of the total FLOPs required for a forward pass). In addition, provide a one-to-two sentence description of how varying the model size changes the proportional FLOPs of each component.

- (e) Take GPT-2 XL and increase the context length to 16,384. How does the total FLOPs for one forward pass change? How does the relative contribution of FLOPs of the model components change?

Deliverable: A one-to-two sentence response.

4 Training a Transformer LM

We now have the steps to preprocess the data (via tokenizer) and the model (Transformer). What remains is to build all of the code to support training. This consists of the following:

- **Loss:** we need to define the loss function (cross-entropy).
- **Optimizer:** we need to define the optimizer to minimize this loss (AdamW).
- **Training loop:** we need all the supporting infrastructure that loads data, saves checkpoints, and manages training.

4.1 Cross-entropy loss

Recall that the Transformer language model defines a distribution $p_\theta(x_{i+1} \mid x_{1:i})$ for each sequence x of length $m + 1$ and $i = 1, \dots, m$. Given a training set D consisting of sequences of length $m + 1$, we define the standard cross-entropy (negative log-likelihood) loss function:

$$\ell(\theta; D) = \frac{1}{|D|} \sum_{x \in D} \sum_{i=1}^m -\log p_\theta(x_{i+1} \mid x_{1:i}). \quad (16)$$

(Note that a single forward pass in the Transformer yields $p_\theta(x_{i+1} \mid x_{1:i})$ for *all* $i = 1, \dots, m$.)

In particular, the Transformer computes logits $o_i \in \mathbb{R}^{\text{vocab_size}}$ for each position i , which results in:⁶

$$p(x_{i+1} \mid x_{1:i}) = \text{softmax}(o_i)[x_{i+1}] = \frac{\exp(o_i[x_{i+1}])}{\sum_{a=1}^{\text{vocab_size}} \exp(o_i[a])}. \quad (17)$$

The cross-entropy loss is generally defined with respect to the vector of logits $o_i \in \mathbb{R}^{\text{vocab_size}}$ and target x_{i+1} .⁷

⁶Note that $o_i[k]$ refers to value at index k of the vector o_i .

⁷This corresponds to the cross-entropy between the Dirac delta distribution over x_{i+1} and the predicted $\text{softmax}(o_i)$ distribution.

Implementing the cross-entropy loss requires some care with numerical issues, just like in the case of softmax.

Problem (cross_entropy): Implement cross-entropy (1 point)

Deliverable: Write a function to compute the cross-entropy loss, which takes in predicted logits (o_i) and targets (x_{i+1}) and computes the cross-entropy $\ell_i = -\log \text{softmax}(o_i)[x_{i+1}]$. Your function should handle the following:

- Subtract the largest element for numerical stability.
- Cancel out log and exp whenever possible.
- Handle any additional batch dimensions and return the *average* across the batch. As with [Section 3.2](#), we assume batch-like dimensions always come first, before the vocabulary size dimension.

Implement `[adapters.run_cross_entropy]`, then run `uv run pytest -k test_cross_entropy` to test your implementation.

Perplexity

Cross-entropy suffices for training, but when we evaluate the model, we also want to report perplexity. For a sequence of length m where we suffer cross-entropy losses $\ell_1, \dots, \ell_m$:

$$\text{perplexity} = \exp\left(\frac{1}{m} \sum_{i=1}^m \ell_i\right). \quad (18)$$

4.2 The SGD Optimizer

Now that we have a loss function, we will begin our exploration of optimizers. The simplest gradient-based optimizer is Stochastic Gradient Descent (SGD). We start with randomly initialized parameters θ_0 . Then for each step $t = 0, \dots, T - 1$, we perform the following update:

$$\theta_{t+1} \leftarrow \theta_t - \alpha_t \nabla L(\theta_t; B_t), \quad (19)$$

where B_t is a random batch of data sampled from the dataset D , and the *learning rate* α_t and *batch size* $|B_t|$ are hyperparameters.

4.2.1 Implementing SGD in PyTorch

To implement our optimizers, we will subclass the PyTorch `torch.optim.Optimizer` class. An `Optimizer` subclass must implement two methods:

def `__init__`(self, params, ...) should initialize your optimizer. Here, `params` will be a collection of parameters to be optimized (or parameter groups, in case the user wants to use different hyperparameters, such as learning rates, for different parts of the model). Make sure to pass `params` to the `__init__` method of the base class, which will store these parameters for use in `step`. You can take additional arguments depending on the optimizer (e.g., the learning rate is a common one), and pass them to the base class constructor as a dictionary, where keys are the names (strings) you choose for these parameters.

def `step`(self) should make one update of the parameters. During the training loop, this will be called after the backward pass, so you have access to the gradients on the last batch. This method should iterate through each parameter tensor `p` and modify them *in place*, i.e. setting `p.data`, which holds the

tensor associated with that parameter based on the gradient `p.grad` (if it exists), the tensor representing the gradient of the loss with respect to that parameter.

The PyTorch optimizer API has a few subtleties, so it's easier to explain it with an example. To make our example richer, we'll implement a slight variation of SGD where the learning rate decays over training, starting with an initial learning rate α and taking successively smaller steps over time:

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{t+1}} \nabla L(\theta_t; B_t) \quad (20)$$

Let's see how this version of SGD would be implemented as a PyTorch `Optimizer`:

```
from collections.abc import Callable, Iterable
from typing import Optional
import torch
import math

class SGD(torch.optim.Optimizer):
    def __init__(self, params, lr=1e-3):
        if lr < 0:
            raise ValueError(f"Invalid learning rate: {lr}")
        defaults = {"lr": lr}
        super().__init__(params, defaults)

    def step(self, closure: Optional[Callable] = None):
        loss = None if closure is None else closure()
        for group in self.param_groups:
            lr = group["lr"] # Get the learning rate.
            for p in group["params"]:
                if p.grad is None:
                    continue

                state = self.state[p] # Get state associated with p.
                t = state.get("t", 0) # Get iteration number from the state, or 0.
                grad = p.grad.data # Get the gradient of loss with respect to p.
                p.data -= lr / math.sqrt(t + 1) * grad # Update weight tensor in-place.
                state["t"] = t + 1 # Increment iteration number.

        return loss
```

In `__init__`, we pass the parameters, as well as default hyperparameters, to the base class constructor (the parameters might come in groups, each with different hyperparameters). In case the parameters are just a single collection of `torch.nn.Parameter` objects, the base constructor will create a single group and assign it the default hyperparameters. Then, in `step`, we iterate over each parameter group, then over each parameter in that group, and apply [Equation 20](#). Here, we keep the iteration number as a state associated with each parameter: we first read this value, use it in the gradient update, and then update it. The API specifies that the user might pass in a callable `closure` to re-compute the loss before the optimizer step. We won't need this for the optimizers we'll use, but we add it to comply with the API.

To see this working, we can use the following minimal example of a *training loop*:

```
weights = torch.nn.Parameter(5 * torch.randn((10, 10)))
opt = SGD([weights], lr=1)

for t in range(100):
    opt.zero_grad() # Reset the gradients for all learnable parameters.
    loss = (weights**2).mean() # Compute a scalar loss value.
    print(loss.cpu().item())
```

```
loss.backward() # Run backward pass, which computes gradients.
opt.step() # Run optimizer step.
```

This is the typical structure of a training loop: in each iteration, we will compute the loss and run a step of the optimizer. When training language models, our learnable parameters will come from the model (in PyTorch, `m.parameters()` gives us this collection). The loss will be computed over a sampled batch of data, but the basic structure of the training loop will be the same.

Problem (`learning_rate_tuning`): Tuning the learning rate (1 point)

As we will see, one of the hyperparameters that affects training the most is the learning rate. Let's see that in practice in our toy example. Run the SGD example above with three other values for the learning rate: `1e1`, `1e2`, and `1e3`, for just 10 training iterations. What happens with the loss for each of these learning rates? Does it decay faster, slower, or does it diverge (i.e., increase over the course of training)?

Deliverable: A one-to-two sentence response with the behaviors you observed.

4.3 AdamW

Modern language models are typically trained with more sophisticated optimizers, instead of SGD. Most optimizers used recently are derivatives of the Adam optimizer [D. P. Kingma et al., 2015]. We will use AdamW [I. Loshchilov et al., 2019], which is in wide use in recent work. AdamW proposes a modification to Adam that improves regularization by adding *weight decay* (at each iteration, we pull the parameters towards 0), in a way that is decoupled from the gradient update. We will implement AdamW as described in algorithm 2 of [I. Loshchilov et al. [23]].

AdamW is *stateful*: for each parameter, it keeps track of a running estimate of its first and second moments. Thus, AdamW uses additional memory in exchange for improved stability and convergence. Besides the learning rate α , AdamW has a pair of hyperparameters (β_1, β_2) that control the updates to the moment estimates, and a weight decay rate λ . Typical applications set (β_1, β_2) to $(0.9, 0.999)$, but large language models like LLaMA [H. Touvron et al., 2023] and GPT-3 [T. B. Brown et al., 2020] are often trained with $(0.9, 0.95)$. The algorithm can be written as follows, where ε is a small value (e.g., 10^{-8}) used to improve numerical stability in case we get extremely small values in v :

Algorithm 1: AdamW Optimizer

```
1 init( $\theta$ )  ▷ Initialize learnable parameters
2  $m \leftarrow 0$   ▷ Initial value of the first moment vector; same shape as  $\theta$ 
3  $v \leftarrow 0$   ▷ Initial value of the second moment vector; same shape as  $\theta$ 
4 for  $t = 1, \dots, T$  do
5   Sample batch of data  $B_t$ 
6    $g \leftarrow \nabla_{\theta} \ell(\theta; B_t)$   ▷ Compute the gradient of the loss
7    $\alpha_t \leftarrow \alpha \frac{\sqrt{1-\beta_2^t}}{1-\beta_1^t}$   ▷ Compute adjusted  $\alpha$  for iteration  $t$ 
8    $\theta \leftarrow \theta - \alpha \lambda \theta$   ▷ Apply weight decay
9    $m \leftarrow \beta_1 m + (1 - \beta_1) g$   ▷ Update the first moment estimate
10   $v \leftarrow \beta_2 v + (1 - \beta_2) g^2$   ▷ Update the second moment estimate
11   $\theta \leftarrow \theta - \alpha_t \frac{m}{\sqrt{v+\epsilon}}$   ▷ Apply moment-adjusted weight updates
12 end for
```

Algorithm 1: AdamW optimizer pseudocode.

Note that t starts at 1. You will now implement this optimizer.

Problem (adamw): Implement AdamW (2 points)

Deliverable: Implement the AdamW optimizer as a subclass of `torch.optim.Optimizer`. Your class should take the learning rate α in `__init__`, as well as the β , ϵ and λ hyperparameters. To help you keep state, the base `Optimizer` class gives you a dictionary `self.state`, which maps `nn.Parameter` objects to a dictionary that stores any information you need for that parameter (for AdamW, this would be the moment estimates). Implement `[adapters.get_adamw_cls]` and make sure it passes `uv run pytest -k test_adamw`.

Problem (adamw_accounting): Resource accounting for training with AdamW (2 points)

Let us compute how much memory and compute running AdamW requires. Assume we are using float32 for every tensor.

- (a) How much peak memory does running AdamW require? Decompose your answer based on the memory usage of the parameters, activations, gradients, and optimizer state. Express your answer in terms of the `batch_size` and the model hyperparameters (`vocab_size`, `context_length`, `num_layers`, `d_model`, `num_heads`). Assume $d_{\text{ff}} = \frac{8}{3} \times d_{\text{model}}$.

For simplicity, when calculating memory usage of activations, consider only the following components:

- Transformer block
 - ▷ RMSNorm(s)
 - ▷ Multi-head self-attention sublayer: QKV projections, $QK^{\top}$ matrix multiply, softmax, weighted sum of values, output projection.
 - ▷ Position-wise feed-forward (SwiGLU): W_1 , W_2 , SiLU on the gate branch, element-wise product, W_3

- final RMSNorm
- output embedding
- cross-entropy on logits

Deliverable: An algebraic expression for each of parameters, activations, gradients, and optimizer state, as well as the total.

- (b) Instantiate your answer for a GPT-2 XL-shaped model to get an expression that only depends on the `batch_size`. What is the maximum batch size you can use and still fit within 80GB memory?

Deliverable: An expression that looks like $a \cdot \text{batch_size} + b$ for numerical values a, b , and a number representing the maximum batch size.

- (c) How many FLOPs does running one step of AdamW take?

Deliverable: An algebraic expression, with a brief justification.

- (d) Model FLOPs utilization (MFU) is defined as the ratio of observed throughput (tokens per second) relative to the hardware’s theoretical peak FLOP throughput [A. Chowdhery et al., 2022]. An NVIDIA H100 GPU has a theoretical peak of 495 teraFLOP/s for “float32” (actually TensorFloat-32, which in reality is “bfloat19”) operations. Assuming you are able to get 50% MFU, how long would it take to train a GPT-2 XL for 400K steps and a batch size of 1024 on a single H100? Following [J. Kaplan et al. [25]] and [J. Hoffmann et al. [26]], assume that the backward pass has twice the FLOPs of the forward pass.

Deliverable: The number of hours training would take, with a brief justification.

4.4 Learning rate scheduling

The value for the learning rate that leads to the quickest decrease in loss often varies during training. In training Transformers, it is typical to use a learning rate *schedule*, where we start with a bigger learning rate, making quicker updates in the beginning, and slowly decay it to a smaller value as the model trains.⁸ In this assignment, we will implement the cosine annealing schedule used to train LLaMA [H. Touvron et al., 2023].

A scheduler is simply a function that takes the current step t and other relevant parameters (such as the initial and final learning rates), and returns the learning rate to use for the gradient update at step t . The simplest schedule is the constant function, which will return the same learning rate given any t .

The cosine annealing learning rate schedule takes (i) the current iteration t , (ii) the maximum learning rate $\alpha_{\max}$, (iii) the minimum (final) learning rate $\alpha_{\min}$, (iv) the number of *warm-up* iterations T_w , and (v) the final iteration of cosine annealing T_c . The learning rate at iteration t is defined as:

(Warm-up) If $t < T_w$, then $\alpha_t = \frac{t}{T_w} \alpha_{\max}$.

(Cosine annealing) If $T_w \leq t \leq T_c$, then $\alpha_t = \alpha_{\min} + \frac{1}{2} \left(1 + \cos \left(\frac{t - T_w}{T_c - T_w} \pi \right) \right) (\alpha_{\max} - \alpha_{\min})$.

(Post-annealing) If $t > T_c$, then $\alpha_t = \alpha_{\min}$.

Problem (learning_rate_schedule): Implement cosine learning rate schedule with warmup (1 point)

⁸It’s sometimes common to use a schedule where the learning rate rises back up (restarts) to help get past local minima.

Write a function that takes t , $\alpha_{\max}$, $\alpha_{\min}$, T_w and T_c , and returns the learning rate α_t according to the scheduler defined above. Then implement `[adapters.get_lr_cosine_schedule]` and make sure it passes `uv run pytest -k test_get_lr_cosine_schedule`.

4.5 Gradient clipping

During training, we can sometimes hit training examples that yield large gradients, which can destabilize training. To mitigate this, one technique often employed in practice is *gradient clipping*. The idea is to enforce a limit on the norm of the gradient after each backward pass before taking an optimizer step.

Given the gradient (for all parameters) g , we compute its ℓ_2 -norm $\|g\|_2$. If this norm is less than a maximum value M , then we leave g as is; otherwise, we scale g down by a factor of $\frac{M}{\|g\|_2 + \varepsilon}$ (where a small ε , like 10^{-6} , is added for numeric stability). Note that the resulting norm will be just under M .

Problem (gradient_clipping): Implement gradient clipping (1 point)

Write a function that implements gradient clipping. Your function should take a list of parameters and a maximum ℓ_2 -norm. It should modify each parameter gradient in place. Use $\varepsilon = 10^{-6}$ (the PyTorch default). Then, implement the adapter `[adapters.run_gradient_clipping]` and make sure it passes `uv run pytest -k test_gradient_clipping`.

5 Training loop

We will now finally put together the major components we've built so far: the tokenized data, the model, and the optimizer.

5.1 Data Loader

The tokenized data (e.g., that you prepared in `tokenizer_experiments`) is a single sequence of tokens $x = (x_1, \dots, x_n)$. Even though the source data might consist of separate documents (e.g., different web pages, or source code files), a common practice is to concatenate all of those into a single sequence of tokens, adding a delimiter between them (such as the `<|endoftext|>` token).

A *data loader* turns this into a stream of *batches*, where each batch consists of B sequences of length m , paired with the corresponding next tokens, also with length m . For example, for $B = 1, m = 3$, $([x_2, x_3, x_4], [x_3, x_4, x_5])$ would be one potential batch.

Loading data in this way simplifies training for a number of reasons. First, any $1 \leq i \leq n - m$ gives a valid training sequence, so sampling training sequences is trivial. Since all training sequences have the same length, there's no need to pad input sequences, which improves hardware utilization (also by increasing batch size B). Finally, we also don't need to load the full dataset to sample training data, making it easy to handle large datasets that might not otherwise fit in memory.

Problem (data_loading): Implement data loading (2 points)

Deliverable: Write a function that takes a numpy array x (integer array with token IDs), a `batch_size`, a `context_length` and a PyTorch device string (e.g., `'cpu'` or `'cuda:0'`), and returns a pair of tensors: the sampled input sequences and the corresponding next-token targets. Both

tensors should have shape `(batch_size, context_length)` containing token IDs, and both should be placed on the requested device. To test your implementation against our provided tests, you will first need to implement the test adapter at `[adapters.run_get_batch]`. Then, run `uv run pytest -k test_get_batch` to test your implementation.

Low-Resource Tip: Data loading on CPU or Apple Silicon

If you are planning to train your LM on CPU or Apple Silicon, you need to move your data to the correct device (and similarly, you should use the same device for your model later on).

If you are on CPU, you can use the `'cpu'` device string, and on Apple Silicon (M* chips), you can use the `'mps'` device string.

For more on MPS, check out these resources:

- <https://docs.pytorch.org/docs/stable/mps.html>
- <https://docs.pytorch.org/docs/stable/notes/mps.html>
- <https://developer.apple.com/documentation/metalperformanceshaders>

What if the dataset is too big to load into memory? We can use a Unix system call named `mmap` which maps a file on disk to virtual memory, and lazily loads the file contents when that memory location is accessed. Thus, you can “pretend” you have the entire dataset in memory. Numpy implements this through `np.memmap` (or the flag `mmap_mode='r'` to `np.load`, if you originally saved the array with `np.save`), which will return a numpy array-like object that loads the entries on-demand as you access them. **When sampling from your dataset (i.e., a numpy array) during training, be sure to load the dataset in memory-mapped mode** (via `np.memmap` or the flag `mmap_mode='r'` to `np.load`, depending on how you saved the array). Make sure you also specify a `dtype` that matches the array that you’re loading. It may be helpful to explicitly verify that the memory-mapped data looks correct (e.g., doesn’t contain values beyond the expected vocabulary size).

5.2 Checkpointing

In addition to loading data, we will also need to save models as we train. When running jobs, we often want to be able to resume a training run that stopped midway through (e.g., due to your job timing out, machine failure, etc). Even when all goes well, we might also want to later have access to intermediate models (e.g., to study training dynamics post-hoc, take samples from models at different stages of training, etc).

A checkpoint should have all the states that we need to resume training. We of course want to be able to restore model weights at a minimum. If using a stateful optimizer (such as AdamW), we will also need to save the optimizer’s state (e.g., in the case of AdamW, the moment estimates). Finally, to resume the learning rate schedule, we will need to know the iteration number we stopped at. PyTorch makes it easy to save all of these: every `nn.Module` has a `state_dict()` method that returns a dictionary with all learnable weights; we can restore these weights later with the sister method `load_state_dict()`. The same goes for any `torch.optim.Optimizer`. Finally, `torch.save(obj, dest)` can dump an object (e.g., a dictionary containing tensors as some values, but also regular Python objects like integers) to a file (path) or file-like object, which can then be loaded back into memory with `torch.load(src)`.

Problem (checkpointing): Implement model checkpointing (1 point)

Implement the following two functions to load and save checkpoints:

def save_checkpoint(model, optimizer, iteration, out) should dump all the state from the model, optimizer and iteration into the file-like object `out`. You can use the `state_dict` method of both the model and the optimizer to get their relevant states and use `torch.save(obj, out)` to dump `obj` into `out` (PyTorch supports either a path or a file-like object here). A typical choice is to have `obj` be a dictionary, but you can use whatever format you want as long as you can load your checkpoint later.

This function expects the following parameters:

model: `torch.nn.Module`
optimizer: `torch.optim.Optimizer`
iteration: `int`
out: `str` | `os.PathLike` | `typing.BinaryIO` | `typing.IO[bytes]`

def load_checkpoint(src, model, optimizer) should load a checkpoint from `src` (path or file-like object), and then recover the model and optimizer states from that checkpoint. Your function should return the iteration number that was saved to the checkpoint. You can use `torch.load(src)` to recover what you saved in your `save_checkpoint` implementation, and the `load_state_dict` method in both the model and optimizer to return them to their previous states.

This function expects the following parameters:

src: `str` | `os.PathLike` | `typing.BinaryIO` | `typing.IO[bytes]`
model: `torch.nn.Module`
optimizer: `torch.optim.Optimizer`

Implement the `[adapters.run_save_checkpoint]` and `[adapters.run_load_checkpoint]` adapters, and make sure they pass `uv run pytest -k test_checkpointing`.

5.3 Training loop

Now, it's finally time to put all of the components you implemented together into your main training script. It will pay off to make it easy to start training runs with different hyperparameters (e.g., by taking them as command-line arguments), since you will be doing these many times later to study how different choices impact training.

Problem (training_together): Put it together (4 points)

Deliverable: Write a script that runs a training loop to train your model on user-provided input. In particular, we recommend that your training script allow for (at least) the following:

- Ability to configure and control the various model and optimizer hyperparameters.
- Memory-efficient loading of large training and validation datasets with `np.memmap`.
- Serializing checkpoints to a user-provided path.

- Periodically logging training and validation performance (e.g., to console and/or an external service like Weights and Biases).⁹

6 Generating text

Now that we can train models, the last piece we need is the ability to generate text from our model. Recall that a language model takes in a (possibly batched) integer sequence of length `sequence_length` and produces a matrix of size `(sequence_length, vocab_size)`, where each element of the sequence is a probability distribution predicting the next token after that position. We will now write a few functions to turn this into a sampling scheme for new sequences.

Softmax

By standard convention, the language model output is the output of the final linear layer (the “logits”) and so we have to turn this into a normalized probability via the *softmax* operation, which we saw earlier in [Equation 10](#)

Decoding

To generate text (decode) from our model, we will provide the model with a sequence of prefix tokens (the “prompt”), and ask it to produce a probability distribution over the vocabulary that predicts the next token in the sequence. Then, we will sample from this distribution over the vocabulary items to determine the next output token.

Concretely, one step of the decoding process should take in a sequence $x_{1...t}$ and return a token x_{t+1} via the following equation,

$$P(x_{t+1} = i \mid x_{1...t}) = \frac{\exp(v_i)}{\sum_j \exp(v_j)} \quad (21)$$

$$v = \text{TransformerLM}(x_{1...t})_t \in \mathbb{R}^{\text{vocab_size}} \quad (22)$$

where `TransformerLM` is our model which takes as input a sequence of length `sequence_length` and produces a matrix of size `(sequence_length, vocab_size)`, and we take the last element of this matrix, as we are looking for the next token prediction at the t -th position.

This gives us a basic decoder by repeatedly sampling from these one-step conditionals (appending our previously-generated output token to the input of the next decoding timestep) until we generate the end-of-sequence token `<|endoftext|>` (or a user-specified maximum number of tokens to generate).

Decoder tricks

We will be experimenting with small models, and small models can sometimes generate very low-quality texts. Two simple decoder tricks can help fix these issues. First, in *temperature scaling* we modify our softmax with a temperature parameter τ , where the new softmax is

$$\text{softmax}(v, \tau)_i = \frac{\exp(v_i/\tau)}{\sum_{j=1}^{\text{vocab_size}} \exp(v_j/\tau)}. \quad (23)$$

Note how setting $\tau \rightarrow 0$ makes it so that the largest element of v dominates, and the output of the softmax becomes a one-hot vector concentrated at this maximal element.

⁹wandb.ai

Second, another trick is *nucleus* or *top-p* sampling, where we modify the sampling distribution by truncating low-probability tokens. Let q be a probability distribution that we get from a (temperature-scaled) softmax of size `vocab_size`. Nucleus sampling with hyperparameter p produces the next token according to the equation

$$P(x_{t+1} = i \mid q) = \begin{cases} \frac{q_i}{\sum_{j \in V(p)} q_j} & \text{if } i \in V(p) \\ 0 & \text{otherwise} \end{cases} \quad (24)$$

where $V(p)$ is the *smallest* set of indices such that $\sum_{j \in V(p)} q_j \geq p$. You can compute this quantity easily by first sorting the probability distribution q by magnitude, and selecting the largest vocabulary elements until you reach the target level of p .

Problem (decoding): Decoding (3 points)

Deliverable: Implement a function to decode from your language model. We recommend that you support the following features:

- Generate completions for a user-provided prompt (i.e., take in some $x_{1...t}$ and sample a completion until you hit an `<|endoftext|>` token).
- Allow the user to control the maximum number of generated tokens.
- Given a desired temperature value, apply softmax temperature scaling to the predicted next-token distributions before sampling.
- Top- p sampling ([A. Holtzman et al., 2020] also referred to as nucleus sampling), given a user-specified threshold value.

7 Experiments

Now it is time to put everything together and train (small) language models on a pretraining dataset.

7.1 How to Run Experiments and Deliverables

The best way to understand the rationale behind the architectural components of a Transformer is to actually modify it and run it yourself. There is no substitute for hands-on experience.

To this end, it's important to be able to experiment **quickly, consistently, and keep records** of what you did. To experiment quickly, we will be running many experiments on a small-scale model (about 17M total parameters) and simple dataset (TinyStories). To do things consistently, you will ablate components and vary hyperparameters in a systematic way, and to keep records we will ask you to submit a log of your experiments and learning curves associated with each experiment.

To make it possible to submit loss curves, **make sure to periodically evaluate validation losses and record both the number of steps and wall-clock times**. You might find logging infrastructure such as Weights and Biases helpful.

Problem (experiment_log): Experiment logging (3 points)

For your training and evaluation code, create experiment tracking infrastructure that allows you to track your experiments and loss curves with respect to gradient steps and wall-clock time.

Deliverable: Logging infrastructure code for your experiments and an experiment log (a document of all the things you tried) for the assignment problems below in this section.

7.2 TinyStories

We are going to start with a very simple dataset (TinyStories; [R. Eldan et al. \[1\]](#)) where models will train quickly, and we can see some interesting behaviors. The instructions for getting this dataset are in [Section 1](#). An example of what this dataset looks like is below.

Example (`tinystories_example`): One example from TinyStories

Once upon a time there was a little boy named Ben. Ben loved to explore the world around him. He saw many amazing things, like beautiful vases that were on display in a store. One day, Ben was walking through the store when he came across a very special vase. When Ben saw it he was amazed! He said, "Wow, that is a really amazing vase! Can I buy it?" The shopkeeper smiled and said, "Of course you can. You can take it home and show all your friends how amazing it is!" So Ben took the vase home and he was so proud of it! He called his friends over and showed them the amazing vase. All his friends thought the vase was beautiful and couldn't believe how lucky Ben was. And that's how Ben found an amazing vase in the store!

7.2.1 Hyperparameter tuning

We will tell you some very basic hyperparameters to start with and ask you to find some settings for others that work well.

Vocab size 10000. Typical vocabulary sizes are in the tens to hundreds of thousands. You should vary this and see how the vocabulary and model behavior change.

Context length 256. Simple datasets such as TinyStories might not need long sequence lengths, but for the later OpenWebText data, you may want to vary this. Try varying this and seeing the impact on both the per-iteration runtime and the final perplexity.

d_model 512. This is slightly smaller than the 768 dimensions used in many small Transformer papers, but this will make things faster.

d_ff 1344. This is roughly $\frac{8}{3}d_{\text{model}}$ while being a multiple of 64, which is good for GPU performance.

RoPE theta parameter Θ 10000.

Number of layers and heads 4 layers, 16 heads. Together, this will give about 17M non-embedding parameters which is a fairly small Transformer.

Total tokens processed 327,680,000 (your batch size $\times$ total step count $\times$ context length should equal roughly this value).

You should do some trial and error to find good defaults for the following other hyperparameters: learning rate, learning rate warmup, other AdamW hyperparameters ($\beta_1, \beta_2, \varepsilon$), and weight decay. You can find some typical choices of such hyperparameters in [D. P. Kingma et al. \[22\]](#).

7.2.2 Putting it together

Now you can put everything together by getting a trained BPE tokenizer, tokenizing the training dataset, and running this in the training loop that you wrote. **Important note:** If your implementation is correct and efficient, the above hyperparameters should result in a roughly 20–30 minute runtime on 1 B200

GPU. If you have runtimes that are much longer, please check and make sure your dataloading, checkpointing, or validation loss code is not bottlenecking your runtimes and that your implementation is properly batched.

7.2.3 Tips and tricks for debugging model architectures

We highly recommend getting comfortable with your IDE's built-in debugger (e.g., VSCode/Zed), which will save you time compared to debugging with print statements. If you use a text editor, you can use something like `ipdb`. A few other good practices when debugging model architectures are:

- A common first step when developing any neural net architecture is to overfit to a single minibatch. If your implementation is correct, you should be able to quickly drive the training loss to near-zero.
- Set debug breakpoints in various model components, and inspect the shapes of intermediate tensors to make sure they match your expectations.
- Monitor the norms of activations, model weights, and gradients to make sure they are not exploding or vanishing.

Problem (**learning_rate**): Tune the learning rate (2 B200 hrs) (3 points)

The learning rate is one of the most important hyperparameters to tune. Taking the base model you've trained, answer the following questions:

- (a) Perform a hyperparameter sweep over the learning rates and report the final losses (or note divergence if the optimizer diverges).

Deliverable: Learning curves associated with multiple learning rates. Explain your hyperparameter search strategy.

Deliverable: A model with validation loss (per-token) on TinyStories of at most 1.45

Low-Resource Tip: Train for a few steps on CPU or Apple Silicon

If you are running on `cpu` or `mps`, you should instead reduce the total tokens processed count to 40,000,000, which will be sufficient to produce reasonably fluent text. You may also increase the target validation loss from 1.45 to 2.00.

Running our solution code with a tuned learning rate on an M4 Max chip and 36 GB of RAM, we use $\text{batch size} \times \text{total step count} \times \text{context length} = 32 \times 5000 \times 256 = 40,960,000$ tokens, which takes 1 hour and 22 minutes on `cpu` and 36 minutes on `mps`. At step 5000, we achieve a validation loss of 1.80.

Some additional tips:

- When using N training steps, we suggest adjusting the cosine learning rate decay schedule to terminate its decay (i.e., reach the minimum learning rate) at precisely step N .
- When using `mps`, do **not** use TF32 kernels, i.e., do **not** set

```
torch.set_float32_matmul_precision('high')
```

as you might with `cuda` devices. We tried enabling TF32 kernels with `mps` (torch version 2.9.0) and found the backend sometimes uses silently broken kernels that cause unstable training.

- You can speed up training by JIT-compiling your model with `torch.compile`. Specifically:

- On `cpu`, compile your model with

```
model = torch.compile(model)
```

- On `mps`, you can somewhat optimize the backward pass using

```
model = torch.compile(model, backend="aot_eager")
```

Compilation with Inductor is not supported on `mps` as of `torch` version 2.9.0.

- (b) Folk wisdom is that the best learning rate is “at the edge of stability.” Investigate how the point at which learning rates diverge is related to your best learning rate.

Deliverable: Learning curves of increasing learning rate which include at least one divergent run and an analysis of how this relates to convergence rates.

Now let’s vary the batch size and see what happens to training. Batch sizes are important – they let us get higher efficiency from our GPUs by doing larger matrix multiplies, but is it true that we always want batch sizes to be large? Let’s run some experiments to find out.

Problem (`batch_size_experiment`): Batch size variations (1 B200 hr) (1 point)

Vary your batch size all the way from 1 to the GPU memory limit. Try at least a few batch sizes in between, including typical sizes like 64 and 128.

Deliverable: Learning curves for runs with different batch sizes. The learning rates should be optimized again if necessary.

Deliverable: A few sentences discussing your findings on batch sizes and their impacts on training.

With your decoder in hand, we can now generate text! We will generate from the model and see how good it is. As a reference, you should get outputs that look at least as good as the example below.

Example (`ts_generate_example`): Sample output from a TinyStories language model

Once upon a time, there was a pretty girl named Lily. She loved to eat gum, especially the big black one. One day, Lily’s mom asked her to help cook dinner. Lily was so excited! She loved to help her mom. Lily’s mom made a big pot of soup for dinner. Lily was so happy and said, “Thank you, Mommy! I love you.” She helped her mom pour the soup into a big bowl. After dinner, Lily’s mom made some yummy soup. Lily loved it! She said, “Thank you, Mommy! This soup is so yummy!” Her mom smiled and said, “I’m glad you like it, Lily.” They finished cooking and continued to cook together. The end.

Low-Resource Tip: Generate text on CPU or Apple Silicon

If instead you used the low-resource configuration with 40M tokens processed, you should see generations that still resemble English but are not as fluent as above. For example, our sample output from a TinyStories language model trained on 40M tokens is below:

Once upon a time, there was a little girl named Sue. Sue had a tooth that she loved very much. It was his best head. One day, Sue went for a walk and met a ladybug! They became good friends and played on the path together.

“Hey, Polly! Let’s go out!” said Tim. Sue looked at the sky and saw that it was difficult to find a way to dance shining. She smiled and agreed to help the talking!”

As Sue watched the sky moved, what it was. She

Here is the precise problem statement and what we ask for:

Problem (generate): Generate text (1 point)

Using your decoder and your trained checkpoint, report the text generated by your model. You may need to manipulate decoder parameters (temperature, top-p, etc.) to get fluent outputs.

Deliverable: Text dump of at least 256 tokens of text (or until the first `<|endoftext|>` token), and a brief comment on the fluency of this output and at least two factors which affect how good or bad this output is.

7.3 Ablations and architecture modification

The best way to understand the Transformer is to actually modify it and see how it behaves. We will now do a few simple ablations and modifications.

Ablation 1: layer normalization

It is often said that layer normalization is important for the stability of Transformer training. But perhaps we want to live dangerously. Let’s remove RMSNorm from each of our Transformer blocks and see what happens.

Problem (layer_norm_ablation): Remove RMSNorm and train (0.5 B200 hrs) (1 point)

Remove all of the RMSNorms from your Transformer and train. What happens at the previous optimal learning rate? Can you get stability by using a lower learning rate?

Deliverable: A learning curve for when you remove RMSNorms and train, as well as a learning curve for the best learning rate.

Deliverable: A few sentences of commentary on the impact of RMSNorm.

Let’s now investigate another layer normalization choice that seems arbitrary at first glance. *Pre-norm* Transformer blocks are defined as

$$z = x + \text{MultiHeadSelfAttention}(\text{RMSNorm}(x)) \quad (25)$$

$$y = z + \text{FFN}(\text{RMSNorm}(z)). \quad (26)$$

This is one of the few ‘consensus’ modifications to the original Transformer architecture, which used a *post-norm* approach as

$$z = \text{RMSNorm}(x + \text{MultiHeadSelfAttention}(x)) \quad (27)$$

$$y = \text{RMSNorm}(z + \text{FFN}(z)). \quad (28)$$

Let’s revert back to the *post-norm* approach and see what happens.

Problem (pre_norm_ablation): Implement post-norm and train (0.5 B200 hrs) (1 point)

Modify your pre-norm Transformer implementation into a post-norm one. Train with the post-norm model and see what happens.

Deliverable: A learning curve for a post-norm Transformer, compared to the pre-norm one.

We see that layer normalization has a major impact on the behavior of the Transformer, and that even the position of the layer normalization is important.

Ablation 2: position embeddings

We will next investigate the impact of the position embeddings on the performance of the model. Specifically, we will compare our base model (with RoPE) with not including position embeddings at all (NoPE). It turns out that decoder-only transformers, i.e., those with a causal mask as we have implemented, can in theory infer relative or absolute position information without being provided with position embeddings explicitly [Y.-H. H. Tsai et al., 2019; A. Kazemnejad et al., 2023]. We will now test empirically how NoPE performs compared to RoPE.

Problem (no_pos_emb): Implement NoPE (0.5 B200 hrs) (1 point)

Modify your Transformer implementation with RoPE to remove the position embedding information entirely, and see what happens.

Deliverable: A learning curve comparing the performance of RoPE and NoPE.

Ablation 3: SwiGLU vs. SiLU

Next, we will follow [N. Shazeer [20]] and test the importance of gating in the feed-forward network, by comparing the performance of SwiGLU feed-forward networks versus feed-forward networks using SiLU activations but no gated linear unit (GLU):

$$\text{FFN}_{\text{SiLU}}(x) = W_2 \text{SiLU}(W_1 x). \quad (29)$$

Recall that in our SwiGLU implementation, we set the dimensionality of the inner feed-forward layer to be roughly $d_{\text{ff}} = \frac{8}{3}d_{\text{model}}$ (while ensuring that $d_{\text{ff}} \bmod 64 = 0$, to make use of GPU tensor cores). In this ablation baseline, your FFN_{SiLU} implementation should instead set $d_{\text{ff}} = 4 \times d_{\text{model}}$, to approximately match the parameter count of the default SwiGLU feed-forward network (which has three instead of two weight matrices).

Problem (swiglu_ablation): SwiGLU vs. SiLU (0.5 B200 hrs) (1 point)

Deliverable: A learning curve comparing the performance of SwiGLU and SiLU feed-forward networks, with approximately matched parameter counts.

Deliverable: A few sentences discussing your findings.

Low-Resource Tip: Online students with limited GPU resources should test modifications on TinyStories

In the remainder of the assignment, we will move to a larger-scale, noisier web dataset (OpenWebText), experimenting with architecture modifications and (optionally) making a submission to the course leaderboard.

It takes a long time to train an LM to fluency on OpenWebText, so we suggest that online students with limited GPU access continue testing modifications on TinyStories (using validation loss as a metric to evaluate performance).

7.4 Running on OpenWebText

We will now move to a more standard pretraining dataset created from a web crawl. A small sample of OpenWebText [A. Gokaslan et al., 2019] is also provided as a single text file: see [Section 1] for how to access this file.

Here is an example from OpenWebText. Note how the text is much more realistic, complex, and varied. You may want to look through the training dataset to get a sense of what training data looks like for a web-scraped corpus.

Example (owt_example): One example from OWT

Baseball Prospectus director of technology Harry Pavlidis took a risk when he hired Jonathan Judge.

Pavlidis knew that, as Alan Schwarz wrote in *The Numbers Game*, “no corner of American culture is more precisely counted, more passionately quantified, than performances of baseball players.” With a few clicks here and there, you can find out that Noah Syndergaard’s fastball revolves more than 2,100 times per minute on its way to the plate, that Nelson Cruz had the game’s highest average exit velocity among qualified hitters in 2016 and myriad other tidbits that seem ripped from a video game or science fiction novel. The rising ocean of data has empowered an increasingly important actor in baseball’s culture: the analytical hobbyist.

That empowerment comes with added scrutiny – on the measurements, but also on the people and publications behind them. With Baseball Prospectus, Pavlidis knew all about the backlash that accompanies quantitative imperfection. He also knew the site’s catching metrics needed to be reworked, and that it would take a learned mind – someone who could tackle complex statistical modeling problems – to complete the job.

“He freaks us out.” Harry Pavlidis

Pavlidis had a hunch that Judge “got it” based on the latter’s writing and their interaction at a site-sponsored ballpark event. [...]

Note: You may have to re-tune your hyperparameters such as learning rate or batch size for this experiment.

Problem (main_experiment): Experiment on OWT (2 B200 hrs) (2 points)

Train your language model on OpenWebText with the same model architecture and total training iterations as TinyStories. How well does this model do?

Deliverable: A learning curve of your language model on OpenWebText. Describe the difference in losses from TinyStories – how should we interpret these losses?

Deliverable: Generated text from OpenWebText LM, in the same format as the TinyStories outputs. How is the fluency of this text? Why is the output quality worse even though we have the same model and compute budget as TinyStories?

7.5 Your own modification + leaderboard

Congratulations on getting to this point. You’re almost done! You will now try to improve upon the Transformer architecture, and see how your hyperparameters and architecture stack up against other students in the class.

Rules for the leaderboard

There are no restrictions other than the following:

Runtime: Your submission can run for at most 45 minutes on a B200. You might want to enforce this in your submission script if you use either SLURM or Modal.

Data: You may only use the OpenWebText training dataset that we provide.

Otherwise, you are free to do whatever your heart desires.

If you are looking for some ideas on what to implement, you can check out some of these resources:

- State-of-the-art open-source LLM families, such as Llama 3 [A. Grattafiori et al., 2024] or Qwen 2.5 [A. Yang et al., 2024].
- The NanoGPT speedrun repository (github.com/KellerJordan/modded-nanogpt), where community members post many interesting modifications for “speedrunning” small-scale language model pretraining. For example, a common modification that dates back to the original Transformer paper is to tie the weights of the input and output embeddings together (see [A. Vaswani et al. [8]] (Section 3.4) and [A. Chowdhery et al. [16]] (Section 2)). If you do try weight tying, you may have to decrease the standard deviation of the embedding/LM head init.

You will want to test these on either a small subset of OpenWebText or on TinyStories before trying the full 45-minute run.

As a caveat, we do note that some of the modifications you may find working well in this leaderboard may not generalize to larger-scale pretraining. We will explore this idea further in the scaling laws unit of the course.

Problem (leaderboard): Leaderboard (10 B200 hrs) (6 points)

You will train a model under the leaderboard rules above with the goal of minimizing the validation loss of your language model within 0.75 B200-hours.

Deliverable: The final validation loss that was recorded, an associated learning curve that clearly shows a wall-clock-time x-axis that is less than 45 minutes, and a description of what you did. We expect a leaderboard submission to beat at least the naive baseline of a 5.0 loss. Submit to the leaderboard here: github.com/stanford-cs336/assignment1-basics-leaderboard.

Bibliography

- [1] R. Eldan and Y. Li, “TinyStories: How Small Can Language Models Be and Still Speak Coherent English?.” 2023.
- [2] A. Gokaslan, V. Cohen, E. Pavlick, and S. Tellex, “OpenWebText corpus.” 2019.
- [3] R. Sennrich, B. Haddow, and A. Birch, “Neural Machine Translation of Rare Words with Subword Units,” in *Proc. of ACL*, 2016.
- [4] C. Wang, K. Cho, and J. Gu, “Neural Machine Translation with Byte-Level Subwords.” 2019.
- [5] P. Gage, “A new algorithm for data compression,” *C Users Journal*, vol. 12, no. 2, pp. 23–38, Feb. 1994.
- [6] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, “Language Models are Unsupervised Multitask Learners.” 2019.
- [7] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, “Improving Language Understanding by Generative Pre-Training.” 2018.
- [8] A. Vaswani *et al.*, “Attention is All you Need,” in *Proc. of NeurIPS*, 2017.
- [9] T. Q. Nguyen and J. Salazar, “Transformers without Tears: Improving the Normalization of Self-Attention,” in *Proc. of IWSWLT*, 2019.
- [10] R. Xiong *et al.*, “On Layer Normalization in the Transformer Architecture,” in *Proc. of ICML*, 2020.
- [11] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization.” 2016.
- [12] H. Touvron *et al.*, “LLaMA: Open and Efficient Foundation Language Models.” 2023.
- [13] B. Zhang and R. Sennrich, “Root Mean Square Layer Normalization,” in *Proc. of NeurIPS*, 2019.
- [14] A. Grattafiori *et al.*, “The Llama 3 Herd of Models.” [Online]. Available: <https://arxiv.org/abs/2407.21783>
- [15] A. Yang *et al.*, “Qwen2.5 Technical Report,” *arXiv preprint arXiv:2412.15115*, 2024.
- [16] A. Chowdhery *et al.*, “PaLM: Scaling Language Modeling with Pathways.” 2022.
- [17] D. Hendrycks and K. Gimpel, “Bridging Nonlinearities and Stochastic Regularizers with Gaussian Error Linear Units.” 2016.
- [18] S. Elfving, E. Uchibe, and K. Doya, “Sigmoid-Weighted Linear Units for Neural Network Function Approximation in Reinforcement Learning.” [Online]. Available: <https://arxiv.org/abs/1702.03118>
- [19] Y. N. Dauphin, A. Fan, M. Auli, and D. Grangier, “Language Modeling with Gated Convolutional Networks.” [Online]. Available: <https://arxiv.org/abs/1612.08083>
- [20] N. Shazeer, “GLU Variants Improve Transformer.” 2020.

- [21] J. Su, Y. Lu, S. Pan, B. Wen, and Y. Liu, “RoFormer: Enhanced Transformer with Rotary Position Embedding.” 2021.
- [22] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *Proc. of ICLR*, 2015.
- [23] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *Proc. of ICLR*, 2019.
- [24] T. B. Brown *et al.*, “Language Models are Few-Shot Learners,” in *Proc. of NeurIPS*, 2020.
- [25] J. Kaplan *et al.*, “Scaling Laws for Neural Language Models.” 2020.
- [26] J. Hoffmann *et al.*, “Training Compute-Optimal Large Language Models.” 2022.
- [27] A. Holtzman, J. Buys, L. Du, M. Forbes, and Y. Choi, “The Curious Case of Neural Text Degeneration,” in *Proc. of ICLR*, 2020.
- [28] Y.-H. H. Tsai, S. Bai, M. Yamada, L.-P. Morency, and R. Salakhutdinov, “Transformer Dissection: An Unified Understanding for Transformer’s Attention via the Lens of Kernel,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, K. Inui, J. Jiang, V. Ng, and X. Wan, Eds., Hong Kong, China: Association for Computational Linguistics, Nov. 2019, pp. 4344–4353. doi: [10.18653/v1/D19-1443](https://doi.org/10.18653/v1/D19-1443).
- [29] A. Kazemnejad, I. Padhi, K. Natesan, P. Das, and S. Reddy, “The Impact of Positional Encoding on Length Generalization in Transformers,” in *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. [Online]. Available: <https://openreview.net/forum?id=Dr1l2gcjzl>